KOTLIN

CHEAT SHEETS

Modern Kotlin on the JVM, from null safety to coroutines

`val / var` `Null Safety` `Smart Casts` `Strings` `when` `Ranges` `runCatching` `Extensions`
`Lambdas` `inline / reified` `Scope Functions` `Collections` `Sequences` `Data Classes` `Sealed`
`Objects` `Value Classes` `Delegation` `Generics` `Variance` `kotlinx.serialization`
`Java Interop` `Kotest` `MockK` `Coroutines` `Flow` `StateFlow` `Ktor` `Spring Boot`

95 REFERENCE SHEETS

CONTENTS

INTRODUCTION & SETUP

01 Introduction & History
02 Installation & Tooling
03 Building with Gradle
04 Program Structure
05 Kotlin Targets

OUTPUT & COMMENTS

06 Output & Input
07 Comments & KDoc

VARIABLES & DATA TYPES

08 val & var
09 Integers
10 Floating-Point Numbers
11 Boolean & Char
12 Null Safety Basics
13 Type Conversion
14 Type Checks & Smart Casts
15 const, lateinit & lazy

STRINGS

16 String Basics
17 Building Strings
18 String Methods — Search
19 String Methods — Transform

FUNCTIONS

36 Defining Functions
37 Default & Named Arguments
38 vararg & Spread
39 Extension Functions & Properties
40 Infix, Local & tailrec Functions
41 Lambdas
42 Higher-Order Functions & Function Types
43 inline & reified
44 Scope Functions
45 Function References & SAM Conversions

COLLECTIONS

46 Lists
47 List Operations
48 Sets
49 Maps — Basics
50 Map Operations
51 Arrays
52 Transformations
53 Aggregation & Grouping
54 Sorting & Searching
55 Sequences

FILES & I/O

71 Reading Files
72 Writing Files & Paths
73 JSON with kotlinx.serialization
74 Resources & use

PACKAGES & JAVA INTEROP

75 Packages & Imports
76 Dependencies & Version Catalogs
77 Calling Java from Kotlin
78 Calling Kotlin from Java
79 Standard Library Tour

TESTING & TOOLING

80 kotlin.test & JUnit
81 Kotest & MockK
82 Lint, Docs & Scripting

COROUTINES & FLOW

83 Coroutine Basics
84 async & Structured Concurrency
85 Dispatchers & Context
86 Cancellation & Timeouts
87 Exceptions in Coroutines
88 Flow
89 StateFlow, SharedFlow & Channels

KT

Table of Contents

95 sheets · syntax · examples · notes

INTRODUCTION & SETUP 01 – 05

- 01 Introduction & History
- 02 Installation & Tooling
- 03 Building with Gradle
- 04 Program Structure
- 05 Kotlin Targets

OUTPUT & COMMENTS 06 – 07

- 06 Output & Input
- 07 Comments & KDoc

VARIABLES & DATA TYPES 08 – 15

- 08 val & var
- 09 Integers
- 10 Floating-Point Numbers
- 11 Boolean & Char
- 12 Null Safety Basics
- 13 Type Conversion
- 14 Type Checks & Smart …
- 15 const, lateinit & lazy

STRINGS 16 – 20

- 16 String Basics
- 17 Building Strings
- 18 String Methods — Sear…
- 19 String Methods — Tran…
- 20 Regex & Parsing

OPERATORS 21 – 25

- 21 Arithmetic
- 22 Comparison & Equality
- 23 Logical & Bitwise
- 24 Null-Safety Operators
- 25 Operator Overloading

CONTROL FLOW 26 – 31

- 26 if as an Expression
- 27 when
- 28 for Loops
- 29 while & do-while
- 30 break, continue & Lab…
- 31 Ranges & Progressions

ERROR HANDLING 32 – 35

- 32 try / catch / finally
- 33 throw, Nothing & Preco…
- 34 Custom Exceptions & …
- 35 runCatching & Result

FUNCTIONS 36 – 45

- 36 Defining Functions
- 37 Default & Named Argu…
- 38 vararg & Spread
- 39 Extension Functions & …
- 40 Infix, Local & tailrec Fu…
- 41 Lambdas
- 42 Higher-Order Function…
- 43 inline & reified
- 44 Scope Functions
- 45 Function References &…

COLLECTIONS 46 – 55

- 46 Lists
- 47 List Operations
- 48 Sets
- 49 Maps — Basics
- 50 Map Operations
- 51 Arrays
- 52 Transformations
- 53 Aggregation & Grouping
- 54 Sorting & Searching
- 55 Sequences

CLASSES & OBJECTS 56 – 70

- 56 Classes & Constructors
- 57 Properties
- 58 Visibility Modifiers
- 59 Inheritance
- 60 Abstract Classes & Int…
- 61 Data Classes
- 62 Enum Classes
- 63 Sealed Classes & Inter…
- 64 Objects & Companion …
- 65 Value Classes
- 66 Nested, Inner & Anony…
- 67 Delegation
- 68 Generics
- 69 Variance
- 70 Any, Unit, Nothing & Ty…

FILES & I/O 71 – 74

- 71 Reading Files
- 72 Writing Files & Paths
- 73 JSON with kotlinx.seria…
- 74 Resources & use

PACKAGES & JAVA INTEROP 75 – 79

- 75 Packages & Imports
- 76 Dependencies & Versi…
- 77 Calling Java from Kotlin
- 78 Calling Kotlin from Java
- 79 Standard Library Tour

TESTING & TOOLING 80 – 82

- 80 kotlin.test & JUnit
- 81 Kotest & MockK
- 82 Lint, Docs & Scripting

COROUTINES & FLOW 83 – 89

- 83 Coroutine Basics
- 84 async & Structured Co…
- 85 Dispatchers & Context
- 86 Cancellation & Timeou…
- 87 Exceptions in Coroutin…
- 88 Flow
- 89 StateFlow, SharedFlow …

ADVANCED & ECOSYSTEM 90 – 94

- 90 Annotations & Reflecti…
- 91 Type-Safe Builders & …
- 92 Ktor
- 93 Spring Boot with Kotlin
- 94 Multiplatform & Androi…

QUICK REFERENCE 95

- 95 Quick Reference — Sy…

Introduction & History

A MODERN LANGUAGE FOR THE JVM

Kotlin was created by **JetBrains** (makers of IntelliJ) and reached 1.0 in **2016**. It compiles to JVM bytecode and calls Java libraries directly, while adding **null safety**, concise syntax and coroutines. Google made it the preferred language for **Android** in 2019, and Kotlin 2.0 (2024) shipped the new **K2** compiler.

KEY POINTS

- Source files: `.kt` ; scripts: `.kts`
- Runs anywhere Java runs; also targets JS, Wasm and native
- Statically typed, with strong type inference
- The Kotlin Foundation (JetBrains + Google) guides the language

HELLO, WORLD

```
// Hello.kt
fun main() {
    println("Hello, World")          // Hello, World
}

// With arguments
fun main(args: Array<String>) {
    val name = args.firstOrNull() ?: "World"
    println("Hello, $name")          // Hello, Bob
}

// Build and run with the command-line compiler:
// kotlinc Hello.kt -include-runtime -d hello.jar
// java -jar hello.jar Bob
```

CODE EXPLAINED

- → `main` is a top-level function: no class needed
- → Semicolons are optional and almost never written
- → `$name` is a string template
- → `?:` supplies a default when `firstOrNull()` returns null

JAVA EQUIVALENT

- `public class Hello { public static void main(String[] args) { … } }`
- Java 25 also allows a bare `void main() { … }` (instance main)
- `String name = args.length > 0 ? args[0] : "World";`

IDIOMATIC KOTLIN

- ⇄ wrapping `main` in a class → top-level `fun main()`
- ⇄ `args[0]` with a length check → `args.firstOrNull() ?: default`

TIMELINE

YEAR	MILESTONE
2011	Announced by JetBrains
2016	Kotlin 1.0
2017	Official Android support
2018	Coroutines stable (1.3)
2019	Android "Kotlin-first"
2024	Kotlin 2.0, K2 compiler

WHY KOTLIN

FEATURE	BENEFIT
`Null safety`	Fewer NullPointerExceptions
`Data classes`	No boilerplate DTOs
`Extensions`	Add methods to any type
`Coroutines`	Simple async code
`Java interop`	Use any Java library

⚠ COMMON MISTAKES

- Assuming Kotlin needs its own runtime environment (it runs on the normal JVM)
- Writing Java-style Kotlin full of `!!` and getters
- Forgetting `-include-runtime` when building a runnable jar by hand

✓ TIPS & BEST PRACTICES

- Use IntelliJ's Java-to-Kotlin converter to compare styles
- Try snippets in the Kotlin Playground (play.kotlinlang.org)
- Learn null safety first: it shapes everything else

QUICK SUMMARY

- `fun main()` — entry point
- JVM bytecode — Java interop
- null safety — core feature
- K2 — Kotlin 2.x compiler

Installation & Tooling

GETTING A KOTLIN TOOLCHAIN

Most developers use **IntelliJ IDEA**, which bundles the Kotlin plugin, plus **Gradle** to build. You can also install the command-line compiler `kotlinc` with Homebrew or SDKMAN!. A JDK (17 or newer recommended) is required either way, because Kotlin/JVM compiles to Java bytecode.

KEY POINTS

- IntelliJ Community is free and fully supports Kotlin
- Android Studio is IntelliJ with Android tools
- `kotlinc` alone is fine for scripts and learning
- Project builds should use Gradle, not raw `kotlinc`

INSTALL AND TRY IT

```
# JDK first
brew install openjdk@21
java -version                       # openjdk 21

# Kotlin compiler
brew install kotlin                 # or: sdk install kotlin
kotlinc -version                    # info: kotlinc-jvm 2.x

# Compile and run
kotlinc Hello.kt -include-runtime -d hello.jar
java -jar hello.jar

# Scripts: .main.kts files run directly
kotlin hello.main.kts

# Interactive shell
kotlinc                             # REPL (or Kotlin Notebook in IntelliJ)
>>> listOf(1, 2, 3).sum()
res0: kotlin.Int = 6
```

CODE EXPLAINED

→ Install the JDK before Kotlin

→ `-include-runtime` bundles the Kotlin standard library into the jar

→ `.main.kts` scripts can declare dependencies with annotations

→ IntelliJ users rarely call `kotlinc` directly

JAVA EQUIVALENT

- `brew install openjdk` gives `javac` and `java`
- `java Hello.java` runs a single file without compiling first
- `jshell` is Java's REPL

IDIOMATIC KOTLIN

⇄ raw `kotlinc` for real projects → Gradle with the Kotlin plugin

⇄ old JDK 8 → JDK 17+ (matches modern Spring and Android)

TOOLS

TOOL	USE
IntelliJ IDEA	IDE, refactoring, converter
Android Studio	Android apps
kotlinc	Command-line compiler
Gradle	Builds and dependencies
Kotlin Playground	Browser sandbox
Kotlin Notebook	Interactive notebooks

FILE TYPES

EXTENSION	MEANING
.kt	Kotlin source
.kts	Kotlin script
.main.kts	Runnable script
build.gradle.kts	Gradle Kotlin DSL

⚠ COMMON MISTAKES

- No JDK installed, so `kotlinc` and Gradle fail
- Mixing Kotlin compiler versions between IDE and build
- Building production jars with `kotlinc` instead of Gradle

✓ TIPS & BEST PRACTICES

- Let the Gradle wrapper pin the build's Kotlin version
- Use the Kotlin Playground to share small examples
- Turn on "Kotlin code style" in IntelliJ for consistent formatting

QUICK SUMMARY

- IntelliJ — main IDE
- `kotlinc` — CLI compiler
- JDK 17+ — required
- `.kts` — scripts

Building with Gradle

A STANDARD KOTLIN PROJECT

Kotlin projects are built with **Gradle**, usually with the **Kotlin DSL** (`build.gradle.kts`). The `kotlin("jvm")` plugin compiles Kotlin, `jvmToolchain` pins the JDK, and the `application` plugin adds a `run` task. Source goes in `src/main/kotlin` and tests in `src/test/kotlin`.

KEY POINTS

- `gradle init` can generate a Kotlin application
- The wrapper (`./gradlew`) pins the Gradle version
- Java and Kotlin can live in the same module
- Maven works too with `kotlin-maven-plugin`

BUILD.GRADLE.KTS

```
plugins {
    kotlin("jvm") version "2.2.0"
    application
}

repositories {
    mavenCentral()
}

dependencies {
    implementation("org.jetbrains.kotlinx:kotlinx-coroutines-core:1.10.2")
    testImplementation(kotlin("test"))
}

kotlin {
    jvmToolchain(21)
}

application {
    mainClass.set("com.example.AppKt")    // file App.kt
}

tasks.test {
    useJUnitPlatform()
}

// ./gradlew run      ./gradlew build      ./gradlew test
```

CODE EXPLAINED

→ `kotlin("jvm")` applies the Kotlin JVM plugin at that version

→ `mainClass` is `AppKt` because top-level functions in `App.kt` compile to class `AppKt`

→ `kotlin("test")` adds kotlin.test wired to JUnit

→ Version numbers are examples; use current releases

JAVA EQUIVALENT

- `plugins { id("java"); application }` in Gradle
- Maven: `pom.xml` with `maven-compiler-plugin`
- `java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }`

IDIOMATIC KOTLIN

⇄ Groovy `build.gradle` → Kotlin DSL `build.gradle.kts`

⇄ hard-coded versions in many files → a version catalog (sheet 76)

PROJECT LAYOUT

PATH	HOLDS
`build.gradle.kts`	Build script
`settings.gradle.kts`	Project name, modules
`src/main/kotlin`	Kotlin source
`src/main/java`	Java source (optional)
`src/test/kotlin`	Tests
`gradle/libs.versions.toml`	Version catalog

TASKS

COMMAND	DOES
`./gradlew build`	Compile, test, package
`./gradlew run`	Run main
`./gradlew test`	Run tests
`./gradlew clean`	Delete build/
`./gradlew dependencies`	Show dependency tree

⚠ COMMON MISTAKES

- `mainClass` set to `App` instead of `AppKt` for a top-level main
- Kotlin plugin and kotlinx library versions that do not match
- Forgetting `useJUnitPlatform()`, so tests never run

✓ TIPS & BEST PRACTICES

- Start with `gradle init --type kotlin-application`
- Pin the JDK with `jvmToolchain` rather than relying on your machine
- See the Gradle set for multi-module builds

QUICK SUMMARY

- `kotlin("jvm")` — plugin
- `jvmToolchain` — JDK version
- `AppKt` — main class name
- `./gradlew run` — run it

Program Structure

HOW A KOTLIN FILE IS ORGANIZED

A Kotlin file starts with an optional `package` line and `imports`, then any mix of **top-level** functions, properties and classes. Unlike Java, functions do not need a class, one file can hold many classes, and the file name does not have to match a class name.

KEY POINTS

- Package names need not match folders (but should)
- Top-level declarations are public by default
- Blocks use braces; statements end at the newline
- Type comes after the name: `val count: Int`

ONE FILE, SEVERAL DECLARATIONS

```
package com.example.shop

import java.time.LocalDate

const val TAX_RATE = 0.06                    // top-level constant

data class Item(val name: String, val price: Double)

fun total(items: List<Item>): Double =       // top-level function
    items.sumOf { it.price } * (1 + TAX_RATE)

class Cart {
    private val items = mutableListOf<Item>()
    fun add(item: Item) { items += item }
    fun checkout() = total(items)
}

fun main() {
    val cart = Cart()
    cart.add(Item("Book", 20.0))
    println("${LocalDate.now()}: ${cart.checkout()}")  // 2026-11-12: 21.2
}
```

CODE EXPLAINED

- → `TAX_RATE`, `total` and `main` live outside any class
- → `data class` and `class` share the file with functions
- → `=` after a function signature makes it an expression body
- → `items += item` calls `plusAssign` on the mutable list

JAVA EQUIVALENT

- Everything must be inside a class; one public class per file
- Constants: `static final double TAX_RATE = 0.06;`
- Helper functions become `static` methods on a utility class

IDIOMATIC KOTLIN

- ⇄ `Utils` classes full of static methods → top-level functions
- ⇄ one class per file always → group small related types in one file

ORDER IN A FILE

PART	EXAMPLE
`File annotations`	@file:JvmName("Shop")
`package`	package com.example
`imports`	import java.time.*
`declarations`	fun, val, class, object

KOTLIN VS JAVA SYNTAX

KOTLIN	JAVA
`val x: Int = 1`	final int x = 1;
`fun f(a: Int): Int`	int f(int a)
`no ';'`	; required
`no 'new'`	new Cart()
`top-level fun`	static method

⚠ COMMON MISTAKES

- Putting type before name like Java (`Int x`)
- Expecting the file name to force a class name
- Making everything top-level and public instead of using visibility

✓ TIPS & BEST PRACTICES

- Keep package and folder structure aligned for tooling
- Use `private` top-level declarations for file-local helpers
- Use expression bodies for one-line functions

QUICK SUMMARY

- top-level — no class needed
- name: Type — type after name
- many classes per file — OK
- `package` + imports first

Kotlin Targets

ONE LANGUAGE, MANY PLATFORMS

Kotlin compiles to several **targets**: JVM bytecode (servers, desktop, Android), JavaScript and WebAssembly for browsers, and native binaries through LLVM for iOS, macOS, Linux and Windows. **Kotlin Multiplatform** (KMP) shares common code across targets while each platform keeps its own UI and APIs.

KEY POINTS

- This set focuses on Kotlin/JVM
- Android uses Kotlin/JVM with the Android SDK
- KMP shares business logic; Compose Multiplatform shares UI
- The standard library works on every target; `java.*` only on JVM

MULTIPLATFORM BUILD SKETCH

```
// build.gradle.kts of a shared module
plugins {
    kotlin("multiplatform") version "2.2.0"
}

kotlin {
    jvm()                       // servers / desktop
    js { browser() }            // web
    iosArm64()                  // iPhone (native)
    linuxX64()                  // native binary

    sourceSets {
        commonMain.dependencies {
            implementation("org.jetbrains.kotlinx:kotlinx-datetime:0.6.2")
        }
        jvmMain.dependencies {
            implementation("org.postgresql:postgresql:42.7.4")
        }
    }
}
```

CODE EXPLAINED

- → Each target block adds a platform to compile for
- → `commonMain` code compiles for every target
- → `jvmMain` can use Java libraries; common code cannot
- → Platform-specific APIs are bridged with `expect` / `actual` (sheet 94)

JAVA EQUIVALENT

- Java targets the JVM only (plus GraalVM native images)
- GraalVM `native-image` builds native executables from Java
- Android also runs Java, but new Android APIs are Kotlin-first

IDIOMATIC KOTLIN

- ⇄ `java.*` imports in shared code → kotlinx libraries (datetime, serialization)
- ⇄ sharing UI too early → share logic first, UI later

TARGETS

TARGET	OUTPUT	USED FOR
Kotlin/JVM	Bytecode	Servers, desktop, Android
Kotlin/JS	JavaScript	Web front ends
Kotlin/Wasm	WebAssembly	Fast web apps
Kotlin/Native	Native binary	iOS, macOS, CLI
Multiplatform	All of the above	Shared logic

⚠ COMMON MISTAKES

- Using Java classes in `commonMain` (they do not exist on other targets)
- Assuming every library supports every target
- Expecting Kotlin/JS to accept any npm package without typed wrappers

✓ TIPS & BEST PRACTICES

- Start on Kotlin/JVM; add targets once the basics are solid
- Check a library's supported platforms on klibs.io
- See sheet 94 for `expect` / `actual`

QUICK SUMMARY

- JVM — main target
- JS / Wasm — browser
- Native — iOS, CLI
- KMP — shared common code

Output & Input

PRINTING AND READING LINES

`println` writes a value and a newline; `print` skips the newline. **String templates** put values straight into strings with `$name` or `${expression}`. `readln()` reads a line from standard input and throws at end of input, while `readlnOrNull()` returns null instead.

KEY POINTS

- `println` accepts any type and calls `toString()`
- Use `${}` for anything more than a simple name
- Write errors with `System.err.println`
- `readln()` was added in Kotlin 1.6, replacing `readLine()!!`

OUTPUT AND INPUT

```
val name = "Ada"
val items = listOf(3, 5, 8)

println("Hello")                        // Hello
print("No newline ")
println("then newline")                 // No newline then newline
println("Hi, $name")                    // Hi, Ada
println("Total: ${items.sum()}")        // Total: 16
println("Name has ${name.length} chars")
println("Price: %.2f".format(19.5))     // Price: 19.50
println(items)                          // [3, 5, 8]
System.err.println("Warning: low stock")

print("Your age? ")
val age = readln().toInt()              // throws if not a number
val line = readlnOrNull()               // null at end of input
val safe = readln().toIntOrNull() ?: 0 // 0 if not a number
```

CODE EXPLAINED

→ `$name` inserts a variable; `${…}` inserts any expression

→ `"%.2f".format(x)` uses Java's `String.format` rules

→ Collections print in readable form

→ `toIntOrNull()` avoids a crash on bad input

JAVA EQUIVALENT

- `System.out.println("Hi, " + name);`
- `String.format("Price: %.2f", 19.5)` or `"…".formatted(19.5)`
- `new Scanner(System.in).nextLine()` / `BufferedReader.readLine()`

IDIOMATIC KOTLIN

⇄ `"Hi, " + name` → `"Hi, $name"`

⇄ `readLine()!!` → `readln()`

OUTPUT

CALL	DOES
`println(x)`	Print + newline
`print(x)`	Print, no newline
`System.err.println(x)`	Standard error
`"%.2f".format(x)`	Formatted string
`"$a ${b.c}"`	Template

INPUT

CALL	AT END OF INPUT
`readln()`	Throws exception
`readlnOrNull()`	Returns null
`readln().toInt()`	Throws on bad text
`readln().toIntOrNull()`	null on bad text

⚠ COMMON MISTAKES

- `"$user.name"` prints the user object followed by ".name" (use `${user.name}`)
- `readln().toInt()` crashing on empty or bad input
- Using templates to build SQL (injection risk)

✓ TIPS & BEST PRACTICES

- Use a logger (SLF4J, kotlin-logging) instead of println in apps
- Use raw strings `"""…"""` for multi-line output
- Prefer `toIntOrNull()` for any user input

QUICK SUMMARY

- `println` / `print` — output
- `$x` / `${expr}` — templates
- `readln()` — read a line
- `toIntOrNull()` — safe parse

Comments & KDoc

COMMENTS AND API DOCS

Kotlin uses // for line comments and /* */ for block comments, which can **nest** (unlike Java). Documentation comments /** */ use **KDoc**, a Markdown-based format with block tags like @param. **Dokka** turns KDoc into HTML documentation, and IntelliJ shows it on hover.

KEY POINTS

- Block comments nest: `/* outer /* inner */ still comment */`
- KDoc uses Markdown, not HTML
- Link to symbols with `[ClassName]` or `[function]`
- The first paragraph is the summary

KDOC IN PRACTICE

```
/**
 * Calculates the monthly payment on a loan.
 *
 * Uses the standard amortization formula. See [Loan] for inputs.
 *
 * @param principal amount borrowed, must be positive
 * @param annualRate yearly rate as a decimal, e.g. `0.05`
 * @param months number of monthly payments
 * @return the payment amount, rounded to cents
 * @throws IllegalArgumentException if [principal] is not positive
 * @sample com.example.LoanSamples.basic
 */
fun monthlyPayment(principal: Double, annualRate: Double, months: Int): Double {
    require(principal > 0) { "principal must be positive" }
    val r = annualRate / 12            // monthly rate
    /* Formula:
       P * r / (1 - (1 + r)^-n)  /* nested comment is fine */ */
    return principal * r / (1 - Math.pow(1 + r, -months.toDouble()))
}
```

CODE EXPLAINED

- → `[Loan]` and `[principal]` become links in generated docs
- → `@sample` embeds a function's code as an example
- → The nested `/* */` inside the block comment is legal Kotlin
- → `require` throws IllegalArgumentException with the message

JAVA EQUIVALENT

- Javadoc `/** … */` with HTML and `{@link Loan}`
- `@param`, `@return`, `@throws` work the same
- Java block comments do not nest

IDIOMATIC KOTLIN

- ⇄ HTML tags in KDoc → Markdown (`**bold**`, backticks)
- ⇄ comments that repeat the code → explain the why

KDOC TAGS

TAG	DOCUMENTS
`@param name`	A parameter
`@return`	Return value
`@throws / @exception`	Exception thrown
`@property name`	Class property
`@constructor`	Primary constructor
`@see`	Related symbol
`@sample`	Example code
`@since`	Version added

⚠ COMMON MISTAKES

- Javadoc `{@link X}` syntax in KDoc (use `[X]`)
- Documenting every private helper and nothing public
- Leaving commented-out code instead of deleting it

✓ TIPS & BEST PRACTICES

- Run Dokka (`./gradlew dokkaHtml` or `dokkaGenerate`) to publish docs
- Use `@property` on data class docs for constructor properties
- Write the first sentence as a one-line summary

QUICK SUMMARY

- `//` `/* */` — comments (nestable)
- `/** */` — KDoc
- `[Name]` — link
- Dokka — HTML docs

val & var

DECLARING VARIABLES

`val` declares a **read-only** variable that is assigned once; `var` declares one you can reassign. Kotlin **infers** the type from the initial value, so annotations are optional. Prefer `val` by default and switch to `var` only when the value really changes.

KEY POINTS

- `val` is like Java `final`; the object it points to may still change
- Types are fixed once inferred: a `var x = 1` stays `Int`
- Locals must be initialized before use
- Top-level and class properties can be `val` or `var` too

VAL VS VAR

```
val name = "Ada"                  // String, inferred
var count = 0                     // Int, inferred
val price: Double = 9.99          // explicit type
val total: Int                    // declare now...
total = 42                        // ...assign once later

count += 1                        // OK: var
// name = "Bob"                   // error: val cannot be reassigned
// count = "ten"                  // error: type mismatch (Int)

val list = mutableListOf(1, 2)    // the reference is read-only
list.add(3)                       // but the list itself can change
println(list)                     // [1, 2, 3]

var maybe: String? = null         // nullable needs the type
maybe = "now set"
```

CODE EXPLAINED

→ `total` is a `val` assigned exactly once after declaration

→ Changing `count` to a String fails: its type is `Int`

→ `val list` cannot point elsewhere, but `add` still mutates the list

→ Use `listOf` (read-only list) when contents should not change

JAVA EQUIVALENT

- `final String name = "Ada";` or `var name = "Ada";` (Java 10+)
- `int count = 0;` for mutable locals
- `final` on a list reference still allows `list.add(…)`

IDIOMATIC KOTLIN

⇄ `var` everywhere → `val` by default

⇄ `val list = mutableListOf()` exposed publicly → expose `List` (read-only)

DECLARATIONS

FORM	MEANING
`val x = 1`	Read-only, inferred
`var x = 1`	Mutable, inferred
`val x: Long = 1`	Explicit type
`val x: Int`	Assign later, once
`const val X = 1`	Compile-time constant
`lateinit var x: T`	Set later (sheet 15)

⚠ COMMON MISTAKES

- Thinking `val` makes the object immutable
- Using `var` for values that never change, hiding real state changes
- Declaring a nullable variable without the `?` type

✓ TIPS & BEST PRACTICES

- Let IntelliJ suggest "Change to val" warnings
- Add explicit types on public API properties
- Keep `var`s as local as possible

QUICK SUMMARY

- `val` — assign once
- `var` — reassignable
- inference — type from value
- read-only ≠ immutable

Integers

WHOLE-NUMBER TYPES

Kotlin has `Byte`, `Short`, `Int` and `Long`, plus unsigned versions (`UByte`, `UShort`, `UInt`, `ULong`). Integer literals default to `Int`, or `Long` if they are too big or end in `L`. All numbers are objects in the type system, but compile to Java primitives where possible.

KEY POINTS

- Underscores improve readability: `1_000_000`
- Hex `0xFF` and binary `0b1010`; no octal literals
- Overflow wraps silently, like Java
- Use `Math.addExact` or `BigInteger` when overflow matters

UNDER THE JVM

- `val x: Int = 5` compiles to a primitive `int`
- `Int?` and `List<Int>` use boxed `java.lang.Integer`
- `UInt` is an inline value class over `Int` (sheet 65)

INTEGER TYPES AND LITERALS

```
val a = 42                    // Int
val big = 3_000_000_000       // Long (too big for Int)
val l = 42L                   // Long
val b: Byte = 127
val s: Short = 30_000
val hex = 0xFF                // 255
val bin = 0b1010              // 10
val u = 42u                   // UInt
val ul = 18_000_000_000uL     // ULong

println(Int.MAX_VALUE)        // 2147483647
println(Int.MAX_VALUE + 1)    // -2147483648 (wraps)
println(Math.addExact(Int.MAX_VALUE, 1))  // ArithmeticException
println(7 / 2)                // 3 (integer division)
println(7 % 2)                // 1
println(255.toString(16))     // ff
println("ff".toInt(16))       // 255
val boxed: Int? = 42          // java.lang.Integer
```

CODE EXPLAINED

- → A literal larger than Int.MAX_VALUE becomes `Long` automatically
- → `u` and `uL` suffixes make unsigned literals
- → Integer division drops the remainder
- → `toString(radix)` and `toInt(radix)` convert bases

JAVA EQUIVALENT

- `int`, `long`, `short`, `byte` primitives; `Integer` wrappers
- `42L`, `0xFF`, `0b1010`, and octal `017` (no octal in Kotlin)
- No unsigned types; use `Integer.toUnsignedString` helpers

IDIOMATIC KOTLIN

- ⇄ `Integer.parseInt(s)` → `s.toInt()` / `s.toIntOrNull()`
- ⇄ magic big numbers → underscores `1_000_000`

TYPES

TYPE	BITS	RANGE
Byte	8	-128..127
Short	16	-32,768..32,767
Int	32	±2.1 billion
Long	64	$\pm 9.2 \times 10^{18}$
UInt	32	0..4.29 billion
ULong	64	$0..1.8 \times 10^{19}$

⚠ COMMON MISTAKES

- Expecting `017` to be octal (Kotlin has no octal literals)
- Silent overflow in totals that should use `Long`
- Comparing boxed `Int?` values with `===` (identity, not value)

✓ TIPS & BEST PRACTICES

- Use `Long` for IDs, counts of records and millisecond times
- Use `Math.*Exact` when overflow must be an error
- Use unsigned types only for bit-level or protocol work

QUICK SUMMARY

- `Int` default, `Long` with `L`
- `1_000` — readable literals
- `u` / `uL` — unsigned
- overflow wraps silently

Floating-Point Numbers

DECIMALS AND THEIR LIMITS

`Double` (64-bit) is the default for decimal literals; `Float` (32-bit) needs an `f` suffix. Both are binary floating point, so values like 0.1 are approximate. Use `kotlin.math` for math functions and `BigDecimal` for money and other exact decimal work.

KEY POINTS

- `1.5` is Double; `1.5f` is Float
- No implicit widening: `val d: Double = 1` is an error
- `0.1 + 0.2 == 0.3` is false
- Division by zero gives Infinity or NaN, not an exception

DOUBLES, ROUNDING AND BIGDECIMAL

```
import kotlin.math.*
import java.math.BigDecimal
import java.math.RoundingMode

val d = 3.14                        // Double
val f = 2.5f                        // Float
val e = 1.2e3                       // 1200.0

println(0.1 + 0.2)                  // 0.30000000000000004
println(abs(0.1 + 0.2 - 0.3) < 1e-9)   // true
println(1.0 / 0)                    // Infinity
println(0.0 / 0)                    // NaN
println(Double.NaN == Double.NaN)       // false
println(2.567.roundToInt())         // 3
println(floor(2.7))                 // 2.0
println(sqrt(16.0))                 // 4.0
println(2.0.pow(10))                // 1024.0
println("%.2f".format(2.567))       // 2.57

val price = BigDecimal("19.99")
val tax = (price * BigDecimal("0.06")).setScale(2, RoundingMode.HALF_UP)
println(tax)                        // 1.20
```

CODE EXPLAINED

- → Compare doubles with a tolerance, not `==`
- → `roundToInt()` rounds halves up: 2.5 → 3, -2.5 → -2
- → `BigDecimal("19.99")` from a String keeps the exact value
- → Kotlin overloads `*` for BigDecimal

JAVA EQUIVALENT

- ☕ `double` / `float`; `Math.round`, `Math.sqrt`, `Math.pow`
- ☕ `new BigDecimal("19.99").multiply(rate).setScale(2, HALF_UP)`
- ☕ `Double.isNaN(x)` to test for NaN

IDIOMATIC KOTLIN

- ⇄ `Math.sqrt(x)` → `sqrt(x)` from kotlin.math
- ⇄ `BigDecimal(19.99)` (from a double) → `BigDecimal("19.99")`

KOTLIN.MATH

FUNCTION	RESULT
`abs(-2.5)`	2.5
`round(2.5)`	2.0 (half even)
`ceil / floor`	Up / down
`sqrt(x)`	Square root
`x.pow(n)`	Power
`ln(x) / log10(x)`	Logs
`PI / E`	Constants

SPECIAL VALUES

VALUE	TEST
`NaN`	x.isNaN()
`Infinity`	x.isInfinite()
`Finite`	x.isFinite()
`Max`	Double.MAX_VALUE

⚠ COMMON MISTAKES

- Money in Double, producing rounding errors
- Expecting `round(2.5)` (kotlin.math) to give 3.0 (it rounds half to even)
- Assigning an Int literal to a Double variable without `.0`

✓ TIPS & BEST PRACTICES

- Use `BigDecimal` with string constructors for currency
- Use `roundToInt()` / `roundToLong()` for display rounding
- Use `compareTo` for BigDecimal equality when scale differs

QUICK SUMMARY

- `Double` default, `Float` with `f`
- compare with tolerance
- `kotlin.math` — functions
- `BigDecimal` — exact money

Boolean & Char

TRUE/FALSE AND SINGLE CHARACTERS

Boolean holds true or false and works with &&, || and !. Char holds one UTF-16 character in single quotes. Unlike Java, a Char is **not** a number: use code to get its numeric value and digitToInt() to turn '7' into 7.

KEY POINTS

- Char literals: 'A', '\n', '\u00E9'
- No char + int arithmetic that gives an Int silently
- Char has many tests: isWhitespace(), isUpperCase()
- Emoji often need two Chars (surrogate pair)

BOOLEANS AND CHARS

```
val isActive = true
val hasStock = false
println(isActive && !hasStock)   // true
println(isActive.and(hasStock))  // false (eager, no short-circuit)

val c = 'A'
val tab = '\t'
val e = '\u00E9'                 // é
println(c.code)                  // 65
println(Char(66))                // B
println(c + 1)                   // B (Char + Int = Char)
println('7'.digitToInt())        // 7
println('7'.isDigit())           // true
println('x'.isLetter())          // true
println('a'.uppercaseChar())     // A
println(c in 'A'..'Z')           // true
println("Kotlin"[0])             // K
println("Kotlin".count { it.isUpperCase() })   // 1
```

CODE EXPLAINED

- → c.code replaces Java's cast (int) c
- → Char + Int moves along the alphabet and stays a Char
- → digitToInt() is the safe way to read a digit character
- → 'A'..'Z' is a CharRange

JAVA EQUIVALENT

- boolean, char primitives
- (int) c gets the code; (char) 66 gets B
- Character.isDigit(c), Character.getNumericValue('7')

IDIOMATIC KOTLIN

- ⇄ c.toInt() (deprecated) → c.code
- ⇄ c - '0' to read a digit → c.digitToInt()

CHAR FUNCTIONS

FUNCTION	EXAMPLE
isDigit()	'7' → true
isLetter()	'x' → true
isWhitespace()	' ' → true
uppercaseChar()	'a' → 'A'
digitToInt()	'7' → 7
code	'A' → 65

ESCAPES

ESCAPE	CHAR
\n	Newline
\t	Tab
\\	Backslash
\'	Quote
\$	Dollar sign
\uXXXX	Unicode

⚠ COMMON MISTAKES

- Using c.toInt() and getting a deprecation warning
- Comparing a Char with an Int (c == 65 does not compile)
- Assuming one Char = one visible character for emoji

✓ TIPS & BEST PRACTICES

- Use code / Char(n) for conversions
- Use '0'..'9' ranges in when for character classes
- Use codePoints() for full Unicode text processing

QUICK SUMMARY

- Boolean — true / false
- Char — not a number
- code / Char(n) — convert
- digitToInt() — digit value

Null Safety Basics

NULLS IN THE TYPE SYSTEM

In Kotlin, a regular type like `String` can **never** hold null. To allow null you write `String?`, and the compiler then forces you to handle it before use: with a check, the safe call `?.`, the Elvis operator `?:`, or `!!` if you are certain it is not null.

KEY POINTS

- `String` is a subtype of `String?`
- After `if (x != null)`, `x` is smart cast to non-null
- `!!` throws NullPointerException when the value is null
- Java values arrive as platform types `String!` (sheet 77)

UNDER THE JVM

- Non-null parameters get `Intrinsics.checkNotNullParameter` calls at public entry points
- Nullable and non-null `String` are both `java.lang.String` in bytecode
- `@Nullable` / `@NotNull` annotations are emitted for Java callers

HANDLING NULLABLE VALUES

```
var name: String = "Ada"
// name = null                    // error: null not allowed

var nick: String? = null          // nullable
println(nick?.length)             // null (safe call)
println(nick?.length ?: 0)        // 0 (Elvis default)

if (nick != null) {
    println(nick.length)          // smart cast: nick is String here
}

nick = "Lovelace"
println(nick!!.length)            // 8, but throws if nick is null

fun greet(user: String?) {
    val shown = user ?: return     // leave if null
    println("Hi, $shown")
}

val lengths = listOf("a", null, "abc").map { it?.length }
println(lengths)                  // [1, null, 3]
```

CODE EXPLAINED

- → `?.` returns null instead of calling on a null receiver
- → `?:` gives a value (or `return` / `throw`) when the left side is null
- → Smart casts work on local `val`s and unchanged local `var`s
- → `!!` is a promise to the compiler; a broken promise is an NPE

JAVA EQUIVALENT

- Any reference may be null; checks happen only at runtime
- `Optional<String>` for optional return values
- `Objects.requireNonNull(x)` ≈ `x!!`

IDIOMATIC KOTLIN

- ⇄ `if (x != null) x.foo() else null` → `x?.foo()`
- ⇄ `x!!` sprinkled everywhere → `?.`, `?:`, or a non-null type

TOOLS

SYNTAX	MEANING
`T?`	Nullable type
`x?.f()`	Call if not null
`x ?: y`	y if x is null
`x!!`	Assert not null (NPE)
`x?.let { }`	Run block if not null
`as?`	Safe cast (null on failure)

WHERE NPES STILL COME FROM

CAUSE	EXAMPLE
`!!`	x!!.length
`Java interop`	Platform types
`lateinit`	Read before set
`Init order`	Open member in constructor

⚠ COMMON MISTAKES

- Using `!!` to silence the compiler instead of handling null
- Making every type nullable "just in case"
- Trusting Java return values that can actually be null

✓ TIPS & BEST PRACTICES

- Make types non-null unless null has a real meaning
- Use `?: return` / `?: throw` for early exits
- See sheet 24 for all null-safety operators

QUICK SUMMARY

- `T?` — may be null
- `?.` — safe call
- `?:` — default
- `!!` — assert (avoid)

Type Conversion

no implicit widening · `toInt()` / `toLong()` / `toDouble()` · `toIntOrNull()` · `toString()` · `toBigDecimal()`

CONVERTING BETWEEN TYPES

Kotlin never widens numbers automatically: an `Int` will not become a `Long` just by assignment. Every numeric type has explicit conversion functions such as `toLong()` and `toDouble()`. Strings parse with `toInt()` (throws) or `toIntOrNull()` (returns null).

KEY POINTS

- Arithmetic with mixed types widens the result: `Int + Long` is `Long`
- Narrowing conversions truncate or wrap
- `toInt()` on a Double drops the fraction
- Numbers convert to String with `toString()` or templates

EXPLICIT CONVERSIONS

```
val i: Int = 42
// val l: Long = i                // error: type mismatch
val l: Long = i.toLong()          // 42
val d: Double = i.toDouble()      // 42.0
val mixed = i + 10L               // Long (widened in arithmetic)

println(3.99.toInt())             // 3 (truncated)
println(300.toByte())             // 44 (wrapped)
println((-1).toUInt())            // 4294967295

println("42".toInt())             // 42
println("4x2".toIntOrNull())      // null
println("3.5".toDouble())         // 3.5
println("ff".toInt(16))           // 255
println("true".toBoolean())       // true
println("19.99".toBigDecimal())   // 19.99
println(42.toString())            // "42"
println(42.toString().padStart(5, '0'))   // 00042
```

CODE EXPLAINED

→ `i.toLong()` is required because there is no implicit widening

→ Narrowing to Byte keeps only the low 8 bits

→ `toIntOrNull()` is the safe choice for user input

→ `toBigDecimal()` exists for String, Int, Long and Double

JAVA EQUIVALENT

- `long l = i;` widens automatically in Java
- `Integer.parseInt("42")` throws `NumberFormatException`
- `String.valueOf(42)` or `Integer.toString(42)`

IDIOMATIC KOTLIN

⇄ `Integer.parseInt(s)` → `s.toInt()` / `s.toIntOrNull()`

⇄ `try { s.toInt() } catch …` → `s.toIntOrNull() ?: default`

NUMBER CONVERSIONS

CALL	NOTES
`toByte()` / `toShort()`	May wrap
`toInt()` / `toLong()`	Truncate decimals
`toFloat()` / `toDouble()`	May lose precision
`toChar()`	Deprecated on most types
`toUInt()` / `toULong()`	Unsigned

STRING PARSING

CALL	BAD INPUT
`toInt()` / `toLong()`	NumberFormatException
`toIntOrNull()`	null
`toDouble()`	NumberFormatException
`toDoubleOrNull()`	null
`toBoolean()`	false unless "true"
`toBooleanStrict()`	Exception unless true/false

⚠ COMMON MISTAKES

- Expecting `val l: Long = intValue` to compile
- Parsing user input with `toInt()` and crashing
- Assuming `toInt()` rounds (it truncates)

✓ TIPS & BEST PRACTICES

- Use `roundToInt()` when you mean rounding
- Use the `OrNull` parsers plus `?:` for defaults
- Use `toBooleanStrict()` for config flags

QUICK SUMMARY

- no implicit widening
- `toX()` — explicit conversion
- `toIntOrNull()` — safe parse
- narrowing — truncates / wraps

Sheet 14

Type Checks & Smart Casts

is / !is · smart casts · as / as? · Any · smart casts with when · stable vals only

CHECKING AND CASTING TYPES

is checks a value's type, and after a successful check Kotlin **smart casts** the value, so you can use it as that type with no explicit cast. as casts explicitly and throws on failure; as? returns null instead. Any is the root of all non-null types.

KEY POINTS

- Smart casts work in `if`, `when`, `&&` and `||` branches
- They need a value the compiler knows cannot change
- Generic types are erased: `is List<String>` is not allowed
- `Any?` holds anything, including null

IS, SMART CASTS AND AS?

```
fun describe(x: Any): String = when (x) {
    is Int    -> "Int doubled: ${x * 2}"         // x is Int here
    is String -> "String of length ${x.length}"  // x is String
    is List<*> -> "List with ${x.size} items"
    else      -> "Unknown: $x"
}

println(describe(21))              // Int doubled: 42
println(describe("hi"))            // String of length 2

val obj: Any = "Kotlin"
if (obj is String && obj.length > 3) {   // smart cast after &&
    println(obj.uppercase())       // KOTLIN
}
if (obj !is Int) println("not an Int")

val s = obj as String              // explicit cast, OK
val n = obj as? Int                // null, no exception
// val bad = obj as Int            // ClassCastException

class Box(var item: Any)
val box = Box("x")
// if (box.item is String) box.item.length  // error: var property may change
```

CODE EXPLAINED

- → Inside each `when` branch, `x` already has the checked type
- → Smart casts also flow through `&&`
- → `as?` is the safe cast; combine it with `?:`
- → Mutable properties do not smart cast: copy to a local `val` first

JAVA EQUIVALENT

- `if (x instanceof String s) { s.length(); }` (pattern matching, Java 16+)
- `switch (x) { case Integer i -> … }` (Java 21)
- `(String) obj` throws `ClassCastException` like `as`

IDIOMATIC KOTLIN

- ⇄ `(x as String).length` after an `is` check → rely on the smart cast
- ⇄ `obj as Int` that may fail → `obj as? Int ?: default`

OPERATORS

SYNTAX	MEANING
`x is T`	Type check
`x !is T`	Negated check
`x as T`	Cast or throw
`x as? T`	Cast or null
`x is List<*>`	Check with star projection

SMART CAST WORKS ON

VALUE	SMART CAST?
`Local val`	Yes
`Local var (not changed)`	Yes
`val property, same module`	Yes
`var property`	No
`open / custom getter`	No

⚠ COMMON MISTAKES

- Expecting smart casts on `var` properties
- Checking `is List<String>` (erased; use `List<*>`)
- Using `as` where failure is possible

✓ TIPS & BEST PRACTICES

- Use `when` with `is` branches instead of if-else chains
- Copy a property to a local `val` to enable smart casts
- Prefer sealed types (sheet 63) over open-ended `Any` checks

QUICK SUMMARY

- `is` — check + smart cast
- `as` — cast or throw
- `as?` — cast or null
- local vals — smart cast safely

const, lateinit & lazy

CONSTANTS AND DELAYED INITIALIZATION

`const val` makes a **compile-time constant** that is inlined at use sites. `lateinit var` declares a non-null property that is set later, such as by dependency injection or a test setup. `by lazy` computes a value the first time it is read and caches it.

KEY POINTS

- `const` works only for primitives and String, at top level or in objects
- `lateinit` needs a `var` of a non-null, non-primitive type
- Reading an unset `lateinit` throws UninitializedPropertyAccessException
- `lazy` is thread-safe by default

UNDER THE JVM

- `const val` values are copied into calling code as literals
- `lateinit` compiles to a nullable field plus a check on every read
- `by lazy` creates a hidden `Lazy` object and a `getValue` call

THREE WAYS TO INITIALIZE

```
const val MAX_USERS = 100                  // top-level constant

class Config {
    companion object {
        const val VERSION = "2.1"          // Config.VERSION
    }
}

class UserService {
    lateinit var repo: UserRepository      // set by DI or tests

    fun isReady() = ::repo.isInitialized

    val report: String by lazy {           // computed on first read
        println("building report...")
        repo.findAll().joinToString()
    }
}

fun main() {
    val svc = UserService()
    println(svc.isReady())                 // false
    // svc.repo.findAll()  // UninitializedPropertyAccessException
    svc.repo = UserRepository()
    println(svc.report)                    // building report... then data
    println(svc.report)                    // cached, no second build
}
```

CODE EXPLAINED

- → `MAX_USERS` and `VERSION` are inlined wherever they are used
- → `::repo.isInitialized` checks a lateinit property safely
- → The lazy block runs once; later reads return the cached value
- → Lazy values must be `val`

JAVA EQUIVALENT

- `public static final int MAX_USERS = 100;`
- Fields set later are just nullable fields with no check
- Lazy init: double-checked locking or a holder class

IDIOMATIC KOTLIN

- ⇄ nullable `var repo: Repo? = null` + `!!` → `lateinit var repo: Repo`
- ⇄ expensive work in `init` → `by lazy`

COMPARISON

FEATURE	CONST VAL	LATEINIT VAR	BY LAZY
Mutable	No	Yes	No
When set	Compile time	Any time later	First read
Types	Primitives, String	Non-null objects	Any
Where	Top level, object	Class / top level	Properties
Unset read	—	Exception	Computes it

⚠ COMMON MISTAKES

- `lateinit` on an `Int` (not allowed; use `Delegates.notNull()`)
- Changing a `const` in a library without recompiling users (old value stays inlined)
- Using `lazy` for values that depend on state that changes

✓ TIPS & BEST PRACTICES

- Use `lateinit` for DI-injected and test-setup fields
- Use `lazy(LazyThreadSafetyMode.NONE)` on single-threaded hot paths
- Group related constants in an `object`

QUICK SUMMARY

- `const val` — compile-time
- `lateinit var` — set later
- `by lazy` — first read
- `::x.isInitialized` — check

String Basics

TEXT IN KOTLIN

Kotlin strings are immutable `java.lang.String` objects with extra functions. **Templates** insert values with `$name` or `${expression}`, and **raw strings** in triple quotes keep line breaks and backslashes as written, which suits JSON, SQL and regex. == compares contents.

KEY POINTS

- `s[i]` returns a Char; out of range throws
- `==` compares text; `===` compares references
- `"\$"` or `${'$'}` prints a literal dollar sign
- Strings are iterable: `for (c in s)`

UNDER THE JVM

- Templates compile to `StringBuilder` appends or `invokedynamic` string concatenation
- A Kotlin `String` is exactly a `java.lang.String`
- Raw string `trimIndent()` on a constant is computed at compile time

TEMPLATES AND RAW STRINGS

```
val name = "Ada"
val user = User("Ada", 36)

println("Hello, $name!")                  // Hello, Ada!
println("Next year: ${user.age + 1}")     // Next year: 37
println("Cost: \$5")                      // Cost: $5
println(name.length)                      // 3
println(name[0])                          // A
println(name == "Ada")                    // true

val json = """
    {
      "name": "$name",
      "path": "C:\temp"
    }
""".trimIndent()

val sql = """
    |SELECT *
    |  FROM users
    | WHERE name = ?
""".trimMargin()

for (c in "abc") print(c.uppercaseChar())  // ABC
```

CODE EXPLAINED

- → `${user.age + 1}` evaluates an expression inside the string
- → Raw strings keep `C:\temp` without escaping and still support templates
- → `trimIndent()` removes the common indentation
- → `trimMargin()` strips everything up to and including `|`

JAVA EQUIVALENT

- `"Hello, " + name + "!"` or `"Hello, %s!".formatted(name)`
- Text blocks `"""…"""` (Java 15+) like Kotlin raw strings
- `s.charAt(0)` instead of `s[0]`; `.equals()` for contents

IDIOMATIC KOTLIN

- ⇄ `"Hi " + name + "!"` → `"Hi $name!"`
- ⇄ escaped multi-line strings with `\n` → raw strings + `trimIndent()`

LITERALS

FORM	NOTES
`"text"`	Escapes allowed
`"""text"""`	Raw, multi-line
`"$x"`	Template
`"${a.b}"`	Expression template
`"\$" / "${'$'}"`	Literal $

BASICS

EXPRESSION	RESULT
`s.length`	Char count
`s[0]`	First Char
`s.isEmpty()`	length == 0
`s == t`	Same text
`s + t`	Concatenate
`s.lines()`	List of lines

⚠ COMMON MISTAKES

- `"$user.name"` prints the object then ".name" (needs `${user.name}`)
- Using `===` to compare strings
- Escaping backslashes inside raw strings (they are already literal)

✓ TIPS & BEST PRACTICES

- Use raw strings for regex patterns and JSON
- Use `trimIndent()` so code indentation stays tidy
- Prefer templates over `+` concatenation

QUICK SUMMARY

- `$x` / `${expr}` — templates
- `"""…"""` — raw strings
- `trimIndent()` — clean indents
- `==` — compare text

Building Strings

ASSEMBLING TEXT EFFICIENTLY

For text built in loops, `buildString { }` gives you a `StringBuilder` and returns the final string. `joinToString()` turns any collection into a delimited string with optional prefix, suffix and transform. `padStart`, `padEnd`, `repeat` and `format` handle alignment and number formatting.

KEY POINTS

- `buildString` is the idiomatic StringBuilder
- `joinToString` defaults to `", "` as separator
- `format` uses Java `Formatter` rules
- `repeat(n)` repeats a string n times

BUILDSTRING, JOINTOSTRING, PADDING

```
val items = listOf("apple", "pear", "fig")

val report = buildString {
    appendLine("Fruit report")
    items.forEachIndexed { i, f -> appendLine("${i + 1}. $f") }
    append("Total: ${items.size}")
}
println(report)

println(items.joinToString())                    // apple, pear, fig
println(items.joinToString(" | "))               // apple | pear | fig
println(items.joinToString(prefix = "[", postfix = "]") { it.uppercase() })
                                                 // [APPLE, PEAR, FIG]
println("7".padStart(3, '0'))                    // 007
println("ID".padEnd(6, '.') + "|")               // ID....|
println("=".repeat(10))                          // ==========
println("%-8s%6.2f".format("Tea", 3.5))          // Tea       3.50
println("%,d".format(1234567))                   // 1,234,567
```

CODE EXPLAINED

- → Inside `buildString`, `this` is the StringBuilder
- → `appendLine` adds text plus a newline
- → `joinToString` takes a transform lambda as the last argument
- → `%-8s` left-aligns in 8 columns; `%6.2f` right-aligns a number

JAVA EQUIVALENT

- `var sb = new StringBuilder(); sb.append(…); sb.toString();`
- `String.join(", ", items)` or `Collectors.joining(", ", "[", "]")`
- `"x".repeat(10)` (Java 11+) and `String.format`

IDIOMATIC KOTLIN

- ⇄ `var s = ""` + `s += …` in a loop → `buildString { }`
- ⇄ manual comma logic → `joinToString()`

FUNCTIONS

CALL	RESULT
`buildString { append(…) }`	Built string
`list.joinToString(sep)`	Delimited text
`joinToString(limit = 2)`	"a, b, …"
`s.padStart(n, c)`	Left pad
`s.padEnd(n, c)`	Right pad
`s.repeat(n)`	Repeated
`"%.2f".format(x)`	Formatted

FORMAT SPECS

SPEC	MEANING
`%s`	String
`%d`	Integer
`%,d`	Grouped digits
`%.2f`	2 decimals
`%8s / %-8s`	Right / left align
`%05d`	Zero pad

⚠ COMMON MISTAKES

- String `+=` in large loops (creates many strings)
- Locale surprises: `format` may use a comma decimal separator
- Forgetting that `joinToString` already adds ", "

✓ TIPS & BEST PRACTICES

- Pass `Locale.US` with `String.format(Locale.US, …)` for fixed output
- Use `joinToString(limit = n)` for long previews
- Use `appendLine` instead of `append("\n")`

QUICK SUMMARY

- `buildString` — efficient building
- `joinToString` — delimit lists
- `padStart` / `padEnd` — align
- `format` — numbers

String Methods — Search

contains · startsWith / endsWith · indexOf / lastIndexOf · isBlank() / isEmpty() / isNullOrBlank() · equals(ignoreCase = true) · count { }

FINDING THINGS IN TEXT

Kotlin strings have search functions for substrings, prefixes, positions and predicates. Most take an `ignoreCase` parameter. `isBlank()` checks for whitespace-only text, and `isNullOrBlank()` / `isNullOrEmpty()` work on nullable strings so you can skip a separate null check.

KEY POINTS

- `indexOf` returns -1 when not found
- `in` is shorthand for `contains`
- `isBlank()` is true for "" and " "
- Predicates: `any`, `all`, `none`, `count` work on characters

SEARCH FUNCTIONS

```
val s = "Kotlin Coroutines Guide"

println("Coroutines" in s)                    // true
println(s.contains("guide", ignoreCase = true))   // true
println(s.startsWith("Kot"))                  // true
println(s.endsWith(".pdf"))                   // false
println(s.indexOf('o'))                       // 1
println(s.lastIndexOf('o'))                   // 10
println(s.indexOf("Java"))                    // -1
println(s.count { it.isUpperCase() })         // 3
println(s.any { it.isDigit() })               // false
println(s.equals("kotlin coroutines guide", ignoreCase = true))   // true
println(s.indexOfFirst { it == ' ' })         // 6

val blank = "   "
val missing: String? = null
println(blank.isEmpty())                      // false
println(blank.isBlank())                      // true
println(missing.isNullOrBlank())              // true
println(s.commonPrefixWith("Kotlin Flow"))    // "Kotlin "
```

CODE EXPLAINED

→ `in` reads naturally in conditions

→ `ignoreCase = true` avoids lowercasing both sides

→ `isNullOrBlank()` can be called on a null String safely

→ `indexOfFirst` takes a predicate instead of a value

JAVA EQUIVALENT

- `s.contains("x")`, `s.startsWith("x")`, `s.indexOf("x")`
- `s.isBlank()` (Java 11+); null checks are separate
- `s.equalsIgnoreCase(t)`; `s.chars().filter(…).count()`

IDIOMATIC KOTLIN

⇄ `s == null || s.trim().isEmpty()` → `s.isNullOrBlank()`

⇄ `s.toLowerCase().contains(t.toLowerCase())` → `contains(t, ignoreCase = true)`

FUNCTIONS

CALL	RETURNS
`contains / in`	Boolean
`startsWith / endsWith`	Boolean
`indexOf / lastIndexOf`	Index or -1
`indexOfFirst { }`	Index or -1
`count { }`	Matching chars
`isEmpty / isBlank`	Boolean
`isNullOrEmpty / isNullOrBlank`	Null-safe Boolean

⚠ COMMON MISTAKES

- Checking `isEmpty()` when whitespace-only input should also be rejected
- Using `indexOf(...) > 0` to test presence (misses index 0)
- Lowercasing with the default locale for comparisons

✓ TIPS & BEST PRACTICES

- Use `in` for readability in `if` conditions
- Use `isNullOrBlank()` on form fields
- Use `Regex` (sheet 20) for patterns, not chains of `contains`

QUICK SUMMARY

- `in` / `contains` — substring
- `startsWith` / `endsWith`
- `indexOf` — position or -1
- `isNullOrBlank()` — null-safe check

String Methods — Transform

CHANGING TEXT

Because strings are immutable, transform functions return new strings. Kotlin adds handy ones beyond Java: `substringBefore/substringAfter` split at a delimiter, `take/drop` keep or skip characters, and `removePrefix/removeSuffix` strip known ends.

KEY POINTS

- `uppercase()` uses a locale-independent rule (replaced `toUpperCase()`)
- `split` returns a List, not an array
- `take(n)` never throws if the string is shorter
- `replace` replaces all occurrences

TRANSFORM FUNCTIONS

```
val path = "  /home/ada/report.final.pdf  "
val p = path.trim()                          // "/home/ada/report.final.pdf"

println(p.uppercase())                       // /HOME/ADA/REPORT.FINAL.PDF
println(p.substringAfterLast('/'))           // report.final.pdf
println(p.substringBeforeLast('.'))          // /home/ada/report.final
println(p.substringAfterLast('.'))           // pdf
println(p.removePrefix("/home/"))            // ada/report.final.pdf
println(p.replace("ada", "bob"))             // /home/bob/report.final.pdf
println(p.split('/'))                        // [, home, ada, report.final.pdf]
println(p.split('/').filter { it.isNotEmpty() })
// [home, ada, report.final.pdf]

val code = "ABC-123"
println(code.take(3))                        // ABC
println(code.drop(4))                        // 123
println(code.takeLast(2))                    // 23
println(code.substring(1, 3))                // BC
println(code.reversed())                     // 321-CBA
println("a,b;c".split(',', ';'))             // [a, b, c]
println("line1\nline2".lines())              // [line1, line2]
println("kotlin".replaceFirstChar { it.uppercase() })   // Kotlin
```

CODE EXPLAINED

- → `substringAfterLast('/')` gets a file name without index math
- → `split` keeps empty strings between delimiters
- → `take` / `drop` are safe on short strings; `substring` can throw
- → `replaceFirstChar` replaces the deprecated `capitalize()`

JAVA EQUIVALENT

- ☕ `s.toUpperCase()`, `s.strip()`, `s.replace(a, b)`
- ☕ `s.split("/")` returns `String[]` and takes a regex
- ☕ `s.substring(s.lastIndexOf('/') + 1)` for the file name

IDIOMATIC KOTLIN

- ⇄ `s.substring(s.lastIndexOf('/') + 1)` → `s.substringAfterLast('/')`
- ⇄ `capitalize()` (deprecated) → `replaceFirstChar { it.uppercase() }`

FUNCTIONS

CALL	EXAMPLE → RESULT
`trim()`	" a " → "a"
`uppercase() / lowercase()`	"a" → "A"
`replace(a, b)`	All occurrences
`split(d)`	List<String>
`substringBefore(d)`	"a.b" → "a"
`substringAfter(d)`	"a.b" → "b"
`removePrefix / removeSuffix`	Strip known ends
`take(n) / drop(n)`	Keep / skip
`padStart / reversed`	Pad / reverse

⚠ COMMON MISTAKES

- Expecting `split` to take a regex (it takes literal delimiters; pass `Regex` to use one)
- Calling `substring` with an index past the end
- Using deprecated `toUpperCase()` / `capitalize()`

✓ TIPS & BEST PRACTICES

- Use `substringBefore/After` instead of index arithmetic
- Use `split(...).map { it.trim() }` for CSV-ish input
- Use `take(n)` for safe previews

QUICK SUMMARY

- `uppercase` / `trim` / `replace`
- `split` — List result
- `substringBefore/After`
- `take` / `drop` — safe slices

Regex & Parsing

REGULAR EXPRESSIONS

Kotlin wraps Java regex in the `Regex` class. `matches` tests the whole string, `containsMatchIn` searches anywhere, `find` returns the first `MatchResult` (or null) and `findAll` returns a sequence. Raw strings avoid double escaping, and `destructured` unpacks capture groups into variables.

KEY POINTS

- Pattern syntax is Java's (`java.util.regex`)
- Write patterns in raw strings: `"""\d+"""`
- Named groups: `(?<year>\d{4})` and `groups["year"]`
- `RegexOption.IGNORE_CASE` and others go in the constructor

REGEX IN PRACTICE

```
val datePattern = Regex("""(\d{4})-(\d{2})-(\d{2})""")
val text = "Due 2026-11-12, paid 2026-12-01"

println(datePattern.containsMatchIn(text))      // true
println(datePattern.matches("2026-11-12"))      // true

val first = datePattern.find(text)
println(first?.value)                           // 2026-11-12
val (year, month, day) = first!!.destructured
println("$month/$day/$year")                    // 11/12/2026

datePattern.findAll(text).forEach { println(it.value) }
// 2026-11-12
// 2026-12-01

val masked = datePattern.replace(text) { m ->
    "${m.groupValues[2]}/${m.groupValues[3]}"
}
println(masked)                                 // Due 11/12, paid 12/01

val email = Regex("""[\w.]+@[\w.]+""", RegexOption.IGNORE_CASE)
println("x@y.com".matches(email))               // true
println("a1b22c333".split(Regex("""\d+""")))    // [a, b, c, ]
```

CODE EXPLAINED

- → Raw strings let you write `\d` once instead of `\\d`
- → `destructured` maps groups 1, 2, 3 to variables
- → `groupValues[0]` is the whole match; `[1]` the first group
- → The lambda form of `replace` builds each replacement

JAVA EQUIVALENT

- `Pattern.compile("(\\d{4})-…")` with escaped backslashes
- `Matcher m = p.matcher(text); while (m.find()) m.group(1)`
- `m.replaceAll(r -> …)` (Java 9+)

IDIOMATIC KOTLIN

- ⇄ `Pattern` / `Matcher` boilerplate → `Regex` + `find` / `findAll`
- ⇄ `"\\d+"` escaped strings → raw `"""\d+"""`

REGEX FUNCTIONS

CALL	RETURNS
`matches(s)`	Whole string matches
`containsMatchIn(s)`	Match anywhere
`find(s)`	MatchResult?
`findAll(s)`	Sequence<MatchResult>
`replace(s, r)`	New string
`split(s)`	List<String>

MATCHRESULT

MEMBER	GIVES
`value`	Matched text
`range`	IntRange
`groupValues`	All groups
`groups["name"]`	Named group
`destructured`	Unpack groups

⚠ COMMON MISTAKES

- Double escaping inside raw strings (`"""\\d"""` matches a backslash)
- Using `matches` when you meant "contains"
- Calling `!!` on `find` when there may be no match

✓ TIPS & BEST PRACTICES

- Compile a `Regex` once as a `val`, not inside a loop
- Use named groups for readability
- Use `String.matches(Regex)` or `regex.matches(String)` for validation

QUICK SUMMARY

- `Regex("""…""")` — raw pattern
- `find` / `findAll` — search
- `destructured` — unpack groups
- `replace { }` — computed replace

Arithmetic

MATH OPERATORS

Kotlin's arithmetic operators map to functions (`plus`, `minus`, `times`, `div`, `rem`), which is how operator overloading works. Integer division truncates, `%` is the remainder, and `mod()` gives a result with the divisor's sign. `coerceIn` clamps a value to a range.

KEY POINTS

- `%` is `rem`: sign follows the dividend
- `a.mod(b)`: sign follows the divisor
- Mixed types widen: `Int * Double` is `Double`
- No `**` operator: use `pow`

OPERATORS AND HELPERS

```
val a = 17
val b = 5
println(a + b)              // 22
println(a - b)              // 12
println(a * b)              // 85
println(a / b)              // 3   (truncated)
println(a % b)              // 2
println(a / b.toDouble())   // 3.4
println(-17 % 5)            // -2
println((-17).mod(5))       // 3

var count = 10
count++                     // 11
count += 5                  // 16
count *= 2                  // 32
println(count--)            // 32, then count is 31

println(150.coerceIn(0, 100))      // 100
println((-5).coerceAtLeast(0))     // 0
println(maxOf(3, 9, 4))            // 9
println(2.0.pow(8))                // 256.0 (kotlin.math.pow)
println(7.toDouble() / 2)          // 3.5
```

CODE EXPLAINED

→ Convert one side to Double to get a decimal result

→ `-17 % 5` is -2 but `(-17).mod(5)` is 3

→ Postfix `count--` returns the old value

→ `coerceIn` replaces `min(max(x, lo), hi)`

JAVA EQUIVALENT

☕ Same operators; `%` is remainder

☕ `Math.floorMod(-17, 5)` → 3

☕ `Math.max(0, Math.min(150, 100))` for clamping

IDIOMATIC KOTLIN

⇄ `Math.max(lo, Math.min(x, hi))` → `x.coerceIn(lo, hi)`

⇄ `(a % n + n) % n` → `a.mod(n)`

OPERATORS

OPERATOR	FUNCTION
`a + b`	a.plus(b)
`a - b`	a.minus(b)
`a * b`	a.times(b)
`a / b`	a.div(b)
`a % b`	a.rem(b)
`a++ / a--`	inc() / dec()
`-a`	unaryMinus()

HELPERS

CALL	RESULT
`x.coerceIn(a, b)`	Clamp
`coerceAtLeast / AtMost`	One-sided clamp
`maxOf / minOf`	Largest / smallest
`x.mod(n)`	Non-negative modulo
`x.absoluteValue`	Absolute value

⚠ COMMON MISTAKES

- `7 / 2` expecting 3.5
- Negative remainders breaking wrap-around index math
- Looking for a `**` operator

✓ TIPS & BEST PRACTICES

- Use `mod` for circular indexes
- Use `coerceIn` for user-provided limits
- Use `BigDecimal` operators (they overload too) for money

QUICK SUMMARY

- `/` — truncates for Ints
- `%` vs `mod` — sign rules
- `coerceIn` — clamp
- operators = functions

Comparison & Equality

COMPARING VALUES

In Kotlin, == calls `equals()` and is null-safe, so it compares **contents**. === checks whether two references point to the **same object**. <, >, <= and >= call `compareTo`, so they work on Strings, dates, BigDecimals and anything `Comparable`.

KEY POINTS

- `a == b` is `a?.equals(b) ?: (b === null)`
- Data classes generate `equals` automatically
- `in` checks ranges and collections
- Floating-point `==` follows IEEE rules for typed Doubles

UNDER THE JVM

- `==` compiles to `Intrinsics.areEqual(a, b)` (null-safe equals)
- `<` on objects compiles to `a.compareTo(b) < 0`
- Boxed Ints from -128..127 are cached, so `===` can lie in tests

== VS ===

```
val a = "Kotlin"
val b = StringBuilder("Kot").append("lin").toString()
println(a == b)            // true  (same text)
println(a === b)           // false (different objects)

data class Point(val x: Int, val y: Int)
println(Point(1, 2) == Point(1, 2))     // true  (generated equals)
println(Point(1, 2) === Point(1, 2))    // false

val n: String? = null
println(n == null)         // true, no NPE

println("apple" < "banana")             // true (compareTo)
println(LocalDate.of(2026, 1, 1) < LocalDate.now())
println(BigDecimal("2.0") == BigDecimal("2.00"))       // false (scale differs)
println(BigDecimal("2.0").compareTo(BigDecimal("2.00")) == 0)   // true

val score = 85
println(score in 80..89)   // true
println('q' !in "aeiou")   // true
println(compareValues(3, 7))            // negative
```

CODE EXPLAINED

- → `==` on Strings compares text, unlike Java
- → Data classes compare all constructor properties
- → BigDecimal `equals` checks scale; use `compareTo` for value
- → `in` works with ranges, strings and collections

JAVA EQUIVALENT

- `a.equals(b)` or `Objects.equals(a, b)` for contents
- `a == b` in Java compares references (Kotlin `===`)
- `a.compareTo(b) < 0` instead of `<` on objects

IDIOMATIC KOTLIN

- ⇄ `a.equals(b)` → `a == b`
- ⇄ `a.compareTo(b) < 0` → `a < b`

OPERATORS

KOTLIN	MEANS
`a == b`	equals (null-safe)
`a != b`	!equals
`a === b`	Same reference
`a !== b`	Different reference
`a < b`	compareTo < 0
`a in r`	r.contains(a)

JAVA ↔ KOTLIN

JAVA	KOTLIN
`a.equals(b)`	a == b
`a == b (refs)`	a === b
`a.compareTo(b) > 0`	a > b
`Objects.equals(a, b)`	a == b

⚠ COMMON MISTAKES

- Using `===` to compare strings or numbers
- Expecting BigDecimal `==` to ignore scale
- Overriding `equals` without `hashCode` (use a data class)

✓ TIPS & BEST PRACTICES

- Use `==` everywhere unless identity truly matters
- Implement `Comparable` to get `<` and `>` on your types
- Use `compareBy` for multi-field comparisons (sheet 54)

QUICK SUMMARY

- `==` — contents
- `===` — same object
- `<` `>` — compareTo
- `in` — range / contains

Logical & Bitwise

BOOLEAN LOGIC AND BITS

Boolean logic uses &&, || and !, which short-circuit. For bits, Kotlin uses named **infix functions** instead of symbols: `and`, `or`, `xor`, `inv()`, `shl`, `shr` and `ushr`. On Booleans, `and`/`or` also exist but do not short-circuit.

KEY POINTS

- `&&` / `||` short-circuit; infix `and` / `or` do not
- No `&`, `|`, `^`, `<<`, `>>` symbols
- `ushr` is the unsigned right shift (Java `>>>`)
- Bit functions work on Int and Long

LOGIC AND BIT OPERATIONS

```
val loggedIn = true
val isAdmin = false
println(loggedIn && isAdmin)        // false
println(loggedIn || isAdmin)        // true
println(!isAdmin)                   // true

val a = 0b1100                      // 12
val b = 0b1010                      // 10
println(a and b)                    // 8   (0b1000)
println(a or b)                     // 14  (0b1110)
println(a xor b)                    // 6   (0b0110)
println(a.inv())                    // -13
println(1 shl 4)                    // 16
println(-16 shr 2)                  // -4
println(-16 ushr 28)                // 15
println(a.countOneBits())           // 2
println(Integer.toBinaryString(a)) // 1100

const val READ = 1; const val WRITE = 2; const val EXEC = 4
var perms = READ or WRITE           // 3
println(perms and WRITE != 0)       // true
perms = perms and WRITE.inv()       // remove WRITE → 1
```

CODE EXPLAINED

→ Infix calls read as words: `a and b`

→ `shr` keeps the sign; `ushr` fills with zeros

→ Combine flags with `or`, test with `and`

→ Infix calls bind tighter than `!=`, so this reads as `(perms and WRITE) != 0`

JAVA EQUIVALENT

- `&&`, `||`, `!` are the same
- `a & b`, `a | b`, `a ^ b`, `~a`
- `<<`, `>>`, `>>>`; `Integer.bitCount(a)`

IDIOMATIC KOTLIN

⇄ raw bit flags for options → `EnumSet` or `Set<Flag>`

⇄ magic masks `0x4` → named `const val` flags

BITWISE

KOTLIN	JAVA
`a and b`	a & b
`a or b`	a | b
`a xor b`	a ^ b
`a.inv()`	~a
`a shl n`	a << n
`a shr n`	a >> n
`a ushr n`	a >>> n

BIT HELPERS

CALL	RESULT
`countOneBits()`	Set bits
`countLeadingZeroBits()`	Leading zeros
`takeHighestOneBit()`	Top bit
`rotateLeft(n)`	Rotate
`toString(2)`	Binary text

⚠ COMMON MISTAKES

- Typing `&` or `<<` (not Kotlin operators)
- Using infix `and` on Booleans and losing short-circuit
- Assuming Java precedence: in Kotlin `a and b == c` means `(a and b) == c`

✓ TIPS & BEST PRACTICES

- Add parentheses around infix bit expressions
- Prefer enums and sets for permissions in new code
- See the Bitwise set for patterns

QUICK SUMMARY

- `&&` `||` `!` — logic
- `and` `or` `xor` `inv` — bits
- `shl` `shr` `ushr` — shifts
- parentheses — infix precedence

Null-Safety Operators

WORKING WITH NULLABLE VALUES

Kotlin's null-safety operators chain together: `?.` skips a call on null, `?:` supplies a fallback (which can even be `return` or `throw`), and `?.let { }` runs a block only for non-null values. `!!` forces a non-null type and throws if it is wrong.

KEY POINTS

- A `?.` chain stops at the first null
- `?:` evaluates the right side only when the left is null
- `let` passes the value as `it` and returns the block result
- Collections: `filterNotNull`, `mapNotNull`, `firstOrNull`

CHAINING NULL OPERATORS

```
data class Address(val city: String?)
data class User(val name: String, val address: Address?)

val user: User? = User("Ada", Address(null))

println(user?.address?.city)                 // null
println(user?.address?.city ?: "Unknown")    // Unknown
println(user?.name?.length ?: 0)             // 3

fun cityOf(u: User?): String {
    val addr = u?.address ?: return "No address"
    return addr.city ?: throw IllegalStateException("No city")
}

user?.let { println("Hello, ${it.name}") }   // Hello, Ada

val id: Any = "42"
val asInt = id as? Int ?: -1                 // -1

val raw = listOf("1", null, "x", "3")
println(raw.filterNotNull())                 // [1, x, 3]
println(raw.mapNotNull { it?.toIntOrNull() }) // [1, 3]
val name = user!!.name                       // throws if user is null
```

CODE EXPLAINED

→ The `?.` chain returns null at the first missing link

→ `?: return` exits early with a message

→ `?: throw` turns a missing value into an error

→ `mapNotNull` drops nulls produced by the transform

JAVA EQUIVALENT

☕ `Optional.ofNullable(user).map(User::address).map(Address::city).orElse("Unknown")`

☕ `if (user != null && user.address() != null) …`

☕ `Objects.requireNonNull(user)` ≈ `user!!`

IDIOMATIC KOTLIN

⇄ nested `if (x != null)` checks → `x?.a?.b ?: default`

⇄ `if (x != null) foo(x)` → `x?.let(::foo)`

OPERATORS

SYNTAX	RESULT IF NULL
`a?.b`	null
`a ?: b`	b
`a ?: return`	Leave function
`a ?: throw e`	Throw
`a!!`	NullPointerException
`a?.let { }`	Block skipped
`a as? T`	null on wrong type

COLLECTION HELPERS

CALL	DOES
`filterNotNull()`	Drop nulls
`mapNotNull { }`	Map + drop nulls
`firstOrNull()`	First or null
`getOrNull(i)`	Element or null
`orEmpty()`	null → empty

⚠ COMMON MISTAKES

- Long `!!` chains that hide which value was null
- `x?.let { … } ?: other` surprises when the block itself returns null
- Using `?.let` where a simple `if` is clearer

✓ TIPS & BEST PRACTICES

- Use `?: return` at the top of functions for guard clauses
- Use `orEmpty()` on nullable lists and strings
- Reserve `!!` for tests and impossible states

QUICK SUMMARY

- `?.` — safe call
- `?:` — fallback / return / throw
- `?.let` — run if not null
- `mapNotNull` / `filterNotNull`

Operator Overloading

OPERATORS ON YOUR OWN TYPES

Mark a function with `operator` and Kotlin lets you call it with a symbol: `plus` becomes +, `get` becomes `[]`, `contains` becomes `in`, `invoke` makes an object callable. The set of operators is fixed, so overloading stays predictable.

A MONEY TYPE WITH OPERATORS

```
data class Money(val cents: Long) : Comparable<Money> {
    operator fun plus(o: Money) = Money(cents + o.cents)
    operator fun minus(o: Money) = Money(cents - o.cents)
    operator fun times(n: Int) = Money(cents * n)
    operator fun unaryMinus() = Money(-cents)
    override operator fun compareTo(other: Money) = cents.compareTo(other.cents)
    override fun toString() = "$%d.%02d".format(cents / 100, cents % 100)
}

class Inventory {
    private val stock = mutableMapOf<String, Int>()
    operator fun get(item: String) = stock[item] ?: 0
    operator fun set(item: String, qty: Int) { stock[item] = qty }
    operator fun contains(item: String) = item in stock
}

class Greeter(val greeting: String) {
    operator fun invoke(name: String) = "$greeting, $name"
}

val total = Money(1999) + Money(500) * 2        // $29.99
println(total > Money(2000))                    // true
val inv = Inventory()
inv["apple"] = 5                                // set
println(inv["apple"])                           // 5 (get)
println("pear" in inv)                          // false
println(Greeter("Hi")("Ada"))                   // Hi, Ada
```

KEY POINTS

- Function names are fixed by convention
- Operators can be extensions on existing types
- Compound `+=` uses `plusAssign` or falls back to `plus`
- Keep the meaning obvious: `+` should mean add

CODE EXPLAINED

- → `Money(500) * 2` calls `times`
- → Implementing `Comparable` gives `<` and `>`
- → `inv["apple"] = 5` calls `set`
- → `Greeter("Hi")("Ada")` calls `invoke`

JAVA EQUIVALENT

- No operator overloading: `a.add(b)`, `a.multiply(n)`
- `map.get(k)` / `map.put(k, v)` instead of `[]`
- `BigDecimal.add` is the usual pattern

IDIOMATIC KOTLIN

- ⇄ `a.add(b)` method names on math types → `operator fun plus`
- ⇄ surprising operators (`+` that deletes) → named functions

OPERATOR FUNCTIONS

EXPRESSION	FUNCTION
`a + b, a - b`	plus, minus
`a * b, a / b, a % b`	times, div, rem
`-a, !a`	unaryMinus, not
`a[i] / a[i] = v`	get / set
`a(x)`	invoke
`a in b`	b.contains(a)
`a..b / a..<b`	rangeTo / rangeUntil
`a += b`	plusAssign
`val (x, y) = a`	component1, component2

⚠ COMMON MISTAKES

- Forgetting the `operator` keyword (the symbol will not work)
- Defining both `plus` and `plusAssign` on a mutable type (ambiguous `+=`)
- Operators that do something unexpected

✓ TIPS & BEST PRACTICES

- Use data classes to get `componentN` for free
- Add operators as extensions to Java types you use often
- Implement `Comparable` instead of a custom `compareTo` operator

QUICK SUMMARY

- `operator fun` — enable symbol
- `get` / `set` — `[]`
- `invoke` — callable object
- `contains` — `in`

if as an Expression

`if / else if / else` · `val x = if (…) a else b` · no ternary · last expression is the value · block bodies

IF RETURNS A VALUE

In Kotlin, `if` is an **expression**: it produces a value you can assign or return. That replaces Java's ternary operator. In a block, the last expression is the block's value. When `if` is used as an expression, an `else` branch is required.

KEY POINTS

- There is no `?:` ternary; `?:` is the Elvis operator
- Branches can be blocks `{ }`
- Conditions must be Boolean
- Long chains often read better as `when`

IF STATEMENTS AND EXPRESSIONS

```
val temp = 72

if (temp > 85) {
    println("Hot")
} else if (temp < 50) {
    println("Cold")
} else {
    println("Nice")                          // Nice
}

val label = if (temp > 85) "Hot" else "OK"  // replaces ternary

val fee = if (temp > 90) {
    println("Heat surcharge")
    5.0                                     // value of this branch
} else {
    0.0
}

fun max(a: Int, b: Int) = if (a > b) a else b

val message = if (temp in 60..80) "Ideal" else if (temp < 60) "Cool" else "Warm"
println(message)                            // Ideal
```

CODE EXPLAINED

→ `if` used as a statement needs no `else`

→ As an expression, every path must produce a value

→ The last line of a block is its result (5.0 or 0.0)

→ `max` is a one-line function built on an if expression

JAVA EQUIVALENT

☕ `String label = temp > 85 ? "Hot" : "OK";`

☕ `if` in Java is only a statement

☕ Switch expressions (Java 14+) are the nearest equivalent for multi-way

IDIOMATIC KOTLIN

⇄ `var x; if (c) x = a else x = b` → `val x = if (c) a else b`

⇄ long `else if` ladders → `when`

FORMS

FORM	USE
`if (c) a`	Statement
`if (c) a else b`	Expression
`if (c) { … } else { … }`	Block branches
`else if`	Chains
`= if (…) a else b`	Function body

⚠ COMMON MISTAKES

- Writing `c ? a : b` (not Kotlin)
- Missing `else` on an if expression (compile error)
- Putting a `return` in one branch and a value in another by accident

✓ TIPS & BEST PRACTICES

- Assign from `if` to keep variables as `val`
- Switch to `when` after two `else if`s
- Keep expression branches short

QUICK SUMMARY

- `if` — expression
- no ternary
- `else` required as expression
- block value — last line

when

when (x) · when without subject · is / in branches · multiple values 1, 2 -> · exhaustive when · guard conditions if in branches

KOTLIN'S SWITCH

when matches a value against branches and runs the first that fits, with no fall-through. Branches can test constants, several values, ranges (in), types (is) or any condition. As an expression it must be **exhaustive**, which the compiler checks for enums, sealed types and Booleans.

KEY POINTS

- `else` is the default branch
- Without a subject, each branch is a Boolean condition
- Guard conditions (`is X if …`) arrived in Kotlin 2.1 (stable in 2.2)
- `when (val x = f())` scopes a variable to the when

WHEN FORMS

```
fun grade(score: Int) = when (score) {
    100        -> "Perfect"
    in 90..99  -> "A"
    in 80..89  -> "B"
    0, -1      -> "Missing"
    else       -> "C or below"
}

fun describe(x: Any) = when (x) {
    is Int if x < 0 -> "negative Int"        // guard condition
    is Int    -> "Int $x"
    is String -> "String of ${x.length}"
    else      -> "other"
}

val temp = 72
val advice = when {                          // no subject
    temp > 85 -> "Stay inside"
    temp < 50 -> "Wear a coat"
    else      -> "Go outside"
}

enum class Light { RED, YELLOW, GREEN }
fun action(l: Light) = when (l) {            // exhaustive, no else
    Light.RED    -> "Stop"
    Light.YELLOW -> "Slow"
    Light.GREEN  -> "Go"
}

when (val user = findUser()) {
    null -> println("not found")
    else -> println(user.name)
}
```

CODE EXPLAINED

→ `in 90..99` matches a range

→ `0, -1` shares one branch

→ Covering every enum value removes the need for `else`

→ Adding a new enum value breaks the build until `when` is updated

JAVA EQUIVALENT

- `switch (score) { case 100 -> "Perfect"; default -> … }` (Java 14+)
- `case Integer i when i < 0 ->` guards (Java 21)
- Java switch has no ranges; use `if` chains

IDIOMATIC KOTLIN

⇄ `if/else if` ladders → `when`

⇄ `else` on enum/sealed `when` → list all cases (compiler catches new ones)

BRANCH TYPES

BRANCH	MATCHES
`1 ->`	Equal value
`1, 2 ->`	Either value
`in 1..9 ->`	Range
`!in list ->`	Not in collection
`is T ->`	Type (smart cast)
`is T if cond ->`	Type + guard
`else ->`	Anything else

⚠ COMMON MISTAKES

- Expecting fall-through like C switch
- Adding `else` to enum `when` and missing new values later
- Putting a broad branch before a specific one

✓ TIPS & BEST PRACTICES

- Use subject-less `when` for complex conditions
- Return `when` directly from functions
- Pair `when` with sealed classes for state handling (sheet 63)

QUICK SUMMARY

- `when (x)` — match value
- `in` / `is` — ranges / types
- exhaustive — enums, sealed
- no fall-through

for Loops

ITERATING WITH FOR

Kotlin's `for` loop iterates over anything with an `iterator()`: ranges, collections, strings and maps. There is no C-style `for (i = 0; …)`; use ranges like `0..<n`, `downTo` and `step` instead. `withIndex()` and `indices` give positions when you need them.

KEY POINTS

- `1..5` includes 5; `1..<5` and `1 until 5` exclude it
- `downTo` counts backward
- Destructure map entries: `for ((k, v) in map)`
- Prefer `forEach` / collection functions for transforms

LOOP FORMS

```
for (i in 1..5) print(i)                // 12345
for (i in 0..<5) print(i)               // 01234
for (i in 10 downTo 0 step 3) print(" $i")  //  10 7 4 1
println()

val fruits = listOf("apple", "pear", "fig")
for (f in fruits) println(f)
for (i in fruits.indices) println("$i: ${fruits[i]}")
for ((i, f) in fruits.withIndex()) println("$i -> $f")

val prices = mapOf("tea" to 3.5, "cake" to 4.0)
for ((item, price) in prices) println("$item costs $price")

for (c in "hey") print(c.uppercaseChar())   // HEY
println()

for (row in 1..3) {
    for (col in 1..3) print("${row * col}\t")
    println()
}

fruits.forEachIndexed { i, f -> println("${i + 1}. $f") }
```

CODE EXPLAINED

→ `0..<5` is the clearest way to loop over sizes

→ `step 3` skips by three

→ `withIndex()` gives index and value together

→ Map entries destructure into key and value

JAVA EQUIVALENT

☕ `for (int i = 0; i < 5; i++)`

☕ `for (String f : fruits)`

☕ `for (var e : prices.entrySet()) { e.getKey(); e.getValue(); }`

IDIOMATIC KOTLIN

⇄ `for (i in 0..list.size - 1)` → `for (i in list.indices)`

⇄ index loops to build lists → `map`, `filter` (sheet 52)

RANGES

EXPRESSION	VALUES
`1..5`	1, 2, 3, 4, 5
`1..<5 / 1 until 5`	1, 2, 3, 4
`5 downTo 1`	5, 4, 3, 2, 1
`0..10 step 5`	0, 5, 10
`list.indices`	0..lastIndex
`(1..5).reversed()`	5..1

⚠ COMMON MISTAKES

- `0..list.size` (one past the end)
- Expecting `for (i = 0; …)` syntax
- Modifying a list while iterating it (ConcurrentModificationException)

✓ TIPS & BEST PRACTICES

- Use `indices` or `withIndex` instead of manual counters
- Use `repeat(n) { }` for "do this n times"
- Prefer collection functions over loops that build results

QUICK SUMMARY

- `for (x in xs)` — iterate
- `..` / `..<` — ranges
- `downTo` / `step`
- `withIndex()` — index + value

while & do-while

CONDITION-BASED LOOPS

`while` repeats while a condition is true and checks it first. `do { } while` runs the body once before checking, and variables declared inside the body are visible in its condition. `repeat(n)` runs a block a fixed number of times.

KEY POINTS

- Same semantics as Java
- Variables in `do` are in scope in the `while` condition
- `repeat(n) { }` passes the zero-based index as `it`
- Use `while (true)` with `break` for read loops

WHILE, DO-WHILE, REPEAT

```
var balance = 1000.0
var years = 0
while (balance < 2000) {
    balance *= 1.07
    years++
}
println("$years years")              // 11 years

do {
    print("Enter a number (0 to quit): ")
    val n = readln().toIntOrNull() ?: -1
    if (n > 0) println("Square: ${n * n}")
} while (n != 0)                     // n is visible here

repeat(3) { i -> println("Attempt ${i + 1}") }

val queue = ArrayDeque(listOf("a", "b", "c"))
while (queue.isNotEmpty()) {
    println(queue.removeFirst())
}

val reader = File("data.txt").bufferedReader()
while (true) {
    val line = reader.readLine() ?: break   // exit at end of file
    println(line)
}
```

CODE EXPLAINED

→ The condition is checked before each pass

→ `n` declared inside `do` can be used in the `while`

→ `repeat` replaces counting loops

→ `readLine() ?: break` exits at end of file

JAVA EQUIVALENT

☕ `while (…) { }` and `do { } while (…);` are the same

☕ In Java, `do` body variables are not visible in the condition

☕ `for (int i = 0; i < 3; i++)` for repeat

IDIOMATIC KOTLIN

⇄ counter `while` loops → `repeat(n)` or `for (i in 0..<n)`

⇄ manual read loops → `useLines { }` (sheet 71)

LOOP CHOICE

NEED	USE
Known count	repeat(n) / for
Over a collection	for / forEach
Until a condition	while
At least once	do-while
Read until null	while (true) + ?: break

⚠ COMMON MISTAKES

- Forgetting to change the loop variable (infinite loop)
- Using `while` with an index where `for` is clearer
- Not handling null from `readLine()`

✓ TIPS & BEST PRACTICES

- Use `ArrayDeque` for queue-style loops
- Use `?: break` for end-of-input
- Keep loop bodies short; extract functions

QUICK SUMMARY

- `while` — check first
- `do-while` — run once first
- `repeat(n)` — fixed count
- `?: break` — exit on null

break, continue & Labels

JUMPING OUT OF LOOPS

`break` leaves the nearest loop and `continue` skips to its next pass. **Labels** (`outer@`) let you target an outer loop. Inside lambdas such as `forEach`, a plain `return` leaves the enclosing **function**; use `return@forEach` to skip to the next element instead.

KEY POINTS

- Label syntax: `name@` before the loop, `@name` after the jump
- Lambdas get implicit labels named after the function
- `break` / `continue` inside `forEach` work only since Kotlin 2.2 (non-local break/continue)
- Prefer `any`, `first`, `takeWhile` over complex jumps

LABELS AND LAMBDA RETURNS

```
for (i in 1..10) {
    if (i % 2 == 0) continue       // skip evens
    if (i > 7) break               // stop at 9
    print("$i ")                   // 1 3 5 7
}
println()

outer@ for (row in 1..3) {
    for (col in 1..3) {
        if (col == 2) continue@outer
        if (row == 3) break@outer
        println("$row,$col")       // 1,1  2,1
    }
}

fun firstNegative(nums: List<Int>): Int? {
    nums.forEach {
        if (it < 0) return it      // returns from firstNegative
    }
    return null
}

listOf(1, -2, 3).forEach {
    if (it < 0) return@forEach     // like continue
    print("$it ")                  // 1 3
}

val found = listOf(4, 9, 16).first { it > 5 }   // 9, no loop needed
```

CODE EXPLAINED

- → `continue@outer` jumps to the next row
- → A plain `return` in an inline lambda leaves the whole function
- → `return@forEach` ends only the current element's block
- → `first { }` often replaces a loop with `break`

JAVA EQUIVALENT

- ☕ `outer: for (…) { … continue outer; }` labeled loops
- ☕ In `stream().forEach`, `return` acts like continue
- ☕ No non-local returns from lambdas in Java

IDIOMATIC KOTLIN

- ⇄ `forEach` + `return@forEach` to stop early → a `for` loop or `first { }`
- ⇄ nested loops with labels → extract a function and `return`

JUMPS

STATEMENT	EFFECT
`break`	Leave loop
`continue`	Next iteration
`break@label`	Leave labeled loop
`continue@label`	Next of labeled loop
`return`	Leave function (also from inline lambda)
`return@forEach`	Leave this lambda call

⚠ COMMON MISTAKES

- Using `return` in `forEach` and leaving the whole function by surprise
- Expecting `break` inside `forEach` to work on older Kotlin
- Deep label jumps that are hard to follow

✓ TIPS & BEST PRACTICES

- Use `for` loops when you need break/continue
- Use `firstOrNull`, `any`, `takeWhile` instead of manual searching
- Name labels for what they loop over (`rows@`)

QUICK SUMMARY

- `break` / `continue`
- `label@` + `break@label`
- `return@forEach` — skip one
- plain `return` — leaves function

Ranges & Progressions

RANGES AS VALUES

A **range** like `1..10` is an object (`IntRange`) with a start and end, not just loop syntax. Ranges support `in` checks, `step` and `reversed()` (which produce **progressions**), and collection-style functions such as `map`, `sum` and `filter`. They exist for Int, Long, Char and any `Comparable`.

KEY POINTS

- `a..b` is closed; `a..<b` is open-ended
- Empty if start > end: `5..1` has no elements
- `step` must be positive; use `downTo` to go backward
- Comparable ranges (`"a".."m"`) support `in` but not iteration

WORKING WITH RANGES

```
val r = 1..10
println(r.first)                    // 1
println(r.last)                     // 10
println(5 in r)                     // true
println(11 !in r)                   // true
println(r.count())                  // 10
println(r.sum())                    // 55
println(r.filter { it % 3 == 0 }) // [3, 6, 9]

val odd = 1..10 step 2
println(odd.toList())               // [1, 3, 5, 7, 9]
println((1..5).reversed().toList())   // [5, 4, 3, 2, 1]
println((5..1).isEmpty())           // true

val letters = 'a'..'e'
println(letters.joinToString(""))  // abcde
println('c' in letters)            // true

val versions = "1.0".."2.5"         // ClosedRange<String>
println("1.8" in versions)          // true (string compare)

val age = 130
println(age.coerceIn(0..120))      // 120
println((1..6).random())           // random die roll
```

CODE EXPLAINED

→ Ranges have properties like `first`, `last` and `count()`

→ `step 2` turns a range into a progression

→ `5..1` is empty; use `5 downTo 1` to count down

→ `random()` picks a random element of the range

JAVA EQUIVALENT

- `IntStream.rangeClosed(1, 10)` for a stream of numbers
- `x >= 1 && x <= 10` for range checks
- `ThreadLocalRandom.current().nextInt(1, 7)` for dice

IDIOMATIC KOTLIN

⇄ `x >= lo && x <= hi` → `x in lo..hi`

⇄ `5..1` expecting a countdown → `5 downTo 1`

RANGE BUILDERS

EXPRESSION	TYPE
`1..10`	IntRange (closed)
`1..<10`	Open-ended range
`10 downTo 1`	IntProgression
`1..10 step 2`	IntProgression
`'a'..'z'`	CharRange
`1L..5L`	LongRange

RANGE MEMBERS

MEMBER	GIVES
`first / last`	Ends
`contains / in`	Membership
`isEmpty()`	No elements
`reversed()`	Backward progression
`random()`	Random element

⚠ COMMON MISTAKES

- Writing `5..1` and getting zero iterations
- Negative `step` (throws)
- Iterating a String range (not iterable)

✓ TIPS & BEST PRACTICES

- Use ranges in `when` branches
- Use `coerceIn(range)` for limits
- Use `..<` for sizes and indexes

QUICK SUMMARY

- `a..b` — closed range
- `..<` — exclusive end
- `step` / `downTo`
- `in` — membership

try / catch / finally

HANDLING EXCEPTIONS

Kotlin uses `try`, `catch` and `finally` like Java, but `try` is an **expression**: the value of the `try` or `catch` block becomes the result. Catch blocks are checked top to bottom, so put specific exception types before general ones. `finally` always runs, and its value is ignored.

KEY POINTS

- Kotlin exceptions are Java exceptions (`kotlin.Exception` = `java.lang.Exception`)
- No multi-catch `catch (A | B e)` : use `when` inside one catch
- The last expression of try or catch is the value
- Prefer `use { }` over `finally` for closing resources

TRY AS AN EXPRESSION

```
fun parseAge(s: String): Int = try {
    s.toInt()
} catch (e: NumberFormatException) {
    -1                                  // value when parsing fails
}

println(parseAge("42"))                 // 42
println(parseAge("x"))                  // -1

fun readConfig(path: String): String {
    try {
        return File(path).readText()
    } catch (e: FileNotFoundException) {
        println("Missing: ${e.message}")
        return ""
    } catch (e: IOException) {
        throw IllegalStateException("Cannot read $path", e)  // wrap
    } finally {
        println("done reading $path")    // always runs
    }
}

try {
    riskyCall()
} catch (e: Exception) {
    when (e) {                          // instead of multi-catch
        is IllegalArgumentException, is IllegalStateException -> log(e)
        else -> throw e
    }
}
```

CODE EXPLAINED

- → `parseAge` returns the try block's value or -1
- → Specific `FileNotFoundException` comes before general `IOException`
- → Passing `e` as the cause keeps the original stack trace
- → `when` with `is` handles several types in one catch

JAVA EQUIVALENT

- `try { … } catch (NumberFormatException e) { … }` as a statement only
- `catch (IllegalArgumentException | IllegalStateException e)` multi-catch
- try-with-resources `try (var r = …)` ≈ Kotlin `use { }`

IDIOMATIC KOTLIN

- ⇄ `var x; try { x = … } catch { x = … }` → `val x = try { … } catch (…) { … }`
- ⇄ empty catch blocks → log, rethrow, or return a fallback on purpose

COMMON EXCEPTIONS

TYPE	THROWN WHEN
`IllegalArgumentException`	Bad argument (require)
`IllegalStateException`	Bad state (check, error)
`NullPointerException`	!! on null
`NumberFormatException`	toInt() on bad text
`IndexOutOfBoundsException`	Bad index
`NoSuchElementException`	first() on empty
`IOException`	File / network I/O
`ArithmeticException`	Int divide by zero

⚠ COMMON MISTAKES

- Catching `Exception` first so specific handlers never run
- Catching `Throwable` and swallowing errors like OutOfMemoryError
- Returning a value from `finally` (it is ignored as an expression)

✓ TIPS & BEST PRACTICES

- Wrap low-level exceptions with context and the original cause
- Use `toIntOrNull()` style APIs to avoid exceptions for expected failures
- Use `runCatching` for functional style (sheet 35)

QUICK SUMMARY

- `try` — expression
- specific `catch` first
- `finally` — always runs
- wrap with cause

Sheet 33

throw, Nothing & Preconditions

throw as an expression · Nothing type · require / requireNotNull · check / checkNotNull · error() · TODO()

FAILING FAST

`throw` is an expression of type `Nothing`, so it fits anywhere a value is expected, such as after `?:`. The standard library adds precondition helpers: `require` for bad arguments, `check` for bad state, `error()` to throw IllegalStateException, and `TODO()` for unfinished code.

KEY POINTS

- `Nothing` means "never returns"
- `require` throws IllegalArgumentException
- `check` throws IllegalStateException
- Messages are lambdas, built only on failure

PRECONDITIONS AND NOTHING

```
fun withdraw(account: Account, amount: Double) {
    require(amount > 0) { "Amount must be positive: $amount" }
    check(account.isOpen) { "Account ${account.id} is closed" }
    if (amount > account.balance) error("Insufficient funds")
    account.balance -= amount
}

fun findUser(id: Int): User =
    repo.find(id) ?: throw NoSuchElementException("No user $id")

val name: String = requireNotNull(input["name"]) { "name is required" }
val token = checkNotNull(session.token)       // non-null after this

fun fail(msg: String): Nothing = throw IllegalStateException(msg)

val port = config["port"]?.toIntOrNull() ?: fail("bad port")

fun exportPdf(): ByteArray = TODO("PDF export not done yet")
// NotImplementedError: An operation is not implemented: PDF export...
```

CODE EXPLAINED

→ `require` guards arguments; `check` guards object state

→ `throw` after `?:` works because it has type `Nothing`

→ `requireNotNull` returns the value as a non-null type

→ A function returning `Nothing` tells the compiler code after it is unreachable

JAVA EQUIVALENT

☕ `if (amount <= 0) throw new IllegalArgumentException(…)`

☕ `Objects.requireNonNull(x, "msg")`

☕ `throw new UnsupportedOperationException("TODO")`

IDIOMATIC KOTLIN

⇄ `if (!cond) throw IllegalArgumentException(…)` → `require(cond) { … }`

⇄ `x ?: run { throw … }` → `x ?: throw …`

HELPERS

FUNCTION	THROWS
`require(cond) { msg }`	IllegalArgumentException
`requireNotNull(x) { msg }`	IllegalArgumentException
`check(cond) { msg }`	IllegalStateException
`checkNotNull(x) { msg }`	IllegalStateException
`error(msg)`	IllegalStateException
`TODO(reason)`	NotImplementedError

NOTHING

WHERE	EXAMPLE
`throw expression`	x ?: throw E()
`Function return`	fun fail(): Nothing
`TODO()`	Placeholder bodies
`emptyList()`	List<Nothing>

⚠ COMMON MISTAKES

- Using `check` for argument validation (wrong exception type)
- Building expensive messages eagerly instead of in the lambda
- Shipping code with `TODO()` in a reachable path

✓ TIPS & BEST PRACTICES

- Put `require` calls at the top of public functions
- Use `error("…")` for impossible states in `when` branches
- Search for `TODO(` before releases

QUICK SUMMARY

- `require` — arguments
- `check` — state
- `error()` — throw ISE
- `Nothing` — never returns

Custom Exceptions & No Checked Exceptions

`class MyException : Exception()` · no checked exceptions · `@Throws` for Java callers · exception chaining `cause` · sealed error types

YOUR OWN EXCEPTION TYPES

Custom exceptions are ordinary classes extending `Exception` or `RuntimeException`. Kotlin has **no checked exceptions**: you never declare `throws` and callers are never forced to catch. For expected failures, many Kotlin codebases return a **sealed result type** instead of throwing.

KEY POINTS

- Pass `message` and `cause` to the superclass
- Add `@Throws(IOException::class)` if Java code must catch it
- Checked Java exceptions can be thrown and caught freely
- Sealed types make failures part of the return type

UNDER THE JVM

- Checked exceptions are a Java compiler rule; the JVM does not enforce them
- Without `@Throws`, Java callers cannot write `catch (IOException e)` around the call
- Kotlin `Exception` is a typealias for `java.lang.Exception`

EXCEPTIONS VS A SEALED RESULT

```
class InsufficientFundsException(
    val needed: Double,
    cause: Throwable? = null
) : RuntimeException("Need $needed more", cause)

class Account(var balance: Double) {
    @Throws(InsufficientFundsException::class)   // for Java callers
    fun withdraw(amount: Double) {
        if (amount > balance) throw InsufficientFundsException(amount - balance)
        balance -= amount
    }
}

// No throws clause needed, no forced catch:
fun load(path: String): String = File(path).readText()  // may throw IOException

// Expected failure as a value instead:
sealed interface Withdrawal {
    data class Ok(val newBalance: Double) : Withdrawal
    data class Declined(val shortBy: Double) : Withdrawal
}

fun tryWithdraw(acct: Account, amt: Double): Withdrawal =
    if (amt <= acct.balance) Withdrawal.Ok(acct.balance - amt)
    else Withdrawal.Declined(amt - acct.balance)

when (val r = tryWithdraw(Account(50.0), 80.0)) {
    is Withdrawal.Ok       -> println("Left: ${r.newBalance}")
    is Withdrawal.Declined -> println("Short by ${r.shortBy}")  // Short by 30.0
}
```

CODE EXPLAINED

- → The custom exception carries data (`needed`) and an optional cause
- → `@Throws` only affects how Java sees the method
- → `load` can throw IOException without declaring it
- → The sealed result forces callers to handle both outcomes

JAVA EQUIVALENT

- `class InsufficientFundsException extends RuntimeException`
- `void load() throws IOException` is required for checked exceptions
- Sealed interfaces + records (Java 17) for result types

IDIOMATIC KOTLIN

- ⇄ exceptions for normal control flow → sealed result types
- ⇄ catching and rethrowing to satisfy "throws" → just let it propagate

KOTLIN VS JAVA

TOPIC	KOTLIN	JAVA
Checked exceptions	None	Must declare / catch
throws clause	@Throws (interop)	throws IOException
Multi-catch	when inside catch	A | B
Resources	use { }	try-with-resources
Results	Result / sealed types	Optional / exceptions

⚠ COMMON MISTAKES

- Forgetting `@Throws` so Java callers cannot catch a checked exception
- Swallowing exceptions because the compiler no longer forces handling
- Losing the original cause when wrapping

✓ TIPS & BEST PRACTICES

- Throw for programmer errors; return sealed results for expected outcomes
- Document thrown exceptions in KDoc `@throws`
- Keep custom exceptions few and meaningful

QUICK SUMMARY

- extend `Exception`
- no checked exceptions
- `@Throws` — Java interop
- sealed results — expected failures

runCatching & Result

EXCEPTIONS AS VALUES

`runCatching { }` runs a block and wraps the outcome in a `Result<T>`: success with a value, or failure with the exception. You then chain `map`, `onFailure`, `getOrElse` or `fold` instead of writing try/catch. `use { }` closes any `Closeable` resource automatically, even when an exception is thrown.

KEY POINTS

- `runCatching` catches **all** exceptions, including CancellationException
- `Result` has `isSuccess`, `isFailure`, `exceptionOrNull()`
- `getOrThrow()` rethrows the stored exception
- `use` is Kotlin's try-with-resources

RESULT CHAINS AND USE

```
val result: Result<Int> = runCatching { "42x".toInt() }

println(result.isFailure)                       // true
println(result.getOrNull())                     // null
println(result.getOrDefault(0))                 // 0
println(result.getOrElse { e -> -1 })           // -1
println(result.exceptionOrNull()?.message)      // For input string: "42x"

val price = runCatching { fetchPrice("AAPL") }
    .map { it * 1.06 }                          // only on success
    .onFailure { log.warn("price lookup failed", it) }
    .getOrDefault(0.0)

val text = runCatching { File("cfg.txt").readText() }.fold(
    onSuccess = { "Loaded ${it.length} chars" },
    onFailure = { "Using defaults (${it::class.simpleName})" }
)

File("out.txt").bufferedWriter().use { w ->     // closed automatically
    w.write("hello")
}
val firstLine = File("in.txt").bufferedReader().use { it.readLine() }
```

CODE EXPLAINED

→ `map` transforms only successful results

→ `onFailure` runs a side effect and passes the Result along

→ `fold` handles both cases and returns one value

→ `use` closes the writer even if `write` throws

JAVA EQUIVALENT

- `try { … } catch (Exception e) { … }` with a default
- `Optional` chains (`map`, `orElse`) are the closest style
- try-with-resources: `try (var w = Files.newBufferedWriter(p)) { … }`

IDIOMATIC KOTLIN

⇄ try/catch just to return a default → `runCatching { }.getOrDefault(x)`

⇄ `finally { reader.close() }` → `reader.use { }`

RESULT API

MEMBER	DOES
`runCatching { }`	Wrap outcome
`isSuccess / isFailure`	Check
`getOrNull()`	Value or null
`getOrElse { }`	Value or computed
`getOrThrow()`	Value or rethrow
`map { }`	Transform success
`recover { }`	Turn failure into value
`onSuccess / onFailure`	Side effects
`fold(a, b)`	Handle both

⚠ COMMON MISTAKES

- Using `runCatching` inside coroutines and swallowing cancellation
- Catching everything when only one exception is expected
- Forgetting `use` and leaking file handles

✓ TIPS & BEST PRACTICES

- Rethrow `CancellationException` in coroutine code
- Use `runCatching` at boundaries (I/O, parsing), not everywhere
- Use `use` on every stream, reader and connection

QUICK SUMMARY

- `runCatching` — wrap in Result
- `getOrElse` / `getOrDefault`
- `map` / `fold` — chain
- `use { }` — auto-close

Defining Functions

FUNCTIONS IN KOTLIN

Functions are declared with `fun`, parameters as `name: Type`, and the return type after the parentheses. A function that returns no useful value returns `Unit` (Kotlin's `void`), which can be left off. One-line functions can use = and let the compiler infer the return type.

KEY POINTS

- Parameters are read-only (like `val`)
- Block bodies need an explicit return type unless `Unit`
- Functions can be top-level, members, local or extensions
- Use named arguments for clarity at call sites

FUNCTION FORMS

```
fun add(a: Int, b: Int): Int {
    return a + b
}

fun multiply(a: Int, b: Int) = a * b      // expression body, Int inferred

fun greet(name: String): Unit {           // Unit can be omitted
    println("Hello, $name")
}

fun log(msg: String) = println("[LOG] $msg")   // returns Unit

fun isAdult(age: Int) = age >= 18

fun divide(a: Int, b: Int): Int {
    if (b == 0) return 0                  // early return
    return a / b
}

class Calculator {
    fun square(x: Int) = x * x            // member function
}

fun fail(reason: String): Nothing = throw IllegalStateException(reason)

println(add(2, 3))                        // 5
println(multiply(4, 5))                   // 20
println(Calculator().square(6))           // 36
```

CODE EXPLAINED

- → `= a * b` makes an expression body; no braces or `return`
- → `Unit` is a real type with a single value, `Unit`
- → Public API functions should state their return type
- → `Nothing` marks functions that never return normally

JAVA EQUIVALENT

- `int add(int a, int b) { return a + b; }`
- `void greet(String name)` ≈ `fun greet(name: String)`
- All methods live in classes; static methods for utilities

IDIOMATIC KOTLIN

- ⇄ block body with only `return x` → expression body `= x`
- ⇄ static utility classes → top-level functions

FORMS

FORM	EXAMPLE
Block body	fun f(x: Int): Int { return x }
Expression body	fun f(x: Int) = x * 2
Unit return	fun log(s: String) { }
Member	class A { fun f() }
Local	fun outer() { fun inner() }
Extension	fun String.f()
Nothing	fun fail(): Nothing

⚠ COMMON MISTAKES

- Forgetting the return type on a block body that returns a value
- Writing `void` (use `Unit` or nothing)
- Reassigning parameters (they are read-only)

✓ TIPS & BEST PRACTICES

- Use expression bodies for short pure functions
- State return types on public functions
- Keep functions small and named by what they return

QUICK SUMMARY

- `fun name(p: T): R`
- `= expr` — expression body
- `Unit` — no value
- `Nothing` — never returns

Default & Named Arguments

FLEXIBLE PARAMETERS

Parameters can have **default values**, so callers pass only what they need. Callers can also use **named arguments** to make calls readable and to skip defaults in any order. Together they replace most overloaded methods and many builder classes from Java.

KEY POINTS

- Defaults can refer to earlier parameters
- After a named argument, later ones should be named too
- Named arguments do not work when calling Java methods
- `@JvmOverloads` generates Java overloads for defaults

DEFAULTS AND NAMED CALLS

```
fun createUser(
    name: String,
    email: String = "",
    age: Int = 0,
    isAdmin: Boolean = false,
    tags: List<String> = emptyList()
) = User(name, email, age, isAdmin, tags)

createUser("Ada")
createUser("Ada", "ada@example.com")
createUser("Bob", isAdmin = true)                  // skip email, age
createUser(name = "Cy", age = 30, email = "cy@x.io")   // any order

fun connect(host: String, port: Int = if (host == "localhost") 8080 else 443) =
    "$host:$port"
println(connect("localhost"))                      // localhost:8080
println(connect("api.example.com"))                // api.example.com:443

class Greeter @JvmOverloads constructor(
    val greeting: String = "Hello",
    val punctuation: String = "!"
)
// Java sees: Greeter(), Greeter(String), Greeter(String, String)

drawRect(x = 10, y = 20, width = 100, height = 50)  // readable booleans/ints
```

CODE EXPLAINED

→ One function covers what would be several Java overloads

→ `isAdmin = true` skips `email` and `age`

→ A default can be computed from another parameter

→ `@JvmOverloads` helps Java callers who cannot use defaults

JAVA EQUIVALENT

- Several overloads: `createUser(String)`, `createUser(String, String)` …
- Builder pattern: `User.builder().name("Ada").isAdmin(true).build()`
- Java has no named arguments

IDIOMATIC KOTLIN

⇄ telescoping overloads → one function with defaults

⇄ `f(true, false, 3)` → `f(force = true, dryRun = false, retries = 3)`

CALL STYLES

CALL	NOTES
`f("a")`	Use all defaults
`f("a", "b")`	Positional
`f("a", age = 3)`	Positional, then named
`f(age = 3, name = "a")`	All named, any order
`f(*array)`	Spread into vararg

⚠ COMMON MISTAKES

- Mixing named then positional arguments in confusing order
- Expecting named arguments when calling Java APIs
- Changing default values in a library without considering callers

✓ TIPS & BEST PRACTICES

- Name Boolean and numeric arguments at call sites
- Use data class `copy()` for "same but changed" (sheet 61)
- Add `@JvmOverloads` on APIs Java code will call

QUICK SUMMARY

- defaults — fewer overloads
- named args — readable calls
- any order with names
- `@JvmOverloads` — Java interop

vararg & Spread

VARIABLE NUMBERS OF ARGUMENTS

A `vararg` parameter accepts any number of arguments, which arrive as an array inside the function. To pass an existing array, use the **spread operator** `*`. Only one parameter can be `vararg`; if it is not last, the parameters after it must be passed by name.

KEY POINTS

- `vararg x: Int` becomes `IntArray` inside
- `vararg x: String` becomes `Array<out String>`
- Spread a List with `*list.toTypedArray()`
- You can mix spread and individual values

VARARG AND SPREAD

```
fun sum(vararg nums: Int): Int = nums.sum()

println(sum())                  // 0
println(sum(1, 2, 3))           // 6

val arr = intArrayOf(4, 5, 6)
println(sum(*arr))              // 15
println(sum(1, *arr, 10))       // 26  (mix values and spread)

fun log(level: String, vararg parts: Any) {
    println("[$level] " + parts.joinToString(" "))
}
log("INFO", "User", 42, "logged in")    // [INFO] User 42 logged in

fun <T> listOfNonNull(vararg items: T?): List<T> = items.filterNotNull()
println(listOfNonNull("a", null, "b"))  // [a, b]

val names = listOf("x", "y")
val all = listOf("w", *names.toTypedArray(), "z")   // [w, x, y, z]

fun format(vararg values: Int, sep: String) = values.joinToString(sep)
println(format(1, 2, 3, sep = "-"))     // 1-2-3 (sep must be named)
```

CODE EXPLAINED

→ Inside `sum`, `nums` is an `IntArray`

→ `*arr` spreads array elements into the call

→ A List must become an array before spreading

→ A parameter after the vararg must be named

JAVA EQUIVALENT

☕ `int sum(int... nums)`

☕ Passing an array directly works in Java (no spread operator)

☕ `String.format("%s %s", args)` uses Object varargs

IDIOMATIC KOTLIN

⇄ overloads for 1, 2, 3 arguments → one `vararg`

⇄ passing a List to vararg → `*list.toTypedArray()`

VARARG TYPES

DECLARATION	INSIDE FUNCTION
`vararg x: Int`	IntArray
`vararg x: Long`	LongArray
`vararg x: String`	Array<out String>
`vararg x: T`	Array<out T>

SPREAD

CALL	MEANING
`f(*arr)`	Pass array elements
`f(1, *arr)`	Mix with values
`f(*list.toTypedArray())`	From a List
`f(*intList.toIntArray())`	List<Int> → IntArray

⚠ COMMON MISTAKES

- Passing an array without `*` (it becomes one element)
- Two vararg parameters (not allowed)
- Forgetting to name parameters after the vararg

✓ TIPS & BEST PRACTICES

- Put `vararg` last when possible
- Accept `Iterable<T>` too if callers often have lists
- Use `vararg` for builder-style APIs like `listOf`

QUICK SUMMARY

- `vararg` — many arguments
- `*arr` — spread
- array inside function
- params after → named

Extension Functions & Properties

`fun String.shout()` · `this` receiver · extension properties · nullable receivers · resolved statically · extensions on companions

ADDING FUNCTIONS TO EXISTING TYPES

An **extension function** adds a function to a type you do not own, such as `String` or a Java class, without inheritance. Inside, `this` is the receiver object. Extensions are resolved **statically** by the declared type, and they cannot reach private members.

KEY POINTS

- Most of the Kotlin standard library is extensions
- Extensions on nullable types can be called on null
- Members win over extensions with the same signature
- Extension properties need a getter; they have no backing field

UNDER THE JVM

- `fun String.shout()` compiles to `static String shout(String $this)` in `FileNameKt`
- No method is added to the real class
- Static dispatch: the declared type picks the extension, not the runtime type

EXTENSIONS IN ACTION

```
fun String.shout(): String = uppercase() + "!"          // this = the String
println("hello".shout())                                 // HELLO!

fun Int.isEven() = this % 2 == 0
println(4.isEven())                                      // true

val String.wordCount: Int                                // extension property
    get() = trim().split(Regex("\\s+")).size
println("one two three".wordCount)                       // 3

fun String?.orDash(): String = if (isNullOrBlank()) "-" else this
val missing: String? = null
println(missing.orDash())                                // -

fun <T> List<T>.second(): T = this[1]
println(listOf(1, 2, 3).second())                        // 2

open class Shape; class Circle : Shape()
fun Shape.name() = "shape"
fun Circle.name() = "circle"
val s: Shape = Circle()
println(s.name())                                        // shape (static type)

class Temp { companion object }
fun Temp.Companion.fromF(f: Double) = (f - 32) * 5 / 9
println(Temp.fromF(212.0))                               // 100.0
```

CODE EXPLAINED

- → Call extensions like members: `"hello".shout()`
- → A nullable receiver lets `missing.orDash()` work on null
- → Generic extensions work on any `List<T>`
- → `s.name()` uses the declared type `Shape`, not the runtime `Circle`

JAVA EQUIVALENT

- Static helpers: `StringUtils.shout(s)`
- Utility classes like `Collections.sort(list)`
- Java has no extension methods

IDIOMATIC KOTLIN

- ⇄ `Utils.format(date)` static helpers → `date.format()` extensions
- ⇄ extensions that need private state → member functions

EXTENSION KINDS

KIND	EXAMPLE
`Function`	fun String.shout()
`Property`	val String.wordCount get() = …
`Nullable receiver`	fun String?.orDash()
`Generic`	fun <T> List<T>.second()
`Companion`	fun Temp.Companion.fromF()
`Member extension`	Declared inside a class

⚠ COMMON MISTAKES

- Expecting virtual dispatch on extensions
- Defining an extension that a member already provides (member wins)
- Scattering extensions across many files with no structure

✓ TIPS & BEST PRACTICES

- Group extensions by receiver type in files like `StringExt.kt`
- Use nullable receivers for safe helpers like `orEmpty`
- Keep extensions small and obvious

QUICK SUMMARY

- `fun T.f()` — add function
- `this` — receiver
- static resolution
- compiled as static methods

Sheet 40

Infix, Local & tailrec Functions

infix fun · to pairs · local functions · tailrec · recursion · Nothing-returning helpers

SPECIAL FUNCTION FORMS

`infix` functions are called without dots or parentheses: `a to b`. **Local functions** are declared inside another function and can read its variables. `tailrec` tells the compiler to turn a tail-recursive function into a loop, so deep recursion does not overflow the stack.

KEY POINTS

- Infix functions: one parameter, member or extension
- `to` creates a `Pair`: `"a" to 1`
- Local functions keep helpers private to one function
- `tailrec` requires the recursive call to be the very last operation

UNDER THE JVM

- `tailrec` compiles to a `while` loop with no extra stack frames
- Local functions capturing variables compile to helper methods or closure objects
- Infix calls compile to ordinary method calls

INFIX, LOCAL AND TAILREC

```
infix fun Int.percentOf(total: Int) = this * 100.0 / total
println(25 percentOf 200)              // 12.5

val pair = "Ada" to 36                 // Pair("Ada", 36)
val map = mapOf("a" to 1, "b" to 2)

data class Money(val cents: Long) {
    infix fun split(ways: Int) = Money(cents / ways)
}
println(Money(1000) split 4)           // Money(cents=250)

fun validate(user: User): List<String> {
    val errors = mutableListOf<String>()
    fun check(ok: Boolean, msg: String) { if (!ok) errors += msg }  // local
    check(user.name.isNotBlank(), "name required")
    check(user.age >= 0, "age must be >= 0")
    return errors
}

tailrec fun gcd(a: Int, b: Int): Int = if (b == 0) a else gcd(b, a % b)
println(gcd(48, 18))                   // 6

tailrec fun countDown(n: Long): Long = if (n == 0L) 0 else countDown(n - 1)
println(countDown(1_000_000))          // 0, no StackOverflowError

// not tail recursive: multiplies after the call
fun factorial(n: Int): Long = if (n <= 1) 1 else n * factorial(n - 1)
```

CODE EXPLAINED

- → `25 percentOf 200` reads like English
- → The local `check` can add to `errors` from the outer function
- → `gcd` calls itself as the last step, so `tailrec` applies
- → `factorial` multiplies after the call, so it is not tail recursive

JAVA EQUIVALENT

- No infix calls: `Pair.of("Ada", 36)` or `Map.entry("a", 1)`
- No local functions; use a private method or a lambda
- No tail-call optimization in Java; write a loop

IDIOMATIC KOTLIN

- ⇄ private helpers used by one function → local functions
- ⇄ deep recursion → `tailrec` or a loop

RULES

FEATURE	REQUIREMENT
`infix`	Member/extension, one param, no default/vararg
`local fun`	Inside another function
`tailrec`	Recursive call is the last operation
`tailrec`	Not inside try/catch/finally
`to`	Built-in infix for Pair

⚠ COMMON MISTAKES

- Adding `tailrec` to a non-tail function (warning, no optimization)
- Overusing infix for things that read worse without dots
- Recursing deeply without `tailrec` (StackOverflowError)

✓ TIPS & BEST PRACTICES

- Use infix for DSL-like code (`shouldBe`, `to`, `until`)
- Use local functions to avoid passing many parameters
- Watch for the compiler warning on invalid `tailrec`

QUICK SUMMARY

- `infix` — `a op b` calls
- `to` — make a Pair
- local functions — private helpers
- `tailrec` — recursion as a loop

Lambdas

FUNCTION LITERALS

A **lambda** is a function written as a value in braces: parameters, an arrow, then the body. A single parameter can be called `it`. When a lambda is the last argument, it goes **outside** the parentheses (trailing lambda), which is why Kotlin code reads like `list.filter { it > 0 }`.

KEY POINTS

- The last expression is the lambda's return value
- Omit `()` entirely if the lambda is the only argument
- Use `_` for parameters you do not need
- Lambdas can read and change outer `var`s

UNDER THE JVM

- Lambdas compile via `invokedynamic` (LambdaMetafactory) by default since Kotlin 2.0
- Captured `var`s are wrapped in `Ref` objects so they can change
- Passing a lambda to an `inline` function creates no object at all (sheet 43)

LAMBDA SYNTAX

```
val double: (Int) -> Int = { x -> x * 2 }
val add = { a: Int, b: Int -> a + b }
val greet: () -> String = { "Hi" }
println(double(4))                          // 8
println(add(2, 3))                          // 5

val nums = listOf(3, -1, 4, -5)
println(nums.filter { it > 0 })             // [3, 4] (it = the element)
println(nums.map { n -> n * n })            // [9, 1, 16, 25]

println(nums.fold(0) { acc, n -> acc + n })   // 1 (trailing lambda)

val map = mapOf("a" to 1, "b" to 2)
map.forEach { (key, value) -> println("$key=$value") }  // destructuring
map.forEach { (_, value) -> println(value) }            // ignore key

val grade = { score: Int ->
    val bonus = 5
    if (score + bonus >= 90) "A" else "B"   // last expression returned
}
println(grade(87))                          // A

var clicks = 0
val onClick = { clicks++ }                  // captures and changes clicks
onClick(); onClick()
println(clicks)                             // 2
```

CODE EXPLAINED

→ Function types are written `(Int) -> Int`

→ `it` is the default name for a single parameter

→ `fold(0) { … }`: initial value in parentheses, lambda outside

→ Lambdas can capture and modify local `var`s, unlike Java

JAVA EQUIVALENT

- `Function<Integer, Integer> d = x -> x * 2;`
- `list.stream().filter(n -> n > 0).toList()`
- Java lambdas can only capture effectively final variables

IDIOMATIC KOTLIN

⇄ `list.filter({ it > 0 })` → `list.filter { it > 0 }`

⇄ nested lambdas all using `it` → name the parameters

SYNTAX

FORM	MEANING
`{ x -> x * 2 }`	One parameter
`{ it * 2 }`	Implicit it
`{ a, b -> a + b }`	Two parameters
`{ (k, v) -> … }`	Destructure a Pair/entry
`{ _, v -> v }`	Ignore a parameter
`{ }`	No parameters
`f(1) { … }`	Trailing lambda

⚠ COMMON MISTAKES

- Using `it` in nested lambdas so it is unclear which `it`
- Writing `return` in a lambda and leaving the enclosing function
- Confusing `{ (a, b) -> }` (destructure one arg) with `{ a, b -> }` (two args)

✓ TIPS & BEST PRACTICES

- Name lambda parameters when the body is longer than one line
- Use function references (sheet 45) when a function already exists
- Keep lambdas short; extract named functions

QUICK SUMMARY

- `{ x -> body }` — lambda
- `it` — single param
- trailing lambda — outside `()`
- last expression — result

Sheet 42

Higher-Order Functions & Function Types

`(Int) -> Int` · nullable function types · `::functionRef` · returning functions · `T.() -> Unit` receivers · `typealias` for function types

FUNCTIONS THAT TAKE OR RETURN FUNCTIONS

A **higher-order function** takes another function as a parameter or returns one. Function types are written `(A, B) -> R`. A function type with a **receiver**, `T.() -> R`, lets the lambda use `this` as a T, which is how builders like `apply` and DSLs work.

KEY POINTS

- `() -> Unit` takes nothing and returns nothing useful
- `((Int) -> Int)?` is a nullable function; call with `f?.invoke(x)`
- Function references (`::name`) can be passed where a function type is expected
- Name long function types with `typealias`

TAKING AND RETURNING FUNCTIONS

```
fun applyTwice(x: Int, f: (Int) -> Int): Int = f(f(x))
println(applyTwice(3) { it * 2 })           // 12

fun makeMultiplier(factor: Int): (Int) -> Int = { it * factor }
val triple = makeMultiplier(3)
println(triple(5))                          // 15

fun measure(label: String, block: () -> Unit) {
    val start = System.nanoTime()
    block()
    println("$label took ${(System.nanoTime() - start) / 1_000_000} ms")
}
measure("sort") { (1..100_000).shuffled().sorted() }

fun retry(
    times: Int,
    onError: ((Exception) -> Unit)? = null,
    action: () -> Unit
) {
    repeat(times) {
        try { action(); return } catch (e: Exception) { onError?.invoke(e) }
    }
}

fun html(build: StringBuilder.() -> Unit): String =
    StringBuilder().apply(build).toString()
println(html { append("<b>"); append("hi"); append("</b>") })  // <b>hi</b>

typealias Validator = (String) -> Boolean
val notBlank: Validator = { it.isNotBlank() }
fun isEven(n: Int) = n % 2 == 0
println(listOf(1, 2, 3).filter(::isEven))   // [2]
```

CODE EXPLAINED

- → `applyTwice` receives a function and calls it twice
- → `makeMultiplier` returns a lambda that remembers `factor`
- → Inside `html { … }`, `this` is the StringBuilder
- → `::isEven` passes an existing function

JAVA EQUIVALENT

- `Function<Integer,Integer>`, `Supplier<T>`, `Runnable`, `BiFunction`
- Returning lambdas: `IntUnaryOperator mult(int f) { return x -> x * f; }`
- `list.stream().filter(Main::isEven)`

IDIOMATIC KOTLIN

- ⇄ `(String) -> Boolean` repeated everywhere → `typealias Validator`
- ⇄ interfaces with one method for callbacks → function types

FUNCTION TYPES

TYPE	MEANING
`() -> Unit`	No args, no result
`(Int) -> String`	Int in, String out
`(A, B) -> C`	Two args
`((Int) -> Int)?`	Nullable function
`T.() -> Unit`	With receiver
`suspend () -> T`	Suspending function

JAVA EQUIVALENTS

KOTLIN	JAVA
`() -> Unit`	Runnable
`() -> T`	Supplier<T>
`(T) -> Unit`	Consumer<T>
`(T) -> R`	Function<T,R>
`(T) -> Boolean`	Predicate<T>

⚠ COMMON MISTAKES

- Calling a nullable function type directly (needs `?.invoke`)
- Overly long function type signatures with no alias
- Returning lambdas that capture large objects by accident

✓ TIPS & BEST PRACTICES

- Put the function parameter last so callers can use trailing lambdas
- Mark small higher-order functions `inline` (sheet 43)
- Use receivers for builder-style APIs

QUICK SUMMARY

- `(A) -> B` — function type
- take / return functions
- `T.() -> R` — with receiver
- `typealias` — name it

inline & reified

`inline fun` · `noinline` · `crossinline` · non-local returns · `reified T` · how `filterIsInstance<T>()` works

ZERO-COST LAMBDAS AND REAL GENERICS

An `inline` function has its body, and the lambdas passed to it, copied into each call site, so no function object is created. That also allows **non-local returns** from the lambda. Inline functions can declare `reified` type parameters, which keep the actual type at runtime, so `is T` and `T::class` work.

KEY POINTS

- Most collection functions (`map`, `filter`, `forEach`) are inline
- `noinline` keeps one lambda as a real object (to store or pass on)
- `crossinline` forbids non-local returns from a lambda
- Only inline small functions; big bodies bloat code

UNDER THE JVM

- The inline function body is copied into the caller's bytecode
- Reified type arguments are written in as concrete classes at each call site
- Inline functions cannot be called from Java with reified types (they need Kotlin's compiler)

INLINE AND REIFIED

```
inline fun <T> measure(label: String, block: () -> T): T {
    val start = System.nanoTime()
    val result = block()                    // block is inlined, no object
    println("$label: ${(System.nanoTime() - start) / 1000} µs")
    return result
}

fun findFirstNegative(nums: List<Int>): Int? {
    nums.forEach { if (it < 0) return it }  // non-local return
    return null
}

inline fun <reified T> List<Any>.ofType(): List<T> =
    filter { it is T }.map { it as T }
val mixed = listOf(1, "a", 2.5, "b")
println(mixed.ofType<String>())             // [a, b]
println(mixed.filterIsInstance<Double>())   // [2.5] (stdlib, also reified)

inline fun <reified T : Any> typeName() = T::class.simpleName
println(typeName<LocalDate>())              // LocalDate

inline fun runTwice(first: () -> Unit, noinline second: () -> Unit) {
    first()
    scheduler.post(second)                  // stored, so it must be noinline
}

inline fun onClick(crossinline action: () -> Unit) {
    button.setListener { action() }         // runs later: no non-local return
}

// Gson / Jackson style:
inline fun <reified T> Gson.fromJson(json: String): T =
    fromJson(json, T::class.java)
```

CODE EXPLAINED

- → `measure` adds timing without allocating a lambda object
- → `return it` leaves `findFirstNegative` because `forEach` is inline
- → `reified T` makes `it is T` and `T::class` legal
- → `noinline` and `crossinline` handle lambdas that run later or elsewhere

JAVA EQUIVALENT

- Lambdas always allocate (or are cached) objects in Java
- Generics are erased: you must pass `Class<T> type` explicitly
- `gson.fromJson(json, User.class)`

IDIOMATIC KOTLIN

- ⇄ passing `Class<T>` parameters → `inline fun <reified T>`
- ⇄ marking big functions `inline` → only small higher-order functions

MODIFIERS

MODIFIER	EFFECT
`inline`	Copy body + lambdas to call site
`noinline`	Keep this lambda as an object
`crossinline`	Inlined, but no non-local return
`reified`	Type available at runtime
`inline class (old)`	Now @JvmInline value class

⚠ COMMON MISTAKES

- Inlining large functions with no lambda parameters (warning, no benefit)
- Storing an inline lambda in a field without `noinline`
- Expecting reified types to work from Java callers

✓ TIPS & BEST PRACTICES

- Use `reified` for JSON, DI and logging helpers that need `T::class`
- Read the "expected performance impact is insignificant" warning: it means remove `inline`
- Prefer stdlib inline functions (`map`, `let`) over hand-written loops

QUICK SUMMARY

- `inline` — no lambda objects
- non-local `return` — allowed
- `reified T` — runtime type
- `noinline` / `crossinline`

Scope Functions

LET, RUN, WITH, APPLY, ALSO

Scope functions run a block in the context of an object. They differ in two ways: whether the object is `this` (receiver) or `it` (argument), and whether they return the **object** or the **lambda result**. `apply` configures and returns the object; `let` transforms or handles nullable values.

KEY POINTS

- `apply` / `also` return the object itself
- `let` / `run` / `with` return the lambda result
- `this` -style (`apply` , `run` , `with`) for calling members
- `takeIf { }` returns the object or null

EACH SCOPE FUNCTION

```
data class User(var name: String = "", var email: String = "", var age: Int = 0)

val user = User().apply {                // configure, returns User
    name = "Ada"
    email = "ada@example.com"
}

val nameLength = user.name.let { it.length }       // 3 (result)
val maybe: String? = findEmail()
maybe?.let { sendWelcome(it) }                     // only if not null

val summary = user.run { "$name <$email>" }        // this = user, result

val report = with(StringBuilder()) {               // non-extension form
    append("Users: ")
    append(1)
    toString()
}

val list = mutableListOf(1, 2).also { println("created $it") }   // side effect

val adult = user.takeIf { it.age >= 18 }           // null here (age 0)
val name = user.name.takeUnless { it.isBlank() } ?: "anonymous"

val conn = Connection(url)
    .apply { timeout = 5000 }
    .also { log.info("conn ready") }
```

CODE EXPLAINED

→ `apply` is the go-to for initializing objects

→ `?.let` is the classic "if not null" block

→ `run` and `with` compute a value using the object's members

→ `also` adds logging or validation without changing the chain

JAVA EQUIVALENT

☕ `User u = new User(); u.setName("Ada"); u.setEmail(…);`

☕ `Optional.ofNullable(email).ifPresent(this::sendWelcome)`

☕ No direct equivalents; plain local variables

IDIOMATIC KOTLIN

⇄ repeated `user.` prefixes while configuring → `user.apply { }`

⇄ nested scope functions three deep → plain local `val`s

CHOOSING

FUNCTION	OBJECT AS	RETURNS	TYPICAL USE
`let`	`it`	Lambda result	Null checks, transform
`run`	`this`	Lambda result	Compute from members
`with(x)`	`this`	Lambda result	Group calls on x
`apply`	`this`	The object	Configure / build
`also`	`it`	The object	Side effects, logging
`takeIf`	`it`	Object or null	Filter one value

⚠ COMMON MISTAKES

- Using `apply` when you need the lambda result (you get the object back)
- Chaining many scope functions until the code is unreadable
- Shadowing `this` in nested `apply` blocks

✓ TIPS & BEST PRACTICES

- Default to `apply` for setup and `?.let` for null handling
- Use `also` for logging inside call chains
- If you hesitate, a plain `val` is often clearer

QUICK SUMMARY

- `apply` — configure, return object
- `let` — `it`, return result
- `also` — side effect
- `run` / `with` — `this`, result

Sheet 45

Function References & SAM Conversions

`::method` · bound references `obj::method` · constructor references `::Person` · `fun interface` · SAM conversion for Java interfaces

PASSING EXISTING FUNCTIONS

A **function reference** (`::name`) turns an existing function into a value you can pass, instead of wrapping it in a lambda. References can point to top-level functions, members (`String::length`), bound objects (`printer::print`) and constructors (`::User`). **SAM conversion** lets a lambda stand in for any single-abstract-method interface.

KEY POINTS

- `Type::member` is unbound: the receiver becomes the first argument
- `obj::member` is bound to that object
- Java functional interfaces accept Kotlin lambdas automatically
- Kotlin interfaces need `fun interface` for SAM conversion

REFERENCES AND SAM

```
fun isEven(n: Int) = n % 2 == 0
val nums = listOf(1, 2, 3, 4)

println(nums.filter(::isEven))                  // [2, 4]
println(listOf("a", "bbb").map(String::length)) // [1, 3]

val prefix = "ID-"
println(nums.map(prefix::plus))                 // [ID-1, ID-2, ID-3, ID-4]

data class User(val name: String)
println(listOf("Ada", "Bob").map(::User))  // [User(name=Ada), User(name=Bob)]

val ref: (Int) -> Boolean = ::isEven
println(ref(10))                                // true

val prop = User::name                           // property reference
println(prop.get(User("Cy")))                   // Cy

// SAM conversion with Java interfaces
val runnable = Runnable { println("running") }
val comparator = Comparator<String> { a, b -> a.length - b.length }
executor.submit { println("task") }

// Kotlin fun interface
fun interface PriceRule { fun apply(price: Double): Double }
val tenOff = PriceRule { it * 0.9 }
println(tenOff.apply(100.0))                    // 90.0
```

CODE EXPLAINED

- → `::isEven` passes the function without a lambda wrapper
- → `String::length` takes the String as its argument
- → `::User` calls the constructor for each name
- → `fun interface` enables lambda syntax for a Kotlin interface

JAVA EQUIVALENT

- `list.stream().filter(Main::isEven)`
- `String::length`, `User::new` (constructor reference)
- Any `@FunctionalInterface` accepts a lambda

IDIOMATIC KOTLIN

- ⇄ `map { it.length }` → `map(String::length)` when it reads better
- ⇄ single-method Kotlin interfaces without `fun` → `fun interface` (or a function type)

REFERENCE KINDS

FORM	REFERS TO
`::topLevelFun`	Top-level function
`Type::member`	Member, receiver as arg
`obj::member`	Bound member
`::Class`	Constructor
`Type::property`	Property (KProperty)
`this::method`	Bound to this

⚠ COMMON MISTAKES

- Ambiguous references to overloaded functions (add a type)
- Forgetting `fun` on an interface and wondering why lambdas do not convert
- Using references where a short lambda is clearer

✓ TIPS & BEST PRACTICES

- Use constructor references in `map(::Dto)` conversions
- Use bound references for callbacks on a specific object
- Prefer function types over `fun interface` unless Java interop or naming matters

QUICK SUMMARY

- `::f` — reference a function
- `Type::m` / `obj::m`
- `::Class` — constructor
- `fun interface` — SAM for Kotlin

Lists

ORDERED COLLECTIONS

Kotlin separates **read-only** `List` from `MutableList`. `listOf()` creates a read-only list and `mutableListOf()` one you can change. Index with `list[i]`; use `getOrNull(i)` to avoid exceptions. Read-only means no mutating methods on that reference, not guaranteed immutability.

KEY POINTS

- Both are `java.util.List` at runtime
- `List<T>` is covariant: `List<Int>` is a `List<Number>`
- `first()` / `last()` throw on empty; `firstOrNull()` does not
- `buildList { }` creates a list with a mutable builder

CREATING AND READING LISTS

```
val fruits = listOf("apple", "pear", "fig")     // List<String>
val empty = emptyList<Int>()
val nums = mutableListOf(1, 2, 3)               // MutableList<Int>

println(fruits[0])                               // apple
println(fruits.size)                             // 3
println(fruits.first())                          // apple
println(fruits.last())                           // fig
println(fruits.getOrNull(5))                     // null
println(fruits.getOrElse(5) { "none" })          // none
println(fruits.indexOf("pear"))                  // 1
println("fig" in fruits)                         // true
println(fruits.indices)                          // 0..2
println(fruits.lastIndex)                        // 2
println(empty.firstOrNull())                     // null
// fruits.add("kiwi")                            // error: List is read-only

nums.add(4)
nums[0] = 10
println(nums)                                    // [10, 2, 3, 4]

val squares = List(5) { it * it }                // [0, 1, 4, 9, 16]
val readOnly: List<Int> = nums                   // same list, read-only view
```

CODE EXPLAINED

→ `listOf` has no `add`; `mutableListOf` does

→ `getOrElse` supplies a value for a bad index

→ `List(5) { … }` builds a list from an index function

→ A `List` reference to a mutable list still sees its changes

JAVA EQUIVALENT

- `List.of("apple", "pear")` (immutable, throws on add)
- `new ArrayList<>(List.of(1, 2, 3))` for a mutable list
- `list.get(0)` instead of `list[0]`

IDIOMATIC KOTLIN

⇄ exposing `MutableList` from a class → expose `List`

⇄ `list.get(i)` with bounds checks → `getOrNull(i)`

CREATING

FUNCTION	RESULT
`listOf(a, b)`	Read-only List
`mutableListOf()`	MutableList (ArrayList)
`arrayListOf()`	ArrayList
`emptyList()`	Empty List
`List(n) { }`	Generated
`buildList { add(…) }`	Built read-only

READING

CALL	ON MISSING
`list[i]`	IndexOutOfBounds
`getOrNull(i)`	null
`first() / last()`	NoSuchElement
`firstOrNull()`	null
`indexOf(x)`	-1

⚠ COMMON MISTAKES

- Assuming `listOf` lists are deeply immutable
- Calling `first()` on a possibly empty list
- Casting a `List` to `MutableList` to modify it

✓ TIPS & BEST PRACTICES

- Use read-only types in function signatures
- Use `buildList` to construct lists with conditions
- Use `orEmpty()` for nullable lists

QUICK SUMMARY

- `listOf` — read-only
- `mutableListOf` — changeable
- `getOrNull` — safe index
- `List(n) { }` — generate

List Operations

CHANGING AND COMBINING LISTS

Mutable lists support `add`, `remove`, `removeAt`, `removeIf` and index assignment. Read-only lists can still be **combined** with + and –, which return new lists. `toMutableList()` makes a changeable copy, and `buildList` gives a mutable builder that ends as a read-only list.

KEY POINTS

- `+` and `-` never change the original list
- `remove(x)` removes the first match; `removeAt(i)` by index
- `subList` is a view: changes show through
- `+=` on a `val` MutableList adds in place

MUTATING AND COMBINING

```
val nums = mutableListOf(5, 3, 8, 3)
nums.add(1)                     // [5, 3, 8, 3, 1]
nums.add(0, 9)                  // insert at 0 → [9, 5, 3, 8, 3, 1]
nums.addAll(listOf(7, 7))       // [9, 5, 3, 8, 3, 1, 7, 7]
nums.remove(3)                  // first 3 removed
nums.removeAt(0)                // removes 9
nums.removeIf { it == 7 }       // [5, 8, 3, 1]
nums[1] = 80                    // [5, 80, 3, 1]
nums += 2                       // [5, 80, 3, 1, 2]
nums.sort()                     // in place → [1, 2, 3, 5, 80]
println(nums.subList(1, 3))     // [2, 3]

val base = listOf("a", "b")
val more = base + "c"           // [a, b, c] (new list)
val joined = base + listOf("x", "y")
val less = joined - "x"         // [a, b, y]
println(base)                   // [a, b] unchanged

val copy = base.toMutableList().apply { add("z") }   // [a, b, z]

val menu = buildList {
    add("Coffee")
    if (isMorning) add("Bagel")
    addAll(specials)
}
nums.clear()                    // []
```

CODE EXPLAINED

- → `add(index, x)` inserts and shifts later elements
- → `removeIf` removes every match in one pass
- → `base + "c"` creates a new list; `base` stays the same
- → `buildList` is ideal for conditional contents

JAVA EQUIVALENT

- `list.add`, `list.remove(Integer.valueOf(3))`, `list.removeIf(…)`
- Combining: `Stream.concat(a.stream(), b.stream()).toList()`
- `new ArrayList<>(base)` to copy

IDIOMATIC KOTLIN

- ⇄ copy + add + return → `base + item`
- ⇄ mutable list built step by step → `buildList { }`

MUTATING (MUTABLELIST)

CALL	EFFECT
`add(x) / add(i, x)`	Append / insert
`addAll(xs)`	Append many
`remove(x)`	First match
`removeAt(i)`	By index
`removeIf { }`	All matches
`retainAll { }`	Keep matches
`sort() / shuffle()`	In place
`clear()`	Empty it

NEW LISTS

CALL	RETURNS
`list + x`	Added
`list - x`	Removed
`list + other`	Concatenated
`toMutableList()`	Mutable copy
`sorted()`	Sorted copy

⚠ COMMON MISTAKES

- `nums.remove(3)` on a MutableList<Int> removes the value 3, not index 3
- Expecting `sorted()` to sort in place (use `sort()`)
- Changing a list while iterating it

✓ TIPS & BEST PRACTICES

- Use `removeAt` for index removal to be explicit
- Prefer `+` / `-` on read-only lists for clarity
- Use `sortedBy` for new sorted lists (sheet 54)

QUICK SUMMARY

- `add` / `remove` / `removeAt`
- `+` / `-` — new lists
- `toMutableList()` — copy
- `sort()` vs `sorted()`

Sets

UNIQUE ELEMENTS

A **set** holds each element once. `setOf()` and `mutableSetOf()` create sets that keep insertion order (`LinkedHashSet`); `hashSetOf()` gives no order guarantee and `sortedSetOf()` keeps elements sorted. Set operations `union`, `intersect` and `subtract` are infix functions.

KEY POINTS

- Uniqueness uses `equals` and `hashCode`
- `add` returns false if the element was already there
- `toSet()` is a quick way to remove duplicates
- `in` checks are fast (hash-based)

SET BASICS AND OPERATIONS

```
val tags = mutableSetOf("kotlin", "jvm")
println(tags.add("kotlin"))          // false (already there)
println(tags.add("android"))         // true
println(tags)                        // [kotlin, jvm, android] (insertion order)
println("jvm" in tags)               // true
tags.remove("jvm")

val a = setOf(1, 2, 3, 4)
val b = setOf(3, 4, 5)
println(a union b)                   // [1, 2, 3, 4, 5]
println(a intersect b)               // [3, 4]
println(a subtract b)                // [1, 2]
println(a + 9)                       // [1, 2, 3, 4, 9]
println(a.containsAll(listOf(1, 2))) // true

val emails = listOf("a@x.com", "b@x.com", "a@x.com")
println(emails.toSet())              // [a@x.com, b@x.com]
println(emails.distinct())           // same, but stays a List

val sorted = sortedSetOf(5, 1, 3)    // TreeSet
println(sorted)                      // [1, 3, 5]
println(sorted.first())              // 1

data class Point(val x: Int, val y: Int)
println(setOf(Point(1, 2), Point(1, 2)).size)   // 1 (data class equals)
```

CODE EXPLAINED

→ `add` reports whether the set changed

→ `union`, `intersect`, `subtract` read naturally as infix calls

→ `distinct()` dedups but keeps a List

→ Data classes work in sets because equals/hashCode are generated

JAVA EQUIVALENT

☕ `Set.of(…)` (immutable), `new HashSet<>()`, `new LinkedHashSet<>()`

☕ `set.addAll(b)` / `retainAll(b)` / `removeAll(b)` mutate in place

☕ `new TreeSet<>(List.of(5, 1, 3))`

IDIOMATIC KOTLIN

⇄ `list.contains` in a loop on big lists → convert to a Set first

⇄ mutating `retainAll` for intersection → `a intersect b`

CREATING

FUNCTION	ORDER
`setOf()`	Insertion (LinkedHashSet)
`mutableSetOf()`	Insertion
`hashSetOf()`	None
`linkedSetOf()`	Insertion
`sortedSetOf()`	Sorted (TreeSet)

OPERATIONS

CALL	RESULT
`a union b`	All elements
`a intersect b`	In both
`a subtract b`	In a only
`a + x / a - x`	New set
`containsAll(c)`	Subset check

⚠ COMMON MISTAKES

- Mutable objects as set elements whose hash changes after insertion
- Expecting `hashSetOf` to keep order
- Using a List for frequent membership tests

✓ TIPS & BEST PRACTICES

- Use `toSet()` or `distinct()` for deduplication
- Use sets for "seen" tracking in loops
- Use `sortedSetOf` or `toSortedSet()` when order matters

QUICK SUMMARY

- `setOf` — unique, ordered
- `union` / `intersect` / `subtract`
- `toSet()` — dedupe
- `sortedSetOf` — sorted

Maps — Basics

KEY-VALUE PAIRS

Maps store values by key. Build them with `mapOf(key to value)` (read-only) or `mutableMapOf()`. `map[key]` returns the value or **null**, so lookups are null-safe by type. `getValue(key)` throws if the key is missing, and `getOrDefault` supplies a fallback.

KEY POINTS

- `mapOf` / `mutableMapOf` keep insertion order (LinkedHashMap)
- `map[key] = value` adds or replaces in a MutableMap
- Iterate with destructuring: `for ((k, v) in map)`
- Keys rely on `equals` / `hashCode`

CREATING AND READING MAPS

```
val prices = mapOf("tea" to 3.5, "cake" to 4.0, "pie" to 5.25)

println(prices["tea"])                    // 3.5
println(prices["coffee"])                 // null
println(prices["coffee"] ?: 0.0)          // 0.0
println(prices.getOrDefault("coffee", 1.0))   // 1.0
println(prices.getValue("cake"))          // 4.0
// prices.getValue("coffee")              // NoSuchElementException
println("pie" in prices)                  // true (containsKey)
println(prices.containsValue(4.0))        // true
println(prices.keys)                      // [tea, cake, pie]
println(prices.values)                    // [3.5, 4.0, 5.25]
println(prices.size)                      // 3

for ((item, price) in prices) println("$item: $price")

val stock = mutableMapOf<String, Int>()
stock["apple"] = 10                       // add
stock["apple"] = 12                       // replace
stock["pear"] = 4
stock.remove("pear")
println(stock)                            // {apple=12}

val byLength = listOf("a", "bb", "cc").associateBy { it.length }  // {1=a, 2=cc}
```

CODE EXPLAINED

- → `"tea" to 3.5` builds a `Pair` used as an entry
- → `map[key]` returns a nullable type, forcing a decision on missing keys
- → `in` checks keys, not values
- → `associateBy` builds a map from a list (later duplicates win)

JAVA EQUIVALENT

- `Map.of("tea", 3.5, "cake", 4.0)` (immutable)
- `map.get(key)` returns null silently; Kotlin's type shows it
- `for (var e : map.entrySet())` with `getKey()` / `getValue()`

IDIOMATIC KOTLIN

- ⇄ `if (map.containsKey(k)) map[k]!!` → `map[k] ?: default`
- ⇄ `map.put(k, v)` → `map[k] = v`

READING

CALL	MISSING KEY
`map[k]`	null
`map[k] ?: d`	d
`getOrDefault(k, d)`	d
`getValue(k)`	NoSuchElementException
`getOrElse(k) { }`	Computed

CREATING

FUNCTION	TYPE
`mapOf(k to v)`	Read-only (ordered)
`mutableMapOf()`	LinkedHashMap
`hashMapOf()`	HashMap
`sortedMapOf()`	TreeMap
`emptyMap()`	Empty

⚠ COMMON MISTAKES

- Using `!!` on `map[key]` instead of handling missing keys
- Expecting `in` to search values
- Duplicate keys in `mapOf` (the last one wins silently)

✓ TIPS & BEST PRACTICES

- Use `getValue` when a missing key is a bug
- Use `associate` / `associateBy` / `groupBy` to build maps from lists
- Expose `Map`, not `MutableMap`, from classes

QUICK SUMMARY

- `mapOf(k to v)` — build
- `map[k]` — nullable lookup
- `getValue` — must exist
- `for ((k, v) in map)` — iterate

Map Operations

TRANSFORMING AND UPDATING MAPS

Maps have functional operations that return new maps: `mapValues`, `mapKeys`, `filterKeys`, `filterValues`, and +/− for adding or removing entries. On mutable maps, `getOrPut` returns the existing value or inserts a computed one, which makes caches and groupings short.

KEY POINTS

- `mapValues { (k, v) -> }` gets each entry
- `getOrPut` computes the value only when missing
- `+ (k to v)` returns a new map with the entry
- `withDefault` changes only `getValue`, not `[]`

MAP TRANSFORMATIONS

```
val prices = mapOf("tea" to 3.5, "cake" to 4.0, "pie" to 5.25)

println(prices.mapValues { (_, p) -> p * 1.06 })   // taxed prices
println(prices.mapKeys { it.key.uppercase() })     // {TEA=…, CAKE=…, PIE=…}
println(prices.filterValues { it > 4.0 })          // {pie=5.25}
println(prices.filterKeys { it.startsWith("c") })  // {cake=4.0}
println(prices + ("coffee" to 2.0))                // new map with coffee
println(prices - "tea")                            // new map without tea
println(prices.maxBy { it.value }.key)             // pie
println(prices.entries.sortedBy { it.value }.map { it.key })
// [tea, cake, pie]

val cache = mutableMapOf<Int, String>()
fun lookup(id: Int) = cache.getOrPut(id) { loadName(id) }   // load once

val wordsByLetter = mutableMapOf<Char, MutableList<String>>()
for (w in listOf("apple", "avocado", "banana")) {
    wordsByLetter.getOrPut(w.first()) { mutableListOf() }.add(w)
}
println(wordsByLetter)              // {a=[apple, avocado], b=[banana]}

val config = buildMap {
    put("env", "prod")
    if (debug) put("log", "verbose")
}

val counts = mutableMapOf<String, Int>()
counts.merge("x", 1, Int::plus); counts.merge("x", 1, Int::plus)
println(counts)                     // {x=2}
```

CODE EXPLAINED

→ `mapValues` keeps keys and transforms values

→ `getOrPut` avoids "check, then put" boilerplate

→ `buildMap` creates a read-only map with conditional entries

→ `merge` is Java's map API, usable directly

JAVA EQUIVALENT

☕ `map.computeIfAbsent(k, key -> new ArrayList<>()).add(w)`

☕ `map.entrySet().stream().collect(toMap(…))` for transforms

☕ `map.merge("x", 1, Integer::sum)`

IDIOMATIC KOTLIN

⇄ `if (!m.containsKey(k)) m[k] = …; m[k]!!` → `m.getOrPut(k) { … }`

⇄ stream collectors on maps → `mapValues` / `filterValues`

OPERATIONS

CALL	RETURNS
`mapValues { }`	New values
`mapKeys { }`	New keys
`filterKeys / filterValues`	Subset
`filter { (k, v) -> }`	Subset
`map + (k to v)`	Added
`map - k`	Removed
`toMutableMap()`	Mutable copy
`toList()`	List<Pair>

MUTABLEMAP HELPERS

CALL	DOES
`getOrPut(k) { }`	Get or insert
`putAll(m)`	Merge in
`remove(k)`	Remove
`merge(k, v, f)`	Combine values
`compute(k) { }`	Recompute

⚠ COMMON MISTAKES

- Using `mapKeys` where keys collide (later ones overwrite)
- Expecting `withDefault` to affect `map[key]`
- Mutating a map while iterating its entries

✓ TIPS & BEST PRACTICES

- Use `getOrPut` for memoization and grouping
- Use `groupBy` (sheet 53) when building lists per key
- Convert with `toSortedMap()` for ordered output

QUICK SUMMARY

- `mapValues` / `mapKeys`
- `filterKeys` / `filterValues`
- `getOrPut` — get or insert
- `+` / `-` — new maps

Arrays

FIXED-SIZE ARRAYS

Kotlin arrays have a fixed size. `Array<T>` holds objects, while `IntArray`, `DoubleArray` and friends hold **primitives** with no boxing. In everyday Kotlin, Lists are preferred; arrays matter for performance, varargs and Java APIs that expect them.

KEY POINTS

- `arr[i]` reads and writes
- `==` on arrays compares references; use `contentEquals`
- `println(arr)` prints an address; use `contentToString()`
- Arrays are invariant: `Array<Int>` is not `Array<Number>`

UNDER THE JVM

- `IntArray` compiles to Java `int[]`; `Array<Int>` to `Integer[]`
- `Array<String>` is `String[]`
- Indexing compiles to plain array load/store instructions

ARRAYS IN PRACTICE

```
val names = arrayOf("Ada", "Bob", "Cy")        // Array<String>
val scores = intArrayOf(90, 85, 77)            // IntArray (int[])
val zeros = DoubleArray(5)                     // [0.0, 0.0, 0.0, 0.0, 0.0]
val squares = IntArray(5) { it * it }          // [0, 1, 4, 9, 16]
val grid = Array(3) { IntArray(3) }            // 3x3 of zeros

names[1] = "Bea"
scores[0] += 5
grid[1][2] = 7

println(names.size)                            // 3
println(names.contentToString())               // [Ada, Bea, Cy]
println(scores.sum())                          // 257
println(scores.average())                      // 85.66666666666667
println(scores.maxOrNull())                    // 95
println(intArrayOf(1, 2) contentEquals intArrayOf(1, 2))   // true
println(intArrayOf(1, 2) == intArrayOf(1, 2)) // false (references)

val list = scores.toList()                     // List<Int>
val back = list.toIntArray()                   // IntArray
val boxed: Array<Int> = scores.toTypedArray() // Integer[]
scores.sort()                                  // in place
for ((i, s) in scores.withIndex()) println("$i: $s")
```

CODE EXPLAINED

- → `IntArray` avoids boxing every number
- → `Array(3) { IntArray(3) }` builds a 2-D grid
- → `contentEquals` compares elements
- → Most collection functions also work on arrays

JAVA EQUIVALENT

- `int[] scores = {90, 85, 77};`
- `Arrays.equals(a, b)` and `Arrays.toString(a)`
- `new int[3][3]` for a grid

IDIOMATIC KOTLIN

- ⇄ `Array<Int>` for numbers → `IntArray`
- ⇄ arrays in APIs → `List` unless performance or Java interop needs arrays

ARRAY TYPES

KOTLIN	JAVA
`IntArray`	int[]
`LongArray`	long[]
`DoubleArray`	double[]
`BooleanArray`	boolean[]
`CharArray`	char[]
`Array<String>`	String[]
`Array<Int>`	Integer[]

HELPERS

CALL	DOES
`contentToString()`	Readable text
`contentEquals(b)`	Element equality
`toList()` / `toIntArray()`	Convert
`copyOf(n)`	Copy / resize
`fill(x)`	Set all
`sort()`	In place

⚠ COMMON MISTAKES

- Comparing arrays with `==`
- Printing an array directly (shows `[I@1b6d3586`)
- Using `Array<Int>` and paying boxing costs

✓ TIPS & BEST PRACTICES

- Use `IntArray` etc. for numeric performance
- Convert to List at API boundaries
- Use `copyOf` instead of manual copying loops

QUICK SUMMARY

- `Array<T>` — objects
- `IntArray` — primitives
- `contentEquals` — compare
- `toList()` — convert

Transformations

RESHAPING COLLECTIONS

Collection functions return **new** collections and never change the source. `map` transforms each element, `filter` keeps matches, `flatMap` maps and flattens, `zip` pairs two lists, `associate` builds maps, and `chunked`/`windowed` split into groups. They chain naturally.

KEY POINTS

- All of these are eager on collections (see Sequences, sheet 55)
- Each step creates an intermediate list
- `mapNotNull` maps and drops nulls in one pass
- `partition` splits into two lists by a predicate

CHAINING TRANSFORMATIONS

```
data class Order(
    val id: Int, val customer: String,
    val items: List<String>, val total: Double
)
val orders = listOf(
    Order(1, "Ada", listOf("tea", "cake"), 12.5),
    Order(2, "Bob", listOf("pie"), 5.25),
    Order(3, "Ada", listOf("tea"), 3.5)
)

println(orders.map { it.total })                          // [12.5, 5.25, 3.5]
println(orders.filter { it.total > 5 }.map { it.id })     // [1, 2]
println(orders.flatMap { it.items })  // [tea, cake, pie, tea]
println(orders.flatMap { it.items }.distinct())           // [tea, cake, pie]
println(listOf(listOf(1, 2), listOf(3)).flatten())        // [1, 2, 3]
println(listOf("1", "x", "3").mapNotNull { it.toIntOrNull() })   // [1, 3]
println(orders.mapIndexed { i, o -> "${i + 1}. ${o.customer}" })

val names = listOf("Ada", "Bob")
val ages = listOf(36, 41)
println(names zip ages)  // [(Ada, 36), (Bob, 41)]
println(names.zip(ages) { n, a -> "$n is $a" })  // [Ada is 36, Bob is 41]

println(orders.associateBy { it.id })  // {1=Order…, 2=…, 3=…}
println(orders.associate { it.id to it.total })  // {1=12.5, 2=5.25, 3=3.5}
println((1..10).chunked(4))          // [[1, 2, 3, 4], [5, 6, 7, 8], [9, 10]]
println((1..5).windowed(3))  // [[1, 2, 3], [2, 3, 4], [3, 4, 5]]
val (big, small) = orders.partition { it.total > 5 }
```

CODE EXPLAINED

- → `flatMap` turns each order into its items, then flattens
- → `zip` with a lambda combines pairs directly
- → `associate` builds a map from pairs
- → `partition` destructures into two lists

JAVA EQUIVALENT

- `orders.stream().map(Order::total).toList()`
- `.flatMap(o -> o.items().stream())`
- `Collectors.toMap(Order::id, o -> o)` for associate

IDIOMATIC KOTLIN

- ⇄ `.stream().map(…).collect(toList())` → `.map { … }`
- ⇄ `map { }.filterNotNull()` → `mapNotNull { }`

TRANSFORMATIONS

FUNCTION	RESULT
`map { }`	Same size, transformed
`mapNotNull { }`	Transformed, nulls dropped
`filter { } / filterNot { }`	Subset
`filterIsInstance<T>()`	By type
`flatMap { } / flatten()`	Flattened
`zip(other)`	Pairs
`associate / associateBy / associateWith`	Maps
`chunked(n) / windowed(n)`	Groups
`partition { }`	Pair of lists

⚠ COMMON MISTAKES

- Long chains on huge lists creating many intermediate lists (use sequences)
- Using `associateBy` with duplicate keys and losing data (use `groupBy`)
- Expecting `map` to modify the original list

✓ TIPS & BEST PRACTICES

- Read chains top to bottom: one operation per line
- Use `asSequence()` for long chains on big data
- Use `associateWith` to map keys to computed values

QUICK SUMMARY

- `map` / `filter` — core
- `flatMap` — flatten
- `zip` / `associate` — combine
- `chunked` / `windowed` — group

Aggregation & Grouping

SUMMARIZING COLLECTIONS

Aggregations reduce a collection to one value: `sumOf`, `count`, `average`, `maxOf`, or a custom `fold`. `groupBy` turns a list into a map of lists by key, and `groupingBy` with `eachCount()` or `fold` aggregates each group without building the lists.

KEY POINTS

- `reduce` throws on empty collections; `fold` takes a start value
- `maxByOrNull` returns the element; `maxOf` returns the selector value
- `sumOf` works with Int, Long, Double, BigDecimal
- `groupBy` keeps insertion order of keys

TOTALS AND GROUPS

```
data class Sale(val region: String, val rep: String, val amount: Double)
val sales = listOf(
    Sale("East", "Ada", 120.0), Sale("West", "Bob", 80.0),
    Sale("East", "Cy", 200.0), Sale("West", "Bob", 50.0)
)

println(sales.sumOf { it.amount })                    // 450.0
println(sales.count { it.amount > 100 })              // 2
println(sales.map { it.amount }.average())            // 112.5
println(sales.maxByOrNull { it.amount }?.rep)         // Cy
println(sales.minOf { it.amount })                    // 50.0
println(sales.fold(0.0) { acc, s -> acc + s.amount }) // 450.0
println(listOf(1, 2, 3, 4).reduce { a, b -> a * b })  // 24

val byRegion = sales.groupBy { it.region }
println(byRegion.keys)                                // [East, West]
println(byRegion.mapValues { (_, list) -> list.sumOf { it.amount } })
// {East=320.0, West=130.0}
println(sales.groupBy({ it.rep }, { it.amount }))
// {Ada=[120.0], Bob=[80.0, 50.0], Cy=[200.0]}
println(sales.groupingBy { it.rep }.eachCount())      // {Ada=1, Bob=2, Cy=1}

val (big, small) = sales.partition { it.amount >= 100 }
println(big.size to small.size)                       // (2, 2)
println(listOf("a", "b", "a").groupingBy { it }.eachCount())   // {a=2, b=1}
```

CODE EXPLAINED

- → `sumOf { }` sums a selected property
- → `maxByOrNull` returns the whole Sale; `?.rep` reads its rep
- → `groupBy` with two lambdas picks keys and values
- → `groupingBy { }.eachCount()` counts per key without lists

JAVA EQUIVALENT

- `sales.stream().mapToDouble(Sale::amount).sum()`
- `Collectors.groupingBy(Sale::region, summingDouble(Sale::amount))`
- `Collectors.partitioningBy(s -> s.amount() >= 100)`

IDIOMATIC KOTLIN

- ⇄ `map { }.sum()` → `sumOf { }`
- ⇄ counting with a mutable map loop → `groupingBy { }.eachCount()`

AGGREGATES

FUNCTION	RETURNS
`sumOf { }`	Total
`count { }`	Matches
`average()`	Double mean
`maxOf / minOf { }`	Max/min value
`maxByOrNull { }`	Element with max
`fold(init) { }`	Custom
`reduce { }`	Custom (no init)
`any / all / none { }`	Boolean

GROUPING

FUNCTION	RESULT
`groupBy { }`	Map<K, List<T>>
`groupBy({ k }, { v })`	Map<K, List<V>>
`groupingBy { }.eachCount()`	Map<K, Int>
`groupingBy { }.fold(…)`	Map<K, R>
`partition { }`	Pair<List, List>

⚠ COMMON MISTAKES

- `reduce` on an empty list (exception)
- `maxBy` (throws on empty) instead of `maxByOrNull`
- Floating-point sums for money (use BigDecimal `sumOf`)

✓ TIPS & BEST PRACTICES

- Use `groupingBy` for large data to avoid building lists
- Use `sumOf { it.price }` with BigDecimal for currency
- Use `partition` when you need both sides of a filter

QUICK SUMMARY

- `sumOf` / `count` / `average`
- `fold` / `reduce` — custom
- `groupBy` — map of lists
- `eachCount()` — counts per key

Sorting & Searching

ORDERING AND FINDING

`sorted()` returns a new sorted list; `sort()` sorts a MutableList in place. `sortedBy { }` sorts by a property, and `compareBy` with `thenBy` builds multi-key comparators. For searching, `find`/`firstOrNull` return the first match or null, `any`/`all`/`none` test conditions, and `binarySearch` works on sorted lists.

KEY POINTS

- Sorting is stable: equal elements keep their order
- `sortedByDescending` reverses one key
- `first { }` throws if nothing matches; `firstOrNull` does not
- `binarySearch` returns a negative insertion point when missing

SORT AND SEARCH

```
data class Emp(val name: String, val dept: String, val salary: Int)
val emps = listOf(
    Emp("Cy", "Eng", 95), Emp("Ada", "Eng", 120),
    Emp("Bob", "Ops", 80), Emp("Dee", "Ops", 80)
)

println(listOf(3, 1, 2).sorted())                        // [1, 2, 3]
println(listOf(3, 1, 2).sortedDescending())              // [3, 2, 1]
println(emps.sortedBy { it.name }.map { it.name })       // [Ada, Bob, Cy, Dee]
println(emps.sortedByDescending { it.salary }.first().name)   // Ada

val byDeptThenSalary = emps.sortedWith(
    compareBy<Emp> { it.dept }.thenByDescending { it.salary }.thenBy { it.name }
)
println(byDeptThenSalary.map { it.name })                // [Ada, Cy, Bob, Dee]

println(emps.find { it.dept == "Ops" }?.name)            // Bob
println(emps.firstOrNull { it.salary > 200 })            // null
println(emps.any { it.salary > 100 })                    // true
println(emps.all { it.salary >= 80 })                    // true
println(emps.none { it.dept == "HR" })                   // true
println(emps.indexOfFirst { it.name == "Bob" })          // 2

val ids = listOf(10, 20, 30, 40)
println(ids.binarySearch(30))                            // 2
println(ids.binarySearch(25))                            // -3 (insert at 2)

val mutable = emps.toMutableList()
mutable.sortBy { it.salary }                             // in place
```

CODE EXPLAINED

- → `compareBy … thenByDescending … thenBy` sorts by three keys
- → Bob and Dee tie on salary, so `thenBy { it.name }` orders them
- → `find` is an alias for `firstOrNull`
- → A missing binary search key returns `-(insertionPoint) - 1`

JAVA EQUIVALENT

- `list.sort(Comparator.comparing(Emp::dept).thenComparing(Emp::salary, reverseOrder()))`
- `stream().filter(…).findFirst()` returns Optional
- `Collections.binarySearch(list, 30)`

IDIOMATIC KOTLIN

- ⇄ `Comparator.comparing(...)` chains → `compareBy { }.thenBy { }`
- ⇄ `filter { }.first()` → `first { }` (or `firstOrNull`)

SORTING

CALL	RETURNS
`sorted() / sortedDescending()`	New list
`sortedBy { } / sortedByDescending { }`	New list by key
`sortedWith(cmp)`	New list, comparator
`sort() / sortBy { }`	In place (Mutable)
`reversed()`	Reversed copy
`compareBy { }.thenBy { }`	Comparator

SEARCHING

CALL	RETURNS
`find / firstOrNull { }`	Element or null
`first { }`	Element or exception
`lastOrNull { }`	Last match
`indexOfFirst { }`	Index or -1
`any / all / none { }`	Boolean
`binarySearch(x)`	Index or negative

⚠ COMMON MISTAKES

- Calling `sorted()` and expecting the original to change
- `first { }` on data that may not match (NoSuchElementException)
- `binarySearch` on an unsorted list

✓ TIPS & BEST PRACTICES

- Implement `Comparable` for a natural order, then use `sorted()`
- Use `compareBy` with several selectors: `compareBy({ it.dept }, { it.name })`
- Use `maxByOrNull` / `minByOrNull` instead of sorting to find extremes

QUICK SUMMARY

- `sorted` vs `sort`
- `sortedBy` / `compareBy` / `thenBy`
- `find` / `firstOrNull`
- `any` / `all` / `none`

Sequences

LAZY COLLECTION PROCESSING

A Sequence processes elements **lazily**, one at a time through the whole chain, instead of building a full list at each step. Nothing runs until a **terminal** operation like `toList()`, `first()` or `sum()`. Sequences help with large data, long chains, early exits and infinite streams.

KEY POINTS

- Intermediate ops (`map` , `filter`) are lazy
- Terminal ops (`toList` , `count` , `first`) trigger work
- Sequences can be infinite: always limit with `take`
- Small collections with short chains are often faster as lists

UNDER THE JVM

- Each sequence step wraps the previous one in an iterator object
- No intermediate `ArrayList` s are allocated
- Similar to Java Streams, but single-threaded and reusable if the source is

EAGER VS LAZY

```
val words = listOf("apple", "banana", "cherry", "date")

// Eager: map runs on all 4, then filter, then first
words.map { println("map $it"); it.uppercase() }
     .filter { it.startsWith("B") }
     .first()

// Lazy: stops as soon as one element passes
words.asSequence()
     .map { println("map $it"); it.uppercase() }    // prints apple, banana only
     .filter { it.startsWith("B") }
     .first()                                       // BANANA

val evens = generateSequence(0) { it + 2 }          // infinite
println(evens.take(5).toList())                     // [0, 2, 4, 6, 8]

val fib = sequence {
    var a = 0; var b = 1
    while (true) { yield(a); val t = a + b; a = b; b = t }
}
println(fib.take(10).toList())  // [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

val lines = File("big.log").useLines { seq ->       // lazy file lines
    seq.filter { "ERROR" in it }.take(20).toList()
}
```

CODE EXPLAINED

- → The sequence version maps only until it finds a match
- → `generateSequence` creates values on demand from a seed
- → `sequence { yield() }` builds a generator with a coroutine
- → `useLines` streams a file without loading it all

JAVA EQUIVALENT

- Java Streams: `words.stream().map(…).filter(…).findFirst()`
- `Stream.iterate(0, n -> n + 2).limit(5)`
- Streams are single-use; Kotlin sequences can be re-iterated if the source allows

IDIOMATIC KOTLIN

- ⇄ long chains over large lists → `asSequence()`
- ⇄ sequences for tiny lists → plain list operations

LIST VS SEQUENCE

	LIST OPS	SEQUENCE OPS
Evaluation	Eager, step by step	Lazy, element by element
Intermediate lists	Yes	No
Early exit	After full steps	Immediately
Infinite data	No	Yes (with take)
Best for	Small data	Large data, long chains

BUILDERS

FUNCTION	CREATES
asSequence()	From a collection
sequenceOf(a, b)	From values
generateSequence(seed) { }	From a rule
sequence { yield(x) }	Generator
File.useLines { }	File lines

⚠ COMMON MISTAKES

- Forgetting a terminal operation (nothing happens)
- Infinite sequence with no `take` (runs forever)
- Using sequences on tiny lists and making code slower

✓ TIPS & BEST PRACTICES

- Add `asSequence()` when a chain has 3+ steps over thousands of items
- End with `toList()` when you need a collection again
- Use `useLines` for large files

QUICK SUMMARY

- `asSequence()` — go lazy
- terminal op — runs it
- `generateSequence` / `sequence { }`
- `take` — limit infinite

Sheet 56

Classes & Constructors

`class Person(val name: String)` · primary constructor · `init` blocks · secondary `constructor` · default params instead of overloads · no new

DEFINING CLASSES

A Kotlin class declares its **primary constructor** in the header, and `val`/`var` parameters become properties automatically. `init` blocks run initialization logic, secondary constructors handle rare extra cases, and objects are created without new. Default parameter values replace most constructor overloads.

KEY POINTS

- Classes are `final` by default (sheet 59)
- `init` blocks run in order with property initializers
- Secondary constructors must delegate to the primary with `this(…)`
- A class with no body needs no braces

CONSTRUCTORS AND INIT

```
class Person(val name: String, var age: Int = 0)   // properties from params

val p = Person("Ada", 36)                          // no `new`
println(p.name)                                    // Ada
p.age += 1

class Account(val id: String, initialBalance: Double) {
    var balance = initialBalance                   // plain param used here
        private set

    init {
        require(initialBalance >= 0) { "negative balance" }
        println("Opened $id")
    }

    constructor(id: String) : this(id, 0.0)        // secondary constructor

    fun deposit(amount: Double) { balance += amount }
}

val a = Account("A-1", 50.0)                       // Opened A-1
val b = Account("A-2")                             // uses secondary
a.deposit(25.0)
println(a.balance)                                 // 75.0

class Empty                                        // no body needed
class Point private constructor(val x: Int, val y: Int) {
    companion object { fun origin() = Point(0, 0) }   // factory
}
```

CODE EXPLAINED

- → `val name` in the header declares and initializes a property
- → `initialBalance` without val/var is only a constructor parameter
- → `init` validates inputs when the object is created
- → A private constructor plus a companion factory controls creation

JAVA EQUIVALENT

- Fields + constructor + getters, or a `record Person(String name, int age)`
- `new Person("Ada", 36)`
- Several overloaded constructors for optional values

IDIOMATIC KOTLIN

- ⇄ fields assigned in the constructor body → `val` properties in the header
- ⇄ multiple constructor overloads → default parameter values

CONSTRUCTOR PARTS

PART	EXAMPLE
`Primary`	class A(val x: Int)
`Property param`	val / var in header
`Plain param`	used in init or initializers
`init block`	init { require(…) }
`Secondary`	constructor(…) : this(…)
`Private`	class A private constructor(…)

INIT ORDER

STEP	RUNS
1	Primary constructor args
2	Property initializers + init blocks (top to bottom)
3	Secondary constructor body
Superclass	Before all of the above

⚠ COMMON MISTAKES

- Calling open functions from `init` (subclass state not ready yet)
- Writing `new Person()`
- Adding secondary constructors where defaults would do

✓ TIPS & BEST PRACTICES

- Put validation in `init` with `require`
- Use private constructors + companion factories for controlled creation
- Use data classes (sheet 61) for value holders

QUICK SUMMARY

- header = primary constructor
- `val` / `var` params → properties
- `init` — setup logic
- defaults replace overloads

Properties

`val` / `var` properties · custom `get()` / `set()` · `field` backing field · `private set` · backing properties `_items` · computed properties

FIELDS WITH BUILT-IN ACCESSORS

Kotlin classes have **properties**, not public fields. Each property has a getter (and a setter for `var`) generated for you. You can customize them with `get()` and `set(value)`, use `field` to reach the backing storage, restrict the setter with `private set`, or compute a value on every read.

KEY POINTS

- Callers use `obj.prop`, never `getProp()`
- Computed properties have no backing field
- `field` is only valid inside accessors
- Backing property pattern: private `_items`, public read-only `items`

CUSTOM ACCESSORS

```
class Rectangle(val width: Double, val height: Double) {
    val area: Double                        // computed on each read
        get() = width * height
    val isSquare get() = width == height
}

class User(name: String) {
    var name: String = name
        set(value) {
            require(value.isNotBlank())
            field = value.trim()            // field = backing storage
        }

    var loginCount = 0
        private set                         // public read, private write

    fun login() { loginCount++ }
}

class Cart {
    private val _items = mutableListOf<String>()
    val items: List<String> get() = _items // read-only view

    fun add(item: String) { _items += item }
}

val r = Rectangle(3.0, 4.0)
println(r.area)                             // 12.0
val u = User("  Ada ")
u.name = " Lovelace "
println(u.name)                             // Lovelace
// u.loginCount = 5                         // error: private setter
```

CODE EXPLAINED

- → `area` has a getter but no stored value
- → The custom setter validates and trims before storing in `field`
- → `private set` blocks outside writes
- → The backing property hides the mutable list behind a `List`

JAVA EQUIVALENT

- `private double width; public double getWidth()`
- `public void setName(String n) { … this.name = n.trim(); }`
- `Collections.unmodifiableList(items)` for read-only views

IDIOMATIC KOTLIN

- ⇄ `getX()` / `setX()` methods → properties with accessors
- ⇄ public `MutableList` fields → private `_items` + public `List`

PROPERTY FORMS

FORM	MEANING
`val x = 1`	Read-only, stored
`var x = 1`	Read-write, stored
`val x get() = …`	Computed
`var x … set(v) { field = v }`	Custom setter
`var x … private set`	Restricted setter
`lateinit var x: T`	Set later
`val x by lazy { }`	Lazy (sheet 15)

JAVA VIEW

KOTLIN	JAVA SEES
`val name`	getName()
`var age`	getAge() / setAge()
`val isActive`	isActive()
`@JvmField val x`	Public field

⚠ COMMON MISTAKES

- Writing `name = value` inside its own setter (infinite recursion; use `field`)
- Expensive work in a getter that looks like a cheap field read
- Exposing mutable collections directly

✓ TIPS & BEST PRACTICES

- Use computed properties for derived values
- Use `private set` for state changed only by methods
- Keep getters cheap and side-effect free

QUICK SUMMARY

- properties — not fields
- `get()` / `set()` — custom
- `field` — backing storage
- `private set` — read-only outside

Visibility Modifiers

WHO CAN SEE WHAT

Kotlin has four visibility modifiers. **public** is the default. **private** means visible inside the class, or inside the file for top-level declarations. **protected** adds subclasses. **internal** means visible anywhere in the same **module** (a Gradle source set or Maven project), which Java has no direct equivalent for.

KEY POINTS

- No package-private visibility in Kotlin
- Top-level `private` = file-private
- `internal` is ideal for library code not meant for users
- Overriding members keep their visibility unless widened

VISIBILITY IN PRACTICE

```
// File: PaymentService.kt  (module :payments)
package com.example.payments

private const val MAX_RETRIES = 3          // visible in this file only

internal class Gateway {                   // visible in module :payments
    fun charge(amount: Long) = true
}

class PaymentService internal constructor( // public class, internal constructor
    private val gateway: Gateway
) {
    fun pay(amount: Long): Boolean {       // public (default)
        repeat(MAX_RETRIES) { if (tryCharge(amount)) return true }
        return false
    }

    private fun tryCharge(amount: Long) = gateway.charge(amount)

    protected open fun audit() {}          // warning: class is final
}

open class Base {
    protected val secret = 42
    internal val moduleOnly = "m"
}
class Derived : Base() {
    fun reveal() = secret                  // OK: subclass
}
// Base().secret                           // error: protected
```

CODE EXPLAINED

- → `MAX_RETRIES` cannot be seen from other files
- → `Gateway` is usable in the payments module but hidden from apps that use it
- → The internal constructor lets only the module create PaymentService
- → `protected` members are visible in subclasses only

JAVA EQUIVALENT

- `public`, `private`, `protected`, plus package-private (no modifier)
- Java `protected` also allows access from the same package
- Module-level hiding needs JPMS `module-info.java` exports

IDIOMATIC KOTLIN

- ⇄ everything public by default → `internal` / `private` where possible
- ⇄ package-private Java habits → file-private or `internal`

MODIFIERS

MODIFIER	CLASS MEMBER	TOP-LEVEL
`public`	Everywhere	Everywhere
`internal`	Same module	Same module
`protected`	Class + subclasses	Not allowed
`private`	Inside the class	Inside the file

KOTLIN VS JAVA

KOTLIN	JAVA
`public (default)`	package-private (default)
`internal`	public (name-mangled)
`protected`	protected (no package access)
`private top-level`	private static in FileKt

⚠ COMMON MISTAKES

- Assuming package-private exists
- Using `public` on everything in a library API
- Expecting `protected` to allow same-package access like Java

✓ TIPS & BEST PRACTICES

- Default to `private`, then widen as needed
- Use `internal` for library implementation classes
- Turn on explicit API mode (`kotlin { explicitApi() }`) in libraries

QUICK SUMMARY

- `public` — default
- `private` — class or file
- `protected` — subclasses
- `internal` — same module

Inheritance

open · final by default · `override` · `super` · calling the superclass constructor · `final override`

EXTENDING CLASSES

Kotlin classes and members are **final by default**: mark a class `open` to allow subclasses, and mark members `open` to allow overriding. Subclasses call the superclass constructor in their header, overriding members must say `override`, and `super` calls the parent version.

KEY POINTS

- Syntax: `class Dog : Animal("dog")`
- `override` members are open unless marked `final override`
- Override `val` with `var`, not the reverse
- Abstract classes and interfaces are open automatically

OPEN, OVERRIDE, SUPER

```
open class Animal(val name: String) {
    open val sound = "..."
    open fun speak() = "$name says $sound"
    fun describe() = "Animal: $name"            // final: cannot override
}

class Dog(name: String) : Animal(name) {        // call super constructor
    override val sound = "Woof"
    override fun speak() = super.speak() + "!"  // reuse parent version
}

open class Cat(name: String) : Animal(name) {
    final override fun speak() = "$name purrs"  // subclasses cannot change it
}

val pets: List<Animal> = listOf(Dog("Rex"), Cat("Tom"))
pets.forEach { println(it.speak()) }
// Rex says Woof!
// Tom purrs

open class View(val id: Int) {
    constructor(id: Int, tag: String) : this(id)
}
class Button : View {
    constructor(id: Int) : super(id)            // no primary constructor
}
```

CODE EXPLAINED

- → `open` on Animal allows `Dog : Animal(name)`
- → `override val sound` replaces the property
- → `super.speak()` calls the parent function
- → `final override` stops further overriding

JAVA EQUIVALENT

- Java classes are open by default; `final` opts out
- `@Override` is optional in Java, `override` is required in Kotlin
- `super(name)` in the constructor body

IDIOMATIC KOTLIN

- ⇄ deep inheritance trees → interfaces + composition / delegation (sheet 67)
- ⇄ making everything `open` → open only what is designed for extension

KEYWORDS

KEYWORD	MEANING
`open class`	Can be subclassed
`open fun / val`	Can be overridden
`override`	Replaces parent member
`final override`	Stop further overrides
`super.f()`	Call parent version
`abstract`	Must be implemented

⚠ COMMON MISTAKES

- Forgetting `open` and getting "this type is final"
- Calling open members from a constructor
- Framework issues with final classes (Spring/JPA need the all-open plugin, sheet 93)

✓ TIPS & BEST PRACTICES

- Prefer composition over inheritance
- Use the `kotlin-spring` / `kotlin-jpa` plugins for frameworks that proxy classes
- Use sealed classes for closed hierarchies (sheet 63)

QUICK SUMMARY

- final by default
- `open` — allow extension
- `override` — required
- `super` — parent call

Sheet 60

Abstract Classes & Interfaces

`abstract class` · `interface` with default methods · properties in interfaces · multiple interfaces · `super<A>.foo()`

CONTRACTS AND PARTIAL IMPLEMENTATIONS

An **abstract class** can hold state and constructors and leaves some members `abstract`. An **interface** declares functions and properties, can provide default implementations, but holds no state. A class can extend one class and implement many interfaces.

KEY POINTS

- Interface properties are abstract or have getters (no backing field)
- Default methods need no keyword in interfaces
- When two interfaces share a default, override and pick with `super<A>`
- Use interfaces for capabilities, abstract classes for shared state

ABSTRACT CLASS + INTERFACES

```
interface Drawable {
    val color: String                       // abstract property
    fun draw(): String                      // abstract function
    fun describe() = "A $color shape"       // default implementation
}

interface Clickable {
    fun click() = "clicked"
    fun describe() = "Something clickable"
}

abstract class Shape(val name: String) {    // has state + constructor
    abstract fun area(): Double
    fun summary() = "$name with area ${"%.1f".format(area())}"
}

class Circle(val r: Double) : Shape("Circle"), Drawable, Clickable {
    override val color = "red"
    override fun area() = Math.PI * r * r
    override fun draw() = "( )"
    override fun describe() =
        super<Drawable>.describe() + " / " + super<Clickable>.describe()
}

val c = Circle(2.0)
println(c.summary())        // Circle with area 12.6
println(c.describe())       // A red shape / Something clickable
val d: Drawable = c
println(d.draw())           // ( )
```

CODE EXPLAINED

→ Circle extends one abstract class and two interfaces

→ Both interfaces define `describe`, so Circle must override it

→ `super<Drawable>` picks one parent's default

→ The abstract class supplies `name` and `summary()`

JAVA EQUIVALENT

- `abstract class Shape`, `interface Drawable` with `default` methods
- `Drawable.super.describe()` to call a specific default
- Java interfaces can hold constants but not instance state

IDIOMATIC KOTLIN

⇄ abstract classes used only as contracts → interfaces

⇄ interfaces with many defaults acting like base classes → composition

COMPARE

FEATURE	ABSTRACT CLASS	INTERFACE
State (fields)	Yes	No
Constructor	Yes	No
Default methods	Yes	Yes
Multiple inheritance	No (one class)	Yes (many)
Use for	Shared base	Capabilities

⚠ COMMON MISTAKES

- Trying to store state in an interface property (no backing field)
- Not overriding a conflicting default (compile error)
- Using abstract classes where a sealed interface fits better

✓ TIPS & BEST PRACTICES

- Name interfaces by capability: `Drawable`, `Closeable`
- Keep interfaces small and focused
- Use `fun interface` for single-method contracts (sheet 45)

QUICK SUMMARY

- `abstract class` — state + abstract
- `interface` — contract + defaults
- many interfaces, one class
- `super<A>` — pick a default

Data Classes

CLASSES FOR HOLDING DATA

A `data class` generates `equals`, `hashCode`, `toString`, `copy` and `componentN` functions from the properties in its primary constructor. Use them for DTOs, values and results. `copy()` creates a changed copy, which pairs well with read-only `val` properties.

KEY POINTS

- Only primary-constructor properties are used in generated methods
- Must have at least one constructor parameter
- Cannot be abstract, open, sealed or inner
- Properties in the body are ignored by equals/toString

UNDER THE JVM

- The compiler emits `equals`, `hashCode`, `toString`, `copy`, `component1..N` as normal methods
- `copy` has default arguments for every property (via a synthetic `copy$default`)
- Data classes are not Java records unless annotated `@JvmRecord`

DATA CLASS FEATURES

```
data class User(val id: Int, val name: String, val email: String = "") {
    var lastLogin: Long = 0            // not part of equals/toString
}

val a = User(1, "Ada", "ada@x.com")
val b = User(1, "Ada", "ada@x.com")

println(a)                             // User(id=1, name=Ada, email=ada@x.com)
println(a == b)                        // true (generated equals)
println(a.hashCode() == b.hashCode())  // true
println(setOf(a, b).size)              // 1

val renamed = a.copy(name = "Ada L.")  // change one property
println(renamed)  // User(id=1, name=Ada L., email=ada@x.com)

val (id, name) = a                     // destructuring via component1/2
println("$id: $name")                  // 1: Ada

val users = listOf(a, User(2, "Bob"))
for ((uid, uname) in users) println("$uid $uname")

@JvmRecord
data class Point(val x: Int, val y: Int)   // also a Java record

data class Page<T>(val items: List<T>, val total: Int)   // generic
```

CODE EXPLAINED

- → Two Users with the same constructor values are equal
- → `copy(name = …)` keeps all other properties
- → Destructuring uses `component1()`, `component2()`, in order
- → `@JvmRecord` makes Java see a real record

JAVA EQUIVALENT

- `record User(int id, String name, String email) {}` (Java 16+)
- Records have no `copy`; use a "wither" or a new instance
- Before records: Lombok `@Data` / `@Value`

IDIOMATIC KOTLIN

- ⇄ hand-written equals/hashCode → `data class`
- ⇄ mutable `var` properties in data classes → `val` + `copy()`

GENERATED MEMBERS

MEMBER	BASED ON
`equals / hashCode`	Constructor properties
`toString`	Constructor properties
`copy(…)`	All, with defaults
`component1..N`	Declaration order

RULES

RULE	DETAIL
`≥ 1 param`	Primary constructor
`val / var params`	Required
`Not open/abstract/sealed`	Final only
`Body props`	Excluded
`Can implement interfaces`	Yes

⚠ COMMON MISTAKES

- Mutable `var` properties used as map/set keys, then changed
- Destructuring by position after reordering properties
- Expecting body properties in `toString` / `equals`

✓ TIPS & BEST PRACTICES

- Prefer `val` properties and `copy()` for updates
- Use data classes for API DTOs and JSON models
- Use `data object` for singletons in sealed hierarchies

QUICK SUMMARY

- `data class` — value holder
- equals/hashCode/toString — free
- `copy()` — modified copy
- destructuring — `val (a, b) = x`

Enum Classes

FIXED SETS OF VALUES

An `enum class` defines a fixed set of constants. Enums can have constructor properties, methods, abstract members per constant, and can implement interfaces. `entries` lists all values (preferred over `values()`), and `when` over an enum is checked for exhaustiveness.

KEY POINTS

- `name` and `ordinal` are built in
- `entries` (Kotlin 1.9+) returns an immutable list
- `valueOf("RED")` throws on unknown names
- Use `enumValueOf<T>()` / `enumEntries<T>()` generically

ENUMS WITH DATA AND BEHAVIOR

```
enum class Planet(val massKg: Double, val radiusM: Double) {
    MERCURY(3.303e23, 2.4397e6),
    EARTH(5.976e24, 6.37814e6);

    val gravity get() = 6.67e-11 * massKg / (radiusM * radiusM)
}

enum class Op(val symbol: String) {
    ADD("+") { override fun apply(a: Int, b: Int) = a + b },
    MUL("*") { override fun apply(a: Int, b: Int) = a * b };

    abstract fun apply(a: Int, b: Int): Int
}

enum class Status { ACTIVE, SUSPENDED, CLOSED }

println(Planet.EARTH.gravity)                // ~9.8
println(Op.MUL.apply(3, 4))                  // 12
println(Status.entries)                      // [ACTIVE, SUSPENDED, CLOSED]
println(Status.CLOSED.ordinal)               // 2
println(Status.valueOf("ACTIVE"))            // ACTIVE
println(Status.entries.find { it.name == "PAUSED" })   // null (safe lookup)

fun label(s: Status) = when (s) {            // exhaustive, no else
    Status.ACTIVE    -> "Open"
    Status.SUSPENDED -> "On hold"
    Status.CLOSED    -> "Closed"
}
```

CODE EXPLAINED

→ Each Planet constant passes values to the constructor

→ A semicolon ends the constants when members follow

→ Each Op constant overrides the abstract `apply`

→ `entries.find` is a safe alternative to `valueOf`

JAVA EQUIVALENT

- `enum Planet { EARTH(5.976e24, 6.37814e6); … }` (very similar)
- `Status.values()` returns a new array each call
- `switch` over enums without exhaustiveness checks (before Java 21)

IDIOMATIC KOTLIN

⇄ `Status.values()` → `Status.entries`

⇄ String constants for states → enum (or sealed class for data per state)

BUILT-INS

MEMBER	RETURNS
`name`	Constant name
`ordinal`	Position (0-based)
`entries`	List of all
`valueOf(s)`	Constant or exception
`compareTo`	By ordinal
`enumValueOf<T>(s)`	Generic lookup

ENUM VS SEALED

NEED	USE
Fixed constants	enum class
Same data per value	enum with properties
Different data per case	sealed class
Many instances per case	sealed class

⚠ COMMON MISTAKES

- Persisting `ordinal` (reordering constants breaks data)
- `valueOf` on user input without handling the exception
- Forgetting the semicolon before enum members

✓ TIPS & BEST PRACTICES

- Store `name` (or an explicit code property) in databases
- Use `entries.associateBy { it.code }` for fast lookups
- Let `when` exhaustiveness catch new constants

QUICK SUMMARY

- `enum class` — fixed set
- properties + methods per enum
- `entries` — all values
- exhaustive `when`

Sealed Classes & Interfaces

`sealed class` / `sealed interface` · exhaustive `when` · state & result modeling · `data object` subclasses · same-package rule

CLOSED TYPE HIERARCHIES

A `sealed` class or interface restricts its subtypes to ones declared in the same package and module, so the compiler knows every case. Each subtype can carry different data. `when` over a sealed type needs no `else`, and adding a new subtype flags every `when` that must handle it.

KEY POINTS

- Subtypes: data classes, classes, objects, or `data object`
- Sealed classes are abstract and cannot be instantiated
- Use for UI state, API results, commands, events
- Sealed interfaces allow a class to belong to several hierarchies

UNDER THE JVM

- Sealed classes compile with a private constructor
- Kotlin emits the `PermittedSubclasses` attribute so Java 17+ sees a sealed class
- Exhaustive `when` compiles to `instanceof` checks (no `else` branch needed)

MODELING RESULTS AND STATE

```
sealed interface Result<out T> {
    data class Success<T>(val data: T) : Result<T>
    data class Failure(val error: String, val code: Int = 0) : Result<Nothing>
    data object Loading : Result<Nothing>
}

fun render(r: Result<List<String>>): String = when (r) {   // no else needed
    is Result.Success -> "Got ${r.data.size} items"
    is Result.Failure -> "Error ${r.code}: ${r.error}"
    Result.Loading    -> "Loading..."
}

println(render(Result.Success(listOf("a", "b"))))   // Got 2 items
println(render(Result.Failure("timeout", 504)))     // Error 504: timeout
println(render(Result.Loading))                     // Loading...

sealed class Shape {
    data class Circle(val r: Double) : Shape()
    data class Rect(val w: Double, val h: Double) : Shape()
}
fun area(s: Shape) = when (s) {
    is Shape.Circle -> Math.PI * s.r * s.r
    is Shape.Rect   -> s.w * s.h
}
```

CODE EXPLAINED

- → Each case carries its own data
- → `data object Loading` is a singleton with a readable toString
- → Smart casts give access to `r.data` and `r.error`
- → Adding `Result.Empty` would make both `when`s fail to compile until handled

JAVA EQUIVALENT

- `sealed interface Result permits Success, Failure {}` (Java 17)
- Records as cases: `record Success<T>(T data) implements Result<T>`
- Pattern matching `switch` over sealed types (Java 21)

IDIOMATIC KOTLIN

- ⇄ status codes + nullable fields → sealed class with one case per state
- ⇄ `else` branch on sealed `when` → list every case

SEALED VS OTHERS

TYPE	CASES KNOWN?	DATA PER CASE
`sealed class/interface`	Yes	Different
`enum class`	Yes	Same shape
`abstract class`	No	Different
`interface`	No	Different

SUBTYPE KINDS

KIND	USE
`data class`	Case with data
`data object`	Case with no data
`class`	Case with behavior
`sealed subclass`	Nested hierarchy

⚠ COMMON MISTAKES

- Adding `else` to a sealed `when` and losing compile-time checks
- Declaring subtypes in another package (not allowed)
- Using sealed classes where a simple enum would do

✓ TIPS & BEST PRACTICES

- Model screen state and API results as sealed types
- Use `data object` for cases without data
- Use `Result<Nothing>` with `out T` so failures fit any result type

QUICK SUMMARY

- `sealed` — closed hierarchy
- exhaustive `when`
- different data per case
- `data object` — singleton case

Sheet 64

Objects & Companion Objects

`object` singletons · `companion object` · `@JvmStatic` · factory methods · object expressions (anonymous) · `data object`

SINGLETONS AND CLASS-LEVEL MEMBERS

An `object` declaration creates a **singleton**: one instance, created lazily on first use. A `companion object` inside a class holds members called on the class itself, replacing Java `static`. **Object expressions** create anonymous objects, such as one-off listener implementations.

KEY POINTS

- Objects can extend classes and implement interfaces
- Companion members: `Config.load()` — no instance needed
- A companion can be named and can implement interfaces
- Object expressions can capture local variables

UNDER THE JVM

- `object Registry` compiles to a class with a static `INSTANCE` field
- A companion becomes a static nested `Companion` class; members are called through it
- `@JvmStatic` adds a real static method so Java can call `Config.load()`

OBJECT, COMPANION, ANONYMOUS

```
object Registry {                               // singleton
    private val items = mutableMapOf<String, Int>()
    fun register(name: String, value: Int) { items[name] = value }
    fun count() = items.size
}
Registry.register("a", 1)
println(Registry.count())                       // 1

class User private constructor(val name: String) {
    companion object {
        const val MAX_NAME = 50
        @JvmStatic fun create(name: String): User {   // factory
            require(name.length <= MAX_NAME)
            return User(name.trim())
        }
    }
}
val u = User.create(" Ada ")                    // called on the class
println(User.MAX_NAME)                          // 50

interface Listener { fun onEvent(e: String) }
var events = 0
val listener = object : Listener {              // object expression
    override fun onEvent(e: String) { events++; println("got $e") }
}
listener.onEvent("click")

val point = object { val x = 1; val y = 2 }     // ad-hoc object (local use)
println(point.x + point.y)                      // 3

sealed interface Cmd { data object Quit : Cmd }   // data object: nice toString
println(Cmd.Quit)                               // Quit
```

CODE EXPLAINED

- → `Registry` exists once and is created on first access
- → `User.create` acts like a static factory
- → `@JvmStatic` lets Java call `User.create(…)` directly
- → The anonymous listener can update the captured `events` var

JAVA EQUIVALENT

- Singleton: `enum Registry { INSTANCE; }` or a static holder class
- `static` fields and methods on the class
- Anonymous classes: `new Listener() { … }`

IDIOMATIC KOTLIN

- ⇄ static utility classes → `object` or top-level functions
- ⇄ Java-style `getInstance()` singletons → `object`

FORMS

FORM	USE
`object Name`	Singleton
`companion object`	Class-level members
`companion object Factory`	Named companion
`object : Interface { }`	Anonymous implementation
`data object`	Singleton with toString/equals
`@JvmStatic`	Real static for Java

⚠ COMMON MISTAKES

- Putting mutable global state in objects without thread safety
- Calling companion members from Java without `@JvmStatic` (needs `User.Companion.create`)
- Using objects where dependency injection would be easier to test

✓ TIPS & BEST PRACTICES

- Use companion objects for factories and constants
- Add `@JvmStatic` / `@JvmField` for Java callers
- Prefer top-level functions over objects for stateless helpers

QUICK SUMMARY

- `object` — singleton
- `companion object` — class-level
- `@JvmStatic` — Java static
- object expression — anonymous

Value Classes

TYPE SAFETY WITHOUT ALLOCATION

A **value class** wraps a single value in a distinct type, so a `UserId` cannot be passed where an `OrderId` is expected, even though both are Longs. At runtime the wrapper usually disappears and the raw value is used, so there is no extra object.

KEY POINTS

- Exactly one `val` property in the primary constructor
- Can have methods, init blocks and implement interfaces
- No identity: `===` is not allowed
- `UInt`, `ULong` and `Duration` are value classes

UNDER THE JVM

- `fun find(id: UserId)` compiles to `find-<hash>(long id)` : the name is mangled
- Boxing happens when used as nullable, generic or as an interface type
- Java callers cannot call mangled functions without `@JvmName`

TYPE-SAFE IDS

```
@JvmInline
value class UserId(val value: Long)

@JvmInline
value class OrderId(val value: Long)

@JvmInline
value class Email(val raw: String) {
    init { require("@" in raw) { "invalid email: $raw" } }
    val domain get() = raw.substringAfter("@")
}

fun findUser(id: UserId): String = "user ${id.value}"

val uid = UserId(42)
val oid = OrderId(42)
println(findUser(uid))            // user 42
// findUser(oid)                  // error: OrderId is not UserId
// findUser(42)                   // error: Long is not UserId

val e = Email("ada@example.com")
println(e.domain)                 // example.com

val ids: List<UserId> = listOf(UserId(1), UserId(2))  // boxed in a generic list
val maybe: UserId? = null                             // boxed when nullable
```

CODE EXPLAINED

- → UserId and OrderId are both Longs at runtime but different types at compile time
- → `init` validates every Email on creation
- → Inside a List or as a nullable, the value is boxed in the wrapper
- → Plain uses compile down to a raw `long` / `String`

JAVA EQUIVALENT

- A record `record UserId(long value)` allocates an object
- Project Valhalla value classes are not yet standard Java
- Usually just `long userId` with no type safety

IDIOMATIC KOTLIN

- ⇄ raw `Long` IDs passed around → `@JvmInline value class UserId`
- ⇄ value classes in Java-facing APIs → `@JvmName` or plain types

RULES

RULE	DETAIL
One property	val in primary constructor
@JvmInline	Required on JVM
Allowed	Methods, init, interfaces
Not allowed	Backing fields, ===, lateinit
Boxed when	Nullable, generic, interface type

VALUE VS DATA CLASS

	VALUE CLASS	DATA CLASS
Properties	1	Many
Allocation	Usually none	Always
copy()	No	Yes
Use for	IDs, units	Records, DTOs

⚠ COMMON MISTAKES

- Expecting no boxing inside collections
- Calling mangled functions from Java
- Putting several properties in a value class (only one allowed)

✓ TIPS & BEST PRACTICES

- Wrap IDs, money in cents and units of measure
- Validate in `init` so invalid values never exist
- Use `kotlin.time.Duration` instead of raw millisecond Longs

QUICK SUMMARY

- `@JvmInline value class`
- one property, no allocation
- type-safe IDs and units
- boxed when nullable / generic

Nested, Inner & Anonymous Classes

nested (static by default) · `inner` · `this@Outer` · anonymous objects · local classes

CLASSES INSIDE CLASSES

A class declared inside another is **nested** and cannot see the outer instance, like a Java static nested class. Mark it `inner` to give it a reference to the outer object. Anonymous objects are object expressions, and local classes can be declared inside functions.

KEY POINTS

- Nested is the default: no outer reference, no leak risk
- `inner` classes reach outer members directly
- `this@Outer` names the outer instance
- Nested classes are accessed as `Outer.Nested`

NESTED VS INNER

```
class Tree(val name: String) {
    class Node(val value: Int)                  // nested: no Tree reference

    inner class Branch(val label: String) {     // inner: has Tree reference
        fun path() = "$name/$label"             // reads outer property
        fun owner() = this@Tree                 // explicit outer this
    }
}

val node = Tree.Node(5)                         // no Tree needed
val tree = Tree("oak")
val branch = tree.Branch("left")                // created from an instance
println(branch.path())                          // oak/left

class Form {
    private var submitted = 0
    fun button() = object : Clickable {         // anonymous object
        override fun click() { submitted++ }
    }
}

fun summarize(items: List<Int>): String {
    class Stats(val min: Int, val max: Int)     // local class
    val s = Stats(items.min(), items.max())
    return "${s.min}..${s.max}"
}
println(summarize(listOf(4, 9, 2)))             // 2..9
```

CODE EXPLAINED

- → `Tree.Node` exists independently of any Tree
- → `Branch` must be created from a Tree instance
- → `this@Tree` refers to the enclosing Tree
- → Local classes are visible only inside their function

JAVA EQUIVALENT

- Java inner classes are inner by default; `static` makes them nested
- `Tree.this` instead of `this@Tree`
- Anonymous classes: `new Clickable() { … }`

IDIOMATIC KOTLIN

- ⇄ `inner` by habit → nested unless the outer instance is needed
- ⇄ anonymous objects for single-method interfaces → lambdas (SAM)

KINDS

KIND	OUTER REFERENCE	CREATED AS
Nested	No	`Outer.Nested()`
inner	Yes	`outer.Inner()`
Anonymous object	Captures scope	`object : T { }`
Local class	Captures scope	Inside a function

⚠ COMMON MISTAKES

- Making classes `inner` unnecessarily, keeping the outer object alive
- Trying to create an inner class without an outer instance
- Confusing Kotlin defaults with Java's (reversed)

✓ TIPS & BEST PRACTICES

- Keep nested classes for builders and helper types
- Use `inner` only when you need outer state
- Prefer lambdas over anonymous objects for SAM interfaces

QUICK SUMMARY

- nested — default, no outer
- `inner` — has outer ref
- `this@Outer`
- local / anonymous classes

Delegation

REUSING BEHAVIOR WITH BY

Class delegation (`class A(b: B) : B by b`) implements an interface by forwarding to another object, with no boilerplate, which makes composition as easy as inheritance. **Delegated properties** (`val x by …`) hand a property's get/set to a helper: `lazy`, `observable`, `vetoable`, a map, or your own delegate.

KEY POINTS

- You can override some delegated methods and forward the rest
- `observable` runs a callback after each change
- `vetoable` can reject a change
- Custom delegates implement `getValue` / `setValue`

UNDER THE JVM

- `B by b` generates forwarding methods for every interface member
- A delegated property stores a hidden `$$delegate` field and calls `getValue` / `setValue`
- Kotlin 2.x optimizes some delegates (like references to other properties) to avoid extra objects

CLASS AND PROPERTY DELEGATION

```
interface Repository { fun find(id: Int): String; fun save(x: String) }

class DbRepository : Repository {
    override fun find(id: Int) = "row $id"
    override fun save(x: String) = println("saved $x")
}

class LoggingRepository(private val inner: Repository) : Repository by inner {
    override fun save(x: String) {                  // override one method
        println("LOG save")
        inner.save(x)
    }                                               // find() is forwarded
}

class Settings(map: Map<String, Any?>) {
    val theme: String by map                        // read from map["theme"]
    val fontSize: Int by map
}
val s = Settings(mapOf("theme" to "dark", "fontSize" to 14))
println(s.theme)                                    // dark

class Profile {
    var name: String by Delegates.observable("?") { _, old, new ->
        println("name: $old -> $new")
    }
    var age: Int by Delegates.vetoable(0) { _, _, new -> new >= 0 }
}
val p = Profile()
p.name = "Ada"                                      // name: ? -> Ada
p.age = -5                                          // rejected, stays 0
```

CODE EXPLAINED

- → `Repository by inner` forwards every method not overridden
- → `by map` reads properties from map entries by name
- → `observable` reports each change; `vetoable` can block it
- → Custom delegates implement `ReadWriteProperty` (`getValue` / `setValue`)

JAVA EQUIVALENT

- Forwarding methods written by hand, or the decorator pattern
- Property change support: `PropertyChangeSupport` listeners
- No delegated properties in Java

IDIOMATIC KOTLIN

- ⇄ inheritance to reuse behavior → class delegation with `by`
- ⇄ repeated getter/setter logic → a custom property delegate

STANDARD DELEGATES

DELEGATE	BEHAVIOR
`by lazy { }`	Compute once on first read
`Delegates.observable(init) { }`	Callback after change
`Delegates.vetoable(init) { }`	Approve/reject change
`Delegates.notNull()`	lateinit for primitives
`by map`	Read from Map
`by mutableMap`	Read/write Map
`by ::other`	Alias another property

⚠ COMMON MISTAKES

- Map-backed properties with missing keys (NoSuchElementException on read)
- Expecting delegated classes to forward calls made from inside the delegate back to the wrapper
- Heavy logic in delegates that look like simple properties

✓ TIPS & BEST PRACTICES

- Use class delegation for decorators (logging, caching)
- Use `Delegates.notNull()` for `lateinit` on Int/Long
- Use `by lazy` for expensive, rarely used values

QUICK SUMMARY

- `: I by impl` — class delegation
- `val x by d` — property delegate
- lazy / observable / vetoable
- custom `getValue` / `setValue`

Generics

TYPE PARAMETERS

Generics let classes and functions work with any type while staying type-safe. Declare type parameters in angle brackets, add **upper bounds** with `:` to require capabilities, and use `where` for several bounds. Type arguments are usually inferred, and they are erased at runtime unless `reified`.

KEY POINTS

- The default upper bound is `Any?` (nullable)
- Use `T : Any` to forbid nulls
- `where` lists multiple constraints
- Raw types do not exist: always give type arguments

GENERIC CLASSES AND FUNCTIONS

```
class Box<T>(val item: T) {
    fun <R> map(f: (T) -> R): Box<R> = Box(f(item))
}
val intBox = Box(42)                        // Box<Int> inferred
val strBox = intBox.map { "#$it" }          // Box<String>
println(strBox.item)                        // #42

fun <T> firstOrDefault(list: List<T>, default: T): T =
    list.firstOrNull() ?: default
println(firstOrDefault(emptyList(), "none"))   // none

fun <T : Comparable<T>> maxOf3(a: T, b: T, c: T): T = maxOf(a, maxOf(b, c))
println(maxOf3(3, 9, 4))                    // 9
println(maxOf3("pear", "apple", "fig"))     // pear

fun <T> copyNonNull(src: List<T>): List<T & Any> = src.filterNotNull()

fun <T> describe(x: T) where T : CharSequence, T : Comparable<T> =
    "${x.length} chars, compares to 'm': ${x.compareTo(x)}"

class Cache<K : Any, V : Any> {
    private val map = mutableMapOf<K, V>()
    operator fun get(k: K): V? = map[k]
    operator fun set(k: K, v: V) { map[k] = v }
}
val cache = Cache<String, Int>()
cache["a"] = 1
```

CODE EXPLAINED

- → `map` introduces its own type parameter `R`
- → `T : Comparable<T>` allows `maxOf` on T
- → `T & Any` is a definitely non-null version of T
- → `where` adds several bounds to one parameter

JAVA EQUIVALENT

- `class Box<T>`, `<T extends Comparable<T>> T max(...)`
- `<T extends CharSequence & Comparable<T>>` for multiple bounds
- Raw types like `List` still compile in Java

IDIOMATIC KOTLIN

- ⇄ `Any` parameters with casts → type parameters
- ⇄ unbounded `T` that must not be null → `T : Any`

SYNTAX

FORM	MEANING
`class A<T>`	Generic class
`fun <T> f()`	Generic function
`T : Number`	Upper bound
`where T : A, T : B`	Multiple bounds
`T : Any`	Non-null T
`T & Any`	Definitely non-null
`inline fun <reified T>`	Runtime type (sheet 43)

KOTLIN VS JAVA

KOTLIN	JAVA
`<T : Number>`	<T extends Number>
`out T (sheet 69)`	? extends T
`in T`	? super T
`*`	?

⚠ COMMON MISTAKES

- Assuming `T` is non-null by default (it is `Any?`)
- Checking `is T` without `reified` (erased)
- Over-generalizing simple code

✓ TIPS & BEST PRACTICES

- Let inference fill type arguments when obvious
- Add bounds only for what you call on T
- See variance (sheet 69) for producer/consumer types

QUICK SUMMARY

- `<T>` — type parameter
- `T : Bound` — constraint
- `where` — many bounds
- erased unless `reified`

Variance

out (covariant) · in (contravariant) · declaration-site variance · use-site projections · star projection * · List vs MutableList

WHEN A LIST<DOG> IS A LIST<ANIMAL>

Variance controls subtyping between generic types. `out T` marks a type that only **produces** T, so `Producer<Dog>` is a `Producer<Animal>` (covariant). `in T` marks a type that only **consumes** T, so `Consumer<Animal>` is a `Consumer<Dog>` (contravariant). Kotlin declares this once on the class.

KEY POINTS

- `List<out E>` is covariant; `MutableList<E>` is invariant
- `out` positions: return types; `in` positions: parameters
- Use-site `out` / `in` work like Java wildcards
- `*` means "some unknown type" (read as `out Any?`)

UNDER THE JVM

- Variance is erased: at runtime all are the same class
- Declaration-site `out T` becomes `? extends T` in Java signatures
- Kotlin may add wildcards to Java signatures automatically (`@JvmSuppressWildcards` turns it off)

OUT, IN AND *

```
open class Animal(val name: String)
class Dog(name: String) : Animal(name)

val dogs: List<Dog> = listOf(Dog("Rex"))
val animals: List<Animal> = dogs           // OK: List is covariant (out E)

val mutableDogs = mutableListOf(Dog("Rex"))
// val ma: MutableList<Animal> = mutableDogs   // error: invariant

interface Source<out T> { fun next(): T }                // producer
interface Sink<in T> { fun put(item: T) }                 // consumer

val dogSource: Source<Dog> = object : Source<Dog> {
    override fun next() = Dog("Fido")
}
val animalSource: Source<Animal> = dogSource               // covariant

val animalSink: Sink<Animal> = object : Sink<Animal> {
    override fun put(item: Animal) = println(item.name)
}
val dogSink: Sink<Dog> = animalSink                        // contravariant

fun copy(from: Array<out Animal>, to: Array<in Animal>) {  // use-site
    for (i in from.indices) to[i] = from[i]
}

fun printAll(list: List<*>) = list.forEach { println(it) } // star projection
```

CODE EXPLAINED

- → A List<Dog> can be read as a List<Animal> because List only produces
- → MutableList could accept a Cat if it were covariant, so it is invariant
- → `Source<out T>` returns T; `Sink<in T>` accepts T
- → `Array<out Animal>` is a use-site projection like `? extends Animal`

JAVA EQUIVALENT

- `List<? extends Animal>` (producer) / `List<? super Dog>` (consumer)
- PECS: Producer Extends, Consumer Super
- `List<?>` ≈ Kotlin `List<*>`

IDIOMATIC KOTLIN

- ⇄ wildcards at every use site → declare `out` / `in` on the type once
- ⇄ invariant producer interfaces → mark them `out T`

VARIANCE

KIND	SYNTAX	SUBTYPING
Covariant	`out T`	`P<Dog> is P<Animal>`
Contravariant	`in T`	`C<Animal> is C<Dog>`
Invariant	`T`	No relation
Star	`*`	Unknown T

STANDARD TYPES

TYPE	VARIANCE
`List<out E>`	Covariant
`MutableList<E>`	Invariant
`Comparable<in T>`	Contravariant
`(P) -> R`	in P, out R
`Map<K, out V>`	Covariant values

⚠ COMMON MISTAKES

- Putting an `out T` in a parameter position (compile error)
- Expecting `MutableList<Dog>` to be a `MutableList<Animal>`
- Using `*` and then trying to add elements

✓ TIPS & BEST PRACTICES

- Ask: does this type only produce T (`out`) or only consume it (`in`)?
- Expose read-only interfaces so they can be covariant
- Use `@UnsafeVariance` only when you are sure

QUICK SUMMARY

- `out` — producer, covariant
- `in` — consumer, contravariant
- MutableList — invariant
- `*` — unknown type

Sheet 70

Any, Unit, Nothing & Type Aliases

Any / Any? · Unit · Nothing · typealias · platform types T! preview · type hierarchy diagram

THE TOP AND BOTTOM OF THE TYPE SYSTEM

Any is the supertype of every non-null type and Any? of everything. Unit is a real type with one value, used where Java has void. Nothing is the subtype of everything and has no values: it types expressions that never finish, like throw. typealias gives an existing type a shorter name.

KEY POINTS

- Any has equals, hashCode, toString
- Nothing? has exactly one value: null
- emptyList() returns List<Nothing>, which fits any List<T>
- Type aliases do not create new types (unlike value classes)

SPECIAL TYPES AND ALIASES

```
fun describe(x: Any?): String = when (x) {
    null -> "null"
    is Any -> "${x::class.simpleName}: $x"
}
println(describe(42))                    // Int: 42

fun log(msg: String): Unit = println(msg)   // Unit (usually omitted)
val u: Unit = log("hi")
println(u)                               // kotlin.Unit

fun fail(msg: String): Nothing = throw IllegalStateException(msg)
val age: Int = parseAge() ?: fail("no age")  // Nothing fits as an Int

val empty: List<String> = emptyList()   // List<Nothing> is a List<String>
val nothing: Nothing? = null

typealias UserMap = Map<Int, List<User>>
typealias Handler = (Event) -> Unit
typealias Predicate<T> = (T) -> Boolean

fun register(h: Handler) {}
val isLong: Predicate<String> = { it.length > 10 }
val byId: UserMap = emptyMap()

// Type hierarchy (top to bottom):
// Any?  ⊃  Any  ⊃  String, Int, User...  ⊃  Nothing
//       ⊃  String?, Int?...  ⊃  Nothing?
```

CODE EXPLAINED

- → x::class.simpleName reads the runtime class name
- → Unit prints as kotlin.Unit
- → Because Nothing is a subtype of Int, ?: fail(…) type-checks
- → Aliases shorten long function and generic types

JAVA EQUIVALENT

- Object is Java's top type (Kotlin maps it to Any)
- void methods; Void is a type with only null
- No Nothing or type aliases in Java

IDIOMATIC KOTLIN

- ⇄ Map<Int, List<User>> repeated everywhere → typealias UserMap
- ⇄ aliases for type safety (they give none) → value classes

SPECIAL TYPES

TYPE	VALUES	USED FOR
Any	Any non-null	Top type
Any?	Anything	Top type incl. null
Unit	Unit	No meaningful result
Nothing	None	Never returns
Nothing?	null	Only null

JAVA MAPPING

KOTLIN	JAVA
Any	Object
Unit (return)	void
Nothing	—
String!	Platform type from Java (sheet 77)

⚠ COMMON MISTAKES

- Treating typealias as a new type for validation
- Returning Any and casting everywhere
- Confusing Unit with Nothing

✓ TIPS & BEST PRACTICES

- Use Nothing for helpers that always throw
- Alias long function types used in many signatures
- Avoid Any in public APIs; use generics or sealed types

QUICK SUMMARY

- Any / Any? — top types
- Unit — Kotlin void
- Nothing — never returns
- typealias — just a name

Reading Files

READING TEXT FILES

Kotlin adds extension functions to `java.io.File` and `java.nio.file.Path` that make reading short: `readText()` for the whole file, `readLines()` for a list of lines, and `useLines { }` to process a large file lazily and close it automatically. UTF-8 is the default charset.

KEY POINTS

- `readText` / `readLines` load the whole file into memory
- `useLines` streams lines as a Sequence and closes the file
- `kotlin.io.path` adds the same helpers for `Path`
- Pass a `Charset` for non-UTF-8 files

READING FILES

```
import java.io.File
import kotlin.io.path.*

val text = File("notes.txt").readText()              // whole file
val lines = File("notes.txt").readLines()            // List<String>
println(lines.size)

val errors = File("app.log").useLines { seq ->       // large file, lazy
    seq.filter { "ERROR" in it }.take(10).toList()
}

File("data.csv").bufferedReader().use { reader ->
    val header = reader.readLine()
    reader.lineSequence().forEach { println(it.split(",")) }
}

val p = Path("config/settings.properties")           // kotlin.io.path
if (p.exists()) println(p.readText())
val words = p.readLines().flatMap { it.split(" ") }

val latin = File("old.txt").readText(Charsets.ISO_8859_1)
val bytes = File("image.png").readBytes()            // ByteArray

val fromResources = object {}.javaClass.getResource("/data.json")?.readText()
```

CODE EXPLAINED

- → `readText()` is simplest for small files
- → `useLines` keeps memory flat for big logs
- → `bufferedReader().use { }` closes the reader automatically
- → `Path(...)` with `readText()` uses NIO underneath

JAVA EQUIVALENT

- `Files.readString(Path.of("notes.txt"))` (Java 11+)
- `Files.readAllLines(path)` / `Files.lines(path)` stream
- try-with-resources around `Files.newBufferedReader(path)`

IDIOMATIC KOTLIN

- ⇄ `readLines()` on huge files → `useLines { }`
- ⇄ manual reader close in finally → `.use { }`

READING FUNCTIONS

CALL	RETURNS
`readText()`	String
`readLines()`	List<String>
`useLines { }`	Block result (lazy lines)
`forEachLine { }`	Unit
`readBytes()`	ByteArray
`bufferedReader()`	BufferedReader
`inputStream()`	InputStream

⚠ COMMON MISTAKES

- Loading multi-gigabyte files with `readText()`
- Forgetting to close readers (use `use`)
- Relative paths resolved from an unexpected working directory

✓ TIPS & BEST PRACTICES

- Prefer `Path` / `kotlin.io.path` in new code
- Use `useLines` for anything that could be large
- Read classpath resources with `getResource(...)?.readText()`

QUICK SUMMARY

- `readText()` — whole file
- `readLines()` — list of lines
- `useLines { }` — lazy + closes
- `Path.readText()` — NIO

Writing Files & Paths

writeText / appendText · printWriter().use { } · Path("a") / "b" · createDirectories() · walk() · deleteIfExists()

WRITING FILES AND WORKING WITH PATHS

`writeText` replaces a file's contents and `appendText` adds to it. For many lines, write through `printWriter().use { }` or `bufferedWriter().use { }`. The `kotlin.io.path` package adds Kotlin-style helpers to NIO `Path`, including the `/` operator for joining paths.

KEY POINTS

- `writeText` creates the file if needed (parent folders must exist)
- `Path / "child"` joins segments
- `walk()` lists a directory tree (marked experimental in older versions)
- Use `createDirectories()` before writing to new folders

WRITING AND PATHS

```
import java.io.File
import kotlin.io.path.*

File("out.txt").writeText("Hello\n")              // overwrite
File("out.txt").appendText("More\n")              // append

File("report.txt").printWriter().use { out ->
    out.println("Report")
    listOf("a", "b").forEach { out.println(" - $it") }
}

val base = Path("build") / "reports" / "2026"     // build/reports/2026
base.createDirectories()
val file = base / "summary.csv"
file.writeLines(listOf("id,total", "1,12.50"))
println(file.fileName)                            // summary.csv
println(file.extension)                           // csv
println(file.nameWithoutExtension)                // summary
println(file.parent)                              // build/reports/2026
println(file.exists() && file.isRegularFile())    // true
println(file.fileSize())                          // bytes

Path("src").walk()
    .filter { it.extension == "kt" }
    .forEach { println(it) }

Path("build/tmp/x.txt").deleteIfExists()
File("old").deleteRecursively()                   // careful
file.copyTo(base / "backup.csv", overwrite = true)
```

CODE EXPLAINED

- → `use` closes the writer even if writing fails
- → `/` on Path joins folders and files
- → `extension` and `nameWithoutExtension` parse the name
- → `deleteRecursively()` removes a whole folder tree

JAVA EQUIVALENT

- ☕ `Files.writeString(path, text)` / `StandardOpenOption.APPEND`
- ☕ `Path.of("build", "reports", "2026")` / `resolve("x")`
- ☕ `Files.walk(path)` stream; `Files.createDirectories(path)`

IDIOMATIC KOTLIN

- ⇄ string concatenation for paths → `Path("a") / "b"`
- ⇄ `File` APIs in new code → `Path` + `kotlin.io.path`

WRITING

CALL	DOES
`writeText(s)`	Overwrite
`appendText(s)`	Append
`writeLines(list)`	Lines (Path)
`writeBytes(b)`	Binary
`printWriter().use { }`	Many lines
`bufferedWriter().use { }`	Buffered output

PATH HELPERS

CALL	GIVES
`Path("a") / "b"`	Join
`name / extension`	Parts
`exists() / isDirectory()`	Checks
`createDirectories()`	Make folders
`listDirectoryEntries()`	Children
`walk()`	Tree
`deleteIfExists()`	Delete

⚠ COMMON MISTAKES

- Writing to a folder that does not exist yet
- Using `writeText` in a loop (rewrites the file each time)
- Running `deleteRecursively` on the wrong path

✓ TIPS & BEST PRACTICES

- Create parent folders with `parent?.createDirectories()`
- Write to a temp file then move it for safe replacement
- Use `Files.newBufferedWriter` options for append/create flags

QUICK SUMMARY

- `writeText` / `appendText`
- `printWriter().use { }`
- `Path / "x"` — join
- `walk()` — tree

JSON with kotlinx.serialization

KOTLIN-FIRST JSON

kotlinx.serialization turns `@Serializable` classes into JSON and back with code generated at compile time, so no reflection is needed. It understands Kotlin nullability and default values. A configured `Json` instance controls leniency, unknown keys and pretty printing.

KEY POINTS

- Needs the `kotlin("plugin.serialization")` Gradle plugin
- Default values are used for missing fields
- Non-null fields without defaults must be present
- Jackson (with `jackson-module-kotlin`) and Moshi are alternatives

ENCODE AND DECODE

```
// build.gradle.kts
// plugins { kotlin("plugin.serialization") version "2.2.0" }
// implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.8.1")

import kotlinx.serialization.*
import kotlinx.serialization.json.*

@Serializable
data class User(
    val id: Int,
    val name: String,
    @SerialName("email_address") val email: String? = null,
    val roles: List<String> = emptyList()
)

val user = User(1, "Ada", "ada@x.com", listOf("admin"))
val json = Json.encodeToString(user)
println(json)
// {"id":1,"name":"Ada","email_address":"ada@x.com","roles":["admin"]}

val parser = Json {
    ignoreUnknownKeys = true
    prettyPrint = true
}
val input = """{"id":2,"name":"Bob","extra":true}"""
val back = parser.decodeFromString<User>(input)
println(back)                    // User(id=2, name=Bob, email=null, roles=[])

val list = Json.decodeFromString<List<User>>("""[{"id":3,"name":"Cy"}]""")
val tree = Json.parseToJsonElement("""{"a":{"b":42}}""")
println(tree.jsonObject["a"]?.jsonObject?.get("b"))   // 42
```

CODE EXPLAINED

- → `@SerialName` maps a Kotlin property to a different JSON key
- → Missing `email` and `roles` fall back to their defaults
- → `ignoreUnknownKeys` skips `"extra"` instead of failing
- → `parseToJsonElement` gives a tree for ad-hoc JSON

JAVA EQUIVALENT

- Jackson: `new ObjectMapper().writeValueAsString(user)`
- `mapper.readValue(json, User.class)`
- `@JsonProperty("email_address")` instead of `@SerialName`

IDIOMATIC KOTLIN

- ⇄ reflection-based Gson with Kotlin classes → kotlinx.serialization or Jackson Kotlin module
- ⇄ failing on new server fields → `ignoreUnknownKeys = true`

JSON OPTIONS

OPTION	EFFECT
`ignoreUnknownKeys`	Skip extra fields
`prettyPrint`	Indented output
`encodeDefaults`	Write default values
`explicitNulls = false`	Omit nulls
`isLenient`	Accept relaxed JSON
`coerceInputValues`	Bad values → defaults

ANNOTATIONS

ANNOTATION	USE
`@Serializable`	Generate serializer
`@SerialName("x")`	Rename key
`@Transient`	Skip property
`@Contextual`	Custom serializer at runtime
`@Serializable(with = X::class)`	Custom serializer

⚠ COMMON MISTAKES

- Forgetting the serialization Gradle plugin ("Serializer not found")
- Non-null fields with no default when the JSON may omit them
- Creating a new `Json { }` instance for every call

✓ TIPS & BEST PRACTICES

- Create one configured `Json` and reuse it
- Give optional fields defaults or make them nullable
- Write custom serializers for `LocalDate` and `BigDecimal`

QUICK SUMMARY

- `@Serializable` — generated
- `encodeToString` / `decodeFromString`
- `Json { ignoreUnknownKeys }`
- `@SerialName` — rename

Resources & use

CLOSING RESOURCES AND LOADING BUNDLED FILES

`use { }` works on any `Closeable` or `AutoCloseable`: it runs the block and then closes the resource, even when an exception is thrown. **Classpath resources** are files packaged in `src/main/resources`; read them with `getResource` or `getResourceAsStream`, never with file paths.

KEY POINTS

- `use` returns the block's result
- Nest `use` calls for several resources
- Resource paths start with `/` for the classpath root
- Resources inside a jar are not files: read them as streams

USE AND RESOURCES

```
import java.util.Properties

val count = File("data.csv").bufferedReader().use { r ->
    r.lineSequence().count()              // reader closed afterwards
}

FileInputStream("in.bin").use { input ->
    FileOutputStream("out.bin").use { output ->
        input.copyTo(output)              // both closed in reverse order
    }
}

class Connection(val url: String) : AutoCloseable {
    init { println("open $url") }
    override fun close() = println("close $url")
}
Connection("db://x").use { println("query on ${it.url}") }
// open db://x / query on db://x / close db://x

// src/main/resources/config.properties
object AppConfig {
    private val props = Properties().apply {
        AppConfig::class.java.getResourceAsStream("/config.properties")
            ?.use { load(it) } ?: error("config.properties missing")
    }
    val dbUrl: String get() = props.getProperty("db.url")
    val poolSize: Int get() = props.getProperty("db.pool", "5").toInt()
}

val template = javaClass.getResource("/templates/email.html")?.readText()
```

CODE EXPLAINED

- → `use` closes the reader once counting finishes
- → Nested `use` closes the output, then the input
- → Any class implementing `AutoCloseable` works with `use`
- → `getResourceAsStream` works both from a folder and inside a jar

JAVA EQUIVALENT

- try-with-resources: `try (var r = Files.newBufferedReader(p)) { … }`
- `getClass().getResourceAsStream("/config.properties")`
- `Properties.load(InputStream)` is the same API

IDIOMATIC KOTLIN

- ⇄ `try { … } finally { x.close() }` → `x.use { }`
- ⇄ `File("src/main/resources/x")` → `getResource("/x")`

USE

ITEM	DETAIL
Works on	Closeable, AutoCloseable
Returns	Block result
Exception in block	Still closes
Exception in close	Added as suppressed
Several	Nest use blocks

RESOURCES

CALL	GIVES
getResource("/x")	URL?
getResourceAsStream("/x")	InputStream?
?.readText()	Text or null
Leading /	From classpath root
No leading /	Relative to class package

⚠ COMMON MISTAKES

- Reading resources with `File` paths (breaks inside a jar)
- Forgetting the leading `/`
- Not handling a null resource (missing file)

✓ TIPS & BEST PRACTICES

- Wrap every stream, reader, socket and connection in `use`
- Fail fast with `?: error("missing …")` for required resources
- Keep config in `src/main/resources` and override with env vars

QUICK SUMMARY

- `use { }` — auto-close
- `AutoCloseable` — your types too
- `getResource("/x")` — bundled files
- null check — missing resource

Packages & Imports

ORGANIZING CODE

A `package` line at the top of a file sets the namespace for everything in it. `import` brings in classes, top-level functions, properties and even single enum constants. `import … as` renames an import to avoid clashes. Many `kotlin.*` packages are imported by default.

KEY POINTS

- Package names do not have to match folders (but should)
- Import a top-level function: `import com.x.util.slugify`
- Wildcards `import kotlin.math.*` are allowed
- No `import static`: functions are imported directly

PACKAGES AND IMPORTS

```
@file:JvmName("StringTools")                // Java sees class StringTools
package com.example.util

import java.time.LocalDate
import java.util.Date as LegacyDate         // alias avoids a clash
import kotlin.math.*                        // wildcard
import com.example.model.User
import com.example.model.Status.ACTIVE      // single enum constant
import com.example.format.slugify           // top-level function

fun label(u: User) = "${u.name.slugify()} (${LocalDate.now()})"

fun isActive(s: com.example.model.Status) = s == ACTIVE

val circle = PI * 2.0.pow(2)                // from kotlin.math

fun toLegacy(d: LocalDate): LegacyDate =
    LegacyDate.from(d.atStartOfDay(ZoneId.systemDefault()).toInstant())
```

CODE EXPLAINED

- → `@file:JvmName` must come before the package line
- → `as LegacyDate` lets both Date types live in one file
- → Top-level functions import like classes
- → Fully qualified names work without imports

JAVA EQUIVALENT

- ☕ `package com.example.util;` with a semicolon, matching folders
- ☕ `import static java.lang.Math.*;` for static members
- ☕ No import aliases in Java

IDIOMATIC KOTLIN

- ⇄ `import static` thinking → import the function directly
- ⇄ name clashes resolved with fully qualified names → `import … as`

DEFAULT IMPORTS

PACKAGE	CONTAINS
`kotlin.*`	Core types, Pair, error()
`kotlin.collections.*`	List, Map, listOf
`kotlin.text.*`	String functions, Regex
`kotlin.ranges.*`	Ranges
`kotlin.io.*`	println, readln
`kotlin.sequences.*`	Sequence
`java.lang.* (JVM)`	Thread, Math…

IMPORT FORMS

FORM	IMPORTS
`import a.b.C`	A class
`import a.b.f`	A top-level function
`import a.b.*`	All in package
`import a.b.C as D`	Alias
`import a.E.X`	Enum constant

⚠ COMMON MISTAKES

- Two classes with the same name imported without an alias
- Package/folder mismatch confusing tools and teammates
- Putting `@file:` annotations after the package line

✓ TIPS & BEST PRACTICES

- Let IntelliJ optimize imports on save
- Use aliases for clashing names like `java.util.Date`
- Keep one package per folder

QUICK SUMMARY

- `package` — namespace
- `import` — classes and functions
- `as` — alias
- kotlin.* — imported by default

Dependencies & Version Catalogs

MANAGING LIBRARIES

Dependencies are declared in `build.gradle.kts` with configurations like `implementation` and `testImplementation`. A **version catalog** (`gradle/libs.versions.toml`) keeps every version in one place and gives type-safe accessors like `libs.ktor.server.core`. **BOMs** align versions across a library family.

KEY POINTS

- `implementation` hides the dependency from consumers; `api` exposes it
- Catalog aliases use dashes; accessors use dots
- Kotlin compiler plugins: serialization, spring, jpa, compose
- See the Gradle set for full detail

VERSION CATALOG + BUILD SCRIPT

```
// gradle/libs.versions.toml
// [versions]
// kotlin = "2.2.0"
// coroutines = "1.10.2"
// ktor = "3.2.0"
//
// [libraries]
// coroutines-core = { module = "org.jetbrains.kotlinx:kotlinx-coroutines-core",
//                     version.ref = "coroutines" }
// ktor-server-core = { module = "io.ktor:ktor-server-core",
//                      version.ref = "ktor" }
// coroutines-bom = { module = "org.jetbrains.kotlinx:kotlinx-coroutines-bom",
//                    version.ref = "coroutines" }
// mockk = "io.mockk:mockk:1.14.2"
//
// [plugins]
// kotlin-jvm = { id = "org.jetbrains.kotlin.jvm", version.ref = "kotlin" }
// kotlin-serialization = { id = "org.jetbrains.kotlin.plugin.serialization",
//                          version.ref = "kotlin" }

// build.gradle.kts
plugins {
    alias(libs.plugins.kotlin.jvm)
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    implementation(libs.coroutines.core)
    implementation(libs.ktor.server.core)
    implementation(platform(libs.coroutines.bom))   // BOM
    testImplementation(kotlin("test"))
    testImplementation(libs.mockk)
}
```

CODE EXPLAINED

- → `version.ref` shares one version across several libraries
- → `coroutines-core` becomes `libs.coroutines.core`
- → `alias(libs.plugins.…)` applies plugins from the catalog
- → `platform(...)` imports a BOM to align versions

JAVA EQUIVALENT

- Maven `<dependencyManagement>` with a BOM
- Gradle Java projects use the same catalogs
- `<properties><kotlin.version>` for shared versions in Maven

IDIOMATIC KOTLIN

- ⇄ versions scattered in build files → `libs.versions.toml`
- ⇄ mismatched Kotlin library versions → BOMs / one `kotlin` version

CONFIGURATIONS

NAME	USE
`implementation`	Compile + runtime, private
`api`	Exposed to consumers (libraries)
`compileOnly`	Compile only
`runtimeOnly`	Runtime only
`testImplementation`	Tests
`kapt / ksp`	Annotation processing

COMMON LIBRARIES

LIBRARY	FOR
`kotlinx-coroutines-core`	Coroutines
`kotlinx-serialization-json`	JSON
`kotlinx-datetime`	Dates (multiplatform)
`ktor-*`	HTTP server/client
`mockk / kotest`	Testing
`exposed`	SQL DSL / ORM

⚠ COMMON MISTAKES

- Kotlin plugin version different from the Kotlin stdlib/library expectations
- Using `api` everywhere and leaking dependencies
- Hard-coding versions in several modules

✓ TIPS & BEST PRACTICES

- Run `./gradlew dependencies` to debug version conflicts
- Use KSP instead of kapt for annotation processing when supported
- Keep one catalog for the whole repository

QUICK SUMMARY

- `implementation` — normal deps
- `libs.versions.toml` — one place
- `platform()` — BOMs
- compiler plugins — per feature

Calling Java from Kotlin

platform types T! · getters/setters as properties · nullability annotations · SAM conversions · Java collections · checked exceptions ignored

USING JAVA LIBRARIES

Kotlin calls Java code directly. Java getters and setters appear as **properties**, single-method interfaces accept lambdas, and checked exceptions need no handling. Values from Java have **platform types** (`String!`): Kotlin does not know if they can be null, so you decide when you assign them.

KEY POINTS

- `T!` can be treated as `T` or `T?`; choosing `T` risks an NPE
- Nullability annotations (`@Nullable`, JSpecify) turn platform types into real types
- `getName()` → `name`; `isActive()` → `isActive`
- Java keywords that clash in Kotlin need backticks: `` `is` ``

UNDER THE JVM

- Kotlin inserts a null check when a platform value is assigned to a non-null type
- Java collections are read as `(Mutable)List<T!>`: read-only or mutable is your choice
- `@NotNull` / `@Nullable` from JetBrains, JSpecify, Jakarta and Android are understood

JAVA APIS FROM KOTLIN

```
import java.time.LocalDate
import java.util.concurrent.Executors

val today = LocalDate.now()
println(today.year)                          // getYear() as a property
println(today.dayOfWeek)                     // getDayOfWeek()

val sb = java.lang.StringBuilder()
sb.append("a").append("b")

val props = System.getProperties()
val home: String? = System.getenv("HOME")  // platform type → choose nullable
val path: String = System.getProperty("user.dir")   // checked at assignment

val executor = Executors.newFixedThreadPool(2)
executor.submit { println("running") }     // lambda → Runnable (SAM)

val list = java.util.ArrayList<String>()   // MutableList at the Kotlin side
list.add("x")

val file = java.io.File("x.txt")
val text = file.readText()  // no try/catch needed for IOException

val obj = SomeJavaClass()
obj.`is`(5)                                  // Java method named "is"
obj.name = "Ada"                             // calls setName("Ada")
```

CODE EXPLAINED

→ `today.year` calls `getYear()`

→ Assigning `System.getenv` to `String?` keeps null handling explicit

→ `submit { }` converts the lambda to a `Runnable`

→ Checked exceptions from Java are not forced to be caught

JAVA EQUIVALENT

- `today.getYear()`, `obj.setName("Ada")`
- `executor.submit(() -> System.out.println("running"))`
- Checked exceptions must be caught or declared

IDIOMATIC KOTLIN

⇄ trusting platform types as non-null → declare `T?` unless the API guarantees non-null

⇄ `obj.getName()` → `obj.name`

JAVA → KOTLIN

JAVA	KOTLIN SEES
`getName() / setName()`	name property
`isActive()`	isActive property
`String (no annotation)`	String!
`@Nullable String`	String?
`@NotNull String`	String
`List<String>`	(Mutable)List<String!>
`void`	Unit
`Object`	Any

KEYWORDS NEEDING BACKTICKS

JAVA NAME	KOTLIN CALL
`is`	obj.`is`()
`in`	obj.`in`
`object`	obj.`object`
`when`	obj.`when`()

⚠ COMMON MISTAKES

- NPE from a platform-typed value assumed non-null
- Modifying a Java list that is actually unmodifiable (UnsupportedOperationException)
- Forgetting that Java collections are mutable underneath

✓ TIPS & BEST PRACTICES

- Annotate your own Java code with JSpecify `@Nullable` / `@NonNull`
- Wrap risky Java APIs in small Kotlin functions with clear null types
- Read IntelliJ's platform type hints

QUICK SUMMARY

- getters → properties
- `T!` — platform type
- lambdas → SAM interfaces
- no checked exceptions

Calling Kotlin from Java

`@JvmStatic` · `@JvmOverloads` · `@JvmField` · `@JvmName` / `@file:JvmName` · `Companion` access · `@Throws`

MAKING KOTLIN NICE FOR JAVA

Kotlin code is callable from Java, but some features look odd there: top-level functions live in a `FileNameKt` class, companion members go through `Companion`, defaults do not exist, and properties become getters. A handful of `@Jvm…` annotations shape what Java sees.

KEY POINTS

- Properties become `getX()` / `setX()`; `isX` stays `isX()`
- Default parameters need `@JvmOverloads`
- Extension functions become static methods taking the receiver
- Non-null parameters are checked at runtime

UNDER THE JVM

- Top-level declarations in `Util.kt` compile to class `UtilKt` with static members
- `@JvmField` removes the getter and exposes a public field
- Inline functions with reified types cannot be called from Java

KOTLIN SIDE AND JAVA SIDE

```kotlin
// Kotlin: Prices.kt
@file:JvmName("Prices")
package com.example

fun withTax(amount: Double, rate: Double = 0.06) = amount * (1 + rate)

class Product @JvmOverloads constructor(
    @JvmField val sku: String,
    val name: String = "",
    var price: Double = 0.0
) {
    companion object {
        @JvmStatic fun of(sku: String) = Product(sku)
        const val CURRENCY = "USD"
    }

    @Throws(java.io.IOException::class)
    fun export(path: String) { java.io.File(path).writeText(name) }
}

fun String.toSlug() = lowercase().replace(" ", "-")

/* Java usage:
   double t = Prices.withTax(10.0, 0.06);      // no default: pass both
   Product p = new Product("A1");              // @JvmOverloads
   String s = p.sku;                           // @JvmField: public field
   p.setPrice(9.99);                           // property setter
   Product q = Product.of("B2");               // @JvmStatic
   String c = Product.CURRENCY;                // const → static field
   String slug = Prices.toSlug("Hello World"); // extension → static
   try { p.export("x.txt"); } catch (IOException e) { }
*/
```

CODE EXPLAINED

- → `@file:JvmName("Prices")` replaces the default `PricesKt` class name
- → `@JvmOverloads` generates the shorter constructors
- → `@JvmStatic` avoids `Product.Companion.of(...)`
- → `@Throws` lets Java catch the checked IOException

JAVA EQUIVALENT

- This is the Java view: static methods, fields and overloads
- Without `@JvmStatic`: `Product.Companion.of("B2")`
- Without `@JvmName`: `PricesKt.withTax(…)`

IDIOMATIC KOTLIN

- ⇄ Java callers using `Companion` and `…Kt` names → `@JvmStatic` and `@file:JvmName`
- ⇄ Java callers passing every default → `@JvmOverloads`

ANNOTATIONS

ANNOTATION	JAVA SEES
`@file:JvmName("X")`	Class X for top-levels
`@JvmStatic`	Static method
`@JvmOverloads`	Overloads for defaults
`@JvmField`	Public field, no getter
`@JvmName("f")`	Renamed method
`@Throws(E::class)`	throws clause
`@JvmRecord`	Java record
`@JvmSuppressWildcards`	No ? extends

⚠ COMMON MISTAKES

- Leaving Java callers with `Companion` and `…Kt` names
- Assuming Java can use default arguments
- Passing null from Java into non-null parameters (runtime exception)

✓ TIPS & BEST PRACTICES

- Design Kotlin APIs used by Java with these annotations from the start
- Use `@JvmName` to fix clashes from generic erasure
- Test the Java call site in a small Java file

QUICK SUMMARY

- `@file:JvmName` — class name
- `@JvmStatic` — static
- `@JvmOverloads` — defaults
- `@JvmField` / `@Throws`

Standard Library Tour

EVERYDAY UTILITIES

The Kotlin standard library covers math (`kotlin.math`), randomness (`Random`), durations and timing (`kotlin.time`), plus text, collections and I/O helpers. Dates on the JVM use `java.time` directly; multiplatform code uses **kotlinx-datetime**.

KEY POINTS

- `Duration` literals: `5.seconds`, `250.milliseconds`
- `measureTime { }` returns a Duration; `measureTimedValue` adds the result
- `Random(seed)` gives repeatable sequences
- kotlinx-datetime is a separate dependency

MATH, RANDOM, TIME

```
import kotlin.math.*
import kotlin.random.Random
import kotlin.time.*
import kotlin.time.Duration.Companion.minutes
import kotlin.time.Duration.Companion.seconds
import java.time.*

println(hypot(3.0, 4.0))                  // 5.0
println(log2(1024.0))                     // 10.0
println(round(2.5))                       // 2.0 (half to even)
println(abs(-7))                          // 7

println(Random.nextInt(1, 7))             // 1..6
println(listOf("a", "b", "c").random())   // random element
val seeded = Random(42)                   // repeatable sequence

val timeout = 2.minutes + 30.seconds
println(timeout)                          // 2m 30s
println(timeout.inWholeSeconds)           // 150

val elapsed = measureTime { Thread.sleep(100) }
println(elapsed)                          // ~100ms
val (sum, took) = measureTimedValue { (1..1_000_000).sum() }

val due = LocalDate.of(2026, 11, 12).plusDays(30)
println(due)                              // 2026-12-12
println(Duration.between(LocalTime.of(9, 0), LocalTime.of(17, 30)))   // PT8H30M
val zoned = ZonedDateTime.now(ZoneId.of("America/Chicago"))
println(timeout.toJavaDuration())         // PT2M30S
```

CODE EXPLAINED

- → `kotlin.math` functions are top-level, no `Math.` prefix
- → `Duration` arithmetic reads like English
- → `measureTimedValue` returns the result and the time
- → `toJavaDuration()` bridges to `java.time.Duration`

JAVA EQUIVALENT

- `Math.hypot`, `ThreadLocalRandom.current().nextInt(1, 7)`
- `Duration.ofMinutes(2).plusSeconds(30)`
- `System.nanoTime()` before and after for timing

IDIOMATIC KOTLIN

- ⇄ `Long` milliseconds everywhere → `kotlin.time.Duration`
- ⇄ `System.currentTimeMillis()` timing → `measureTime { }`

PACKAGES

PACKAGE	HAS
`kotlin.math`	sqrt, pow, abs, round, PI
`kotlin.random`	Random
`kotlin.time`	Duration, measureTime
`kotlin.text`	Regex, string functions
`kotlin.collections`	Lists, maps, ops
`kotlin.io`	println, readln, File ext
`kotlin.io.path`	Path extensions

DATES

NEED	USE
`JVM only`	java.time (LocalDate…)
`Multiplatform`	kotlinx-datetime
`Durations`	kotlin.time.Duration
`Timestamps`	Instant (java.time or kotlin.time)

⚠ COMMON MISTAKES

- Using `java.util.Date` in new code
- Expecting `round(2.5)` to be 3.0
- Raw Long timeouts with unclear units

✓ TIPS & BEST PRACTICES

- Use `Duration` for timeouts and delays (coroutines accept it)
- Seed `Random` in tests for repeatable results
- Use `java.time` on the JVM; it is excellent

QUICK SUMMARY

- `kotlin.math` — math
- `Random` — randomness
- `Duration` / `measureTime`
- `java.time` — dates on JVM

kotlin.test & JUnit

kotlin.test · @Test · assertEquals / assertFailsWith · JUnit Jupiter · backtick test names · runTest for coroutines

UNIT TESTING KOTLIN

kotlin.test provides annotations and assertions that run on JUnit 5/6 on the JVM. Test files live in `src/test/kotlin`. Kotlin lets you name test functions with backticks, so test names can be full sentences. Coroutine code is tested with `runTest` from kotlinx-coroutines-test.

KEY POINTS

- Add `testImplementation(kotlin("test"))` and `useJUnitPlatform()`
- `assertFailsWith<T> { }` returns the exception for further checks
- JUnit annotations (`@BeforeEach` , `@Nested`) work as usual
- See the JUnit set for the full JUnit feature list

A TEST CLASS

```
import kotlin.test.*
import kotlinx.coroutines.test.runTest

class PriceCalculatorTest {
    private lateinit var calc: PriceCalculator

    @BeforeTest
    fun setUp() { calc = PriceCalculator(taxRate = 0.06) }

    @Test
    fun `adds tax to the subtotal`() {
        assertEquals(10.6, calc.total(10.0), absoluteTolerance = 0.001)
    }

    @Test
    fun `rejects negative amounts`() {
        val e = assertFailsWith<IllegalArgumentException> { calc.total(-1.0) }
        assertContains(e.message.orEmpty(), "negative")
    }

    @Test
    fun `discount codes are case-insensitive`() {
        assertTrue(calc.isValidCode("SAVE10"))
        assertTrue(calc.isValidCode("save10"))
        assertNull(calc.findCode("NOPE"))
    }

    @Test
    fun `loads prices asynchronously`() = runTest {
        val prices = calc.loadPrices()          // suspend function
        assertEquals(3, prices.size)
    }
}
```

CODE EXPLAINED

- → Backtick names show as readable sentences in reports
- → `absoluteTolerance` compares doubles safely
- → `assertFailsWith` checks the exception type and returns it
- → `runTest` runs suspend code and skips `delay` s instantly

JAVA EQUIVALENT

- `@Test void addsTaxToSubtotal() { assertEquals(10.6, calc.total(10.0), 0.001); }`
- `assertThrows(IllegalArgumentException.class, () -> calc.total(-1))`
- `@DisplayName("adds tax")` instead of backtick names

IDIOMATIC KOTLIN

- ⇄ camelCase test names → backtick sentences
- ⇄ try/catch + fail() → `assertFailsWith<T> { }`

ASSERTIONS

FUNCTION	CHECKS
`assertEquals(e, a)`	Equality
`assertNotEquals`	Inequality
`assertTrue / assertFalse`	Boolean
`assertNull / assertNotNull`	Null (smart casts)
`assertIs<T>(x)`	Type (smart casts)
`assertContains(c, x)`	Collection / string
`assertFailsWith<T> { }`	Exception
`fail(msg)`	Force failure

ANNOTATIONS

KOTLIN.TEST	JUNIT
`@Test`	@Test
`@BeforeTest`	@BeforeEach
`@AfterTest`	@AfterEach
`@Ignore`	@Disabled

⚠ COMMON MISTAKES

- Forgetting `useJUnitPlatform()` so no tests run
- Comparing doubles without a tolerance
- Using `runBlocking` in tests and waiting for real delays

✓ TIPS & BEST PRACTICES

- Use backtick names that describe behavior
- Use `assertNotNull(x)` to get a non-null smart cast
- Use AssertJ or Kotest matchers for richer assertions

QUICK SUMMARY

- `kotlin.test` on JUnit
- backtick test names
- `assertFailsWith<T>`
- `runTest` — coroutines

Kotest & MockK

Kotest specs (StringSpec, FunSpec) · shouldBe matchers · mockk<T>() · every { } / verify { } · coEvery · relaxed mocks

KOTLIN-NATIVE TESTING TOOLS

Kotest is a Kotlin test framework with expressive specs and infix matchers like `shouldBe`. **MockK** is a mocking library built for Kotlin: it mocks final classes, objects, extension functions and suspend functions, with `every { }` to stub and `verify { }` to check calls.

KEY POINTS

- Kotest runs on the JUnit Platform
- Matchers: `shouldBe`, `shouldContain`, `shouldThrow<T>`
- MockK `coEvery` / `coVerify` handle suspend functions
- `relaxed = true` returns defaults for unstubbed calls

KOTEST SPEC WITH MOCKK

```
import io.kotest.core.spec.style.FunSpec
import io.kotest.matchers.shouldBe
import io.kotest.matchers.collections.shouldContain
import io.kotest.assertions.throwables.shouldThrow
import io.mockk.*

class OrderServiceTest : FunSpec({
    val repo = mockk<OrderRepository>()
    val mailer = mockk<Mailer>(relaxed = true)
    val service = OrderService(repo, mailer)

    test("places an order and sends email") {
        every { repo.save(any()) } returns Order(1, "Ada", 25.0)
        val order = service.place("Ada", 25.0)

        order.id shouldBe 1
        verify(exactly = 1) { mailer.send("Ada", match { "confirmed" in it }) }
    }

    test("loads recent orders") {
        coEvery { repo.recent(any()) } returns listOf(Order(2, "Bob", 5.0))
        service.recentFor("Bob").map { it.customer } shouldContain "Bob"
        coVerify { repo.recent("Bob") }
    }

    test("rejects empty customer") {
        shouldThrow<IllegalArgumentException> { service.place("", 1.0) }
    }
})
```

CODE EXPLAINED

- → `every { … } returns …` stubs a call
- → `relaxed = true` lets `mailer.send` run without a stub
- → `match { }` checks an argument with a predicate
- → `coEvery` / `coVerify` handle the suspend function `recent`

JAVA EQUIVALENT

- Mockito: `when(repo.save(any())).thenReturn(order)`
- `verify(mailer, times(1)).send(eq("Ada"), contains("confirmed"))`
- Mockito needs `mockito-inline` (now default) to mock final classes

IDIOMATIC KOTLIN

- ⇄ Mockito with Kotlin final classes and suspend funs → MockK
- ⇄ many `assertEquals` lines → Kotest matchers

KOTEST SPEC STYLES

STYLE	LOOKS LIKE
StringSpec	"name" { }
FunSpec	test("name") { }
DescribeSpec	describe / it
BehaviorSpec	given / when / then
ShouldSpec	should("…") { }

MOCKK

CALL	DOES
`mockk<T>()`	Strict mock
`mockk<T>(relaxed = true)`	Defaults for all
`every { } returns x`	Stub
`every { } throws e`	Stub exception
`verify { }`	Check call
`slot<T>() / capture`	Capture args
`mockkObject(Obj)`	Mock an object

⚠ COMMON MISTAKES

- Unstubbed calls on strict mocks (MockKException)
- Using `every` for suspend functions (use `coEvery`)
- Over-mocking simple value objects

✓ TIPS & BEST PRACTICES

- Prefer fakes for simple dependencies; mock at boundaries
- Use `confirmVerified(mock)` to catch unexpected calls
- Pick one Kotest style per project

QUICK SUMMARY

- Kotest — specs + matchers
- `shouldBe` — infix assertions
- MockK — `every` / `verify`
- `coEvery` — suspend

Lint, Docs & Scripting

ktlint · detekt · Dokka · Kotlin scripts `.main.kts` · `@file:DependsOn` · IntelliJ Java-to-Kotlin converter

KEEPING KOTLIN CODE HEALTHY

ktlint enforces the official Kotlin style and can auto-format, **detekt** finds code smells and complexity problems, and **Dokka** generates documentation from KDoc. Kotlin **scripts** (`.main.kts`) run directly and can pull Maven dependencies, which makes Kotlin usable for automation.

KEY POINTS

- Official style guide: kotlinlang.org coding conventions
- Both ktlint and detekt have Gradle plugins and IntelliJ integration
- IntelliJ: Code → Convert Java File to Kotlin File
- Scripts need `kotlin` on the PATH

BUILD PLUGINS AND A SCRIPT

```
// build.gradle.kts
plugins {
    kotlin("jvm") version "2.2.0"
    id("org.jlleitschuh.gradle.ktlint") version "12.1.2"
    id("io.gitlab.arturbosch.detekt") version "1.23.8"
    id("org.jetbrains.dokka") version "2.0.0"
}
detekt {
    buildUponDefaultConfig = true
    config.setFrom("config/detekt.yml")
}
// ./gradlew ktlintCheck   ./gradlew ktlintFormat
// ./gradlew detekt        ./gradlew dokkaGenerate

// cleanup.main.kts  — run with:  kotlin cleanup.main.kts logs
@file:DependsOn("com.github.ajalt.clikt:clikt-jvm:5.0.3")

import java.io.File

val dir = File(args.firstOrNull() ?: ".")
val old = dir.walk()
    .filter { it.isFile && it.extension == "log" }
    .filter { System.currentTimeMillis() - it.lastModified() > 7 * 86_400_000L }
    .toList()
old.forEach { println("deleting ${it.name}"); it.delete() }
println("Removed ${old.size} files")
```

CODE EXPLAINED

→ ktlint checks and fixes formatting

→ detekt reports long methods, magic numbers and other smells

→ Dokka turns KDoc into an HTML site

→ The script reads `args` and can declare dependencies inline

JAVA EQUIVALENT

- Checkstyle / Spotless for formatting, PMD / SpotBugs for smells
- Javadoc for API docs
- JBang runs Java scripts with dependencies

IDIOMATIC KOTLIN

⇄ style debates in reviews → ktlint in CI

⇄ bash scripts that grew too big → `.main.kts` scripts

TOOLS

TOOL	DOES
ktlint	Style check + format
detekt	Static analysis
Dokka	API docs from KDoc
IntelliJ inspections	Live warnings
Spotless	Format (ktlint inside)
Kover	Kotlin code coverage

SCRIPT FEATURES

FEATURE	SYNTAX
Run	kotlin x.main.kts
Args	args array
Dependency	@file:DependsOn("g:a:v")
Repository	@file:Repository("url")
Import script	@file:Import("other.main.kts")

⚠ COMMON MISTAKES

- Adding detekt with the default config and drowning in warnings (baseline first)
- Converting Java to Kotlin and keeping Java style (`!!`, getters)
- Mixing formatter versions between IDE and CI

✓ TIPS & BEST PRACTICES

- Create a detekt baseline for existing code
- Run ktlint in a pre-commit hook
- Clean up converted code: remove `!!`, use properties and data classes

QUICK SUMMARY

- ktlint — style
- detekt — smells
- Dokka — docs
- `.main.kts` — scripts

Coroutine Basics

LIGHTWEIGHT CONCURRENCY

Coroutines are lightweight tasks that can **suspend** without blocking a thread. A `suspend fun` can pause at `delay` or other suspending calls and resume later, possibly on another thread. Builders start coroutines: `launch` fires and forgets, `runBlocking` bridges normal code to coroutines (mainly in `main` and tests).

KEY POINTS

- Add `kotlinx-coroutines-core`; language support is built in
- Suspend functions can only be called from coroutines or other suspend functions
- Thousands of coroutines can run on a few threads
- `delay` suspends; `Thread.sleep` blocks (avoid it in coroutines)

UNDER THE JVM

- A `suspend fun` gets a hidden `Continuation` parameter (continuation-passing style)
- Its body compiles to a state machine; each suspension point is a state
- Suspending returns the special `COROUTINE_SUSPENDED` marker instead of blocking

FIRST COROUTINES

```
import kotlinx.coroutines.*

suspend fun fetchUser(id: Int): String {
    delay(500)                          // suspends, does not block the thread
    return "User$id"
}

fun main() = runBlocking {              // bridge from regular code
    println("start on ${Thread.currentThread().name}")

    val job = launch {                  // runs concurrently
        repeat(3) { i ->
            println("tick $i")
            delay(200)
        }
    }

    val user = fetchUser(1)             // waits here (suspends) for 500 ms
    println("got $user")
    job.join()                          // wait for launch to finish
    println("done")
}

// 100,000 coroutines are fine; 100,000 threads are not
fun many() = runBlocking {
    repeat(100_000) { launch { delay(1000); print(".") } }
}
```

CODE EXPLAINED

- → `runBlocking` blocks `main` until all children finish
- → `launch` starts a coroutine that runs alongside the caller
- → Ticks print while `fetchUser` is suspended
- → `job.join()` suspends until the launched coroutine ends

JAVA EQUIVALENT

- Threads: `new Thread(() -> …).start()` or an `ExecutorService`
- Virtual threads (Java 21): `Thread.ofVirtual().start(…)`
- `CompletableFuture.supplyAsync(…)` for async results

IDIOMATIC KOTLIN

- ⇄ `Thread.sleep` inside coroutines → `delay`
- ⇄ `runBlocking` inside app code → `suspend` all the way, scopes at the edges

BUILDING BLOCKS

ITEM	ROLE
`suspend fun`	Can pause and resume
`runBlocking { }`	Block thread until done
`launch { }`	Start, returns Job
`async { }`	Start, returns Deferred (sheet 84)
`delay(ms)`	Suspend for time
`Job.join()`	Wait for completion

COROUTINES VS THREADS

	COROUTINE	THREAD
Cost	Bytes of memory	~1 MB stack
Count	Hundreds of thousands	Thousands
Waiting	Suspends	Blocks
Cancellation	Cooperative, built in	interrupt()

⚠ COMMON MISTAKES

- Calling `runBlocking` on a UI or server request thread
- Blocking calls (JDBC, `Thread.sleep`) inside coroutines without `Dispatchers.IO`
- Calling suspend functions from normal functions (compile error)

✓ TIPS & BEST PRACTICES

- Mark I/O functions `suspend` and switch dispatchers inside them (sheet 85)
- Use `runBlocking` only in `main` and tests (prefer `runTest`)
- Name coroutines with `CoroutineName` for easier debugging

QUICK SUMMARY

- `suspend` — can pause
- `launch` — fire and forget
- `runBlocking` — bridge
- `delay` — non-blocking wait

Sheet 84

async & Structured Concurrency

async / await · awaitAll · coroutineScope · Job · parent-child cancellation · CoroutineScope

RESULTS AND LIFETIMES

`async` starts a coroutine that returns a value through a `Deferred`; `await()` gets it. **Structured concurrency** means every coroutine runs inside a **scope**: a parent waits for its children, and if one child fails, the others are cancelled. `coroutineScope { }` creates such a scope inside a suspend function.

KEY POINTS

- Start several `async`s, then await them to run in parallel
- `coroutineScope` returns only when all children finish
- A failed child cancels its siblings and the parent
- Never use `GlobalScope` in application code

PARALLEL CALLS IN A SCOPE

```
import kotlinx.coroutines.*

suspend fun loadDashboard(userId: Int): Dashboard = coroutineScope {
    val profile = async { api.profile(userId) }     // starts now
    val orders  = async { api.orders(userId) }      // runs in parallel
    val alerts  = async { api.alerts(userId) }
    Dashboard(profile.await(), orders.await(), alerts.await())
}   // returns after all three, or fails fast if any fails

suspend fun prices(skus: List<String>): List<Double> = coroutineScope {
    skus.map { sku -> async { api.price(sku) } }.awaitAll()
}

class ReportService {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default)

    fun start(): Job = scope.launch {
        while (isActive) { refresh(); delay(60_000) }
    }
    fun shutdown() = scope.cancel()              // cancels every child
}

fun main() = runBlocking {
    val time = kotlin.time.measureTime { println(loadDashboard(7)) }
    println("took $time")                        // ≈ slowest call, not the sum
}
```

CODE EXPLAINED

- → Three `async`s run at once; total time is the slowest call
- → If `orders` throws, `profile` and `alerts` are cancelled
- → `awaitAll` waits for a list of Deferreds
- → A class-owned scope lets you cancel all its work on shutdown

JAVA EQUIVALENT

- `CompletableFuture.allOf(a, b, c).join()` then `a.join()` …
- Structured concurrency preview: `StructuredTaskScope` (Java 21+ preview)
- ExecutorService futures do not cancel siblings automatically

IDIOMATIC KOTLIN

- ⇄ `GlobalScope.launch` → a scope tied to a lifecycle (or `coroutineScope`)
- ⇄ sequential awaits right after each async → start all, then await

BUILDERS & SCOPES

ITEM	RETURNS / DOES
`async { }`	Deferred<T>
`await()`	Result or exception
`awaitAll()`	List of results
`coroutineScope { }`	Waits for children; fail fast
`supervisorScope { }`	Children fail independently
`CoroutineScope(ctx)`	Long-lived scope
`scope.cancel()`	Cancel all children

⚠ COMMON MISTAKES

- `async { }.await()` immediately (no parallelism)
- Launching from `GlobalScope` and leaking work
- Forgetting to cancel class-level scopes

✓ TIPS & BEST PRACTICES

- Use `coroutineScope` inside suspend functions that fan out
- Use framework scopes: `viewModelScope`, Ktor `call` scope, Spring coroutines
- Use `supervisorScope` when one failure should not stop the rest (sheet 87)

QUICK SUMMARY

- `async` / `await` — results
- `coroutineScope` — structured
- child failure cancels siblings
- no `GlobalScope`

Dispatchers & Context

CHOOSING THREADS

A **dispatcher** decides which threads a coroutine runs on. `Dispatchers.Default` is for CPU work, `Dispatchers.IO` for blocking I/O such as JDBC and files, and `Dispatchers.Main` for UI threads. `withContext(Dispatchers.IO) { }` switches threads for a block and returns its result.

KEY POINTS

- A coroutine context holds Job, dispatcher, name and exception handler
- Contexts combine with `+`
- `withContext` suspends until the block finishes
- `Main` needs a UI module (Android, JavaFX, Swing)

SWITCHING DISPATCHERS

```
import kotlinx.coroutines.*

suspend fun loadUser(id: Int): User = withContext(Dispatchers.IO) {
    jdbc.queryForObject("SELECT * FROM users WHERE id = ?", id)  // blocks
}

suspend fun scoreAll(items: List<Item>): List<Int> =
    withContext(Dispatchers.Default) {
    items.map { heavyScore(it) }                                  // CPU work
}

fun main() = runBlocking(CoroutineName("main")) {
    launch(Dispatchers.Default + CoroutineName("worker")) {
        val name = coroutineContext[CoroutineName]
        println("$name on ${Thread.currentThread().name}")
        // CoroutineName(worker) on DefaultDispatcher-worker-1
    }

    val user = loadUser(1)          // caller does not care which thread
    println(user)

    val dbPool = Dispatchers.IO.limitedParallelism(10)   // at most 10 at once
    repeat(100) { launch(dbPool) { loadUser(it) } }
}

// Android / UI
// viewModelScope.launch {                    // Main by default
//     val data = repository.load()           // switches to IO inside
//     textView.text = data.title             // back on Main
// }
```

CODE EXPLAINED

- → `loadUser` hides the thread switch, so callers stay simple
- → CPU-heavy work goes to Default, which has one thread per core
- → Context elements like `CoroutineName` show up in logs
- → `limitedParallelism(10)` caps concurrent database calls

JAVA EQUIVALENT

- ☕ `ExecutorService` pools chosen by hand (`newFixedThreadPool`)
- ☕ `CompletableFuture.supplyAsync(task, ioExecutor)`
- ☕ UI: `SwingUtilities.invokeLater` / `Platform.runLater`

IDIOMATIC KOTLIN

- ⇄ blocking I/O on Default → `withContext(Dispatchers.IO)`
- ⇄ switching dispatchers at every call site → switch once inside the function

DISPATCHERS

DISPATCHER	USE FOR	THREADS
`Default`	`CPU work`	`One per core`
`IO`	`Blocking I/O`	`Up to 64 (shared with Default)`
`Main`	`UI updates`	`UI thread`
`Unconfined`	`Special cases/tests`	`Caller's thread`
`limitedParallelism(n)`	`Capped pool view`	`n at a time`

CONTEXT ELEMENTS

ELEMENT	HOLDS
`Job`	Lifecycle, cancellation
`CoroutineDispatcher`	Threads
`CoroutineName`	Debug name
`CoroutineExceptionHandler`	Uncaught errors

⚠ COMMON MISTAKES

- Running blocking JDBC on Default and starving CPU work
- Updating UI from IO threads
- Using `Dispatchers.Unconfined` in production code

✓ TIPS & BEST PRACTICES

- Make suspend functions main-safe: they switch dispatchers themselves
- Turn on `-Dkotlinx.coroutines.debug` to see coroutine names in thread names
- Use `limitedParallelism` to protect databases and APIs

QUICK SUMMARY

- Default — CPU
- IO — blocking I/O
- `withContext` — switch for a block
- context = Job + dispatcher + …

Cancellation & Timeouts

STOPPING WORK COOPERATIVELY

Coroutine cancellation is **cooperative**: `cancel()` marks a job cancelled, and the coroutine stops at its next suspension point or check of `isActive`. Suspending library functions like `delay` check automatically. `withTimeout` cancels a block after a time limit; `withTimeoutOrNull` returns null instead of throwing.

KEY POINTS

- Cancellation throws `CancellationException` at suspension points
- Tight CPU loops must call `ensureActive()` or `yield()`
- Always rethrow `CancellationException` if you catch broad exceptions
- Cleanup that suspends needs `withContext(NonCancellable)`

CANCELLING AND TIMING OUT

```
import kotlinx.coroutines.*
import kotlin.time.Duration.Companion.seconds

fun main() = runBlocking {
    val job = launch {
        repeat(1000) { i ->
            println("working $i")
            delay(100)                      // cancellation point
        }
    }
    delay(350)
    job.cancelAndJoin()                     // stops after ~4 iterations
    println("cancelled")

    val cpu = launch(Dispatchers.Default) {
        var n = 0L
        while (isActive) { n++ }            // checks cancellation
    }
    delay(50); cpu.cancelAndJoin()

    val price = withTimeoutOrNull(2.seconds) { slowPriceApi() }  // null if slow
    println(price ?: "using cached price")

}

suspend fun download(file: String) {
    try {
        transfer(file)
    } catch (e: Exception) {
        if (e is CancellationException) throw e   // never swallow cancellation
        log.error("failed", e)
    } finally {
        withContext(NonCancellable) { cleanupTempFiles() }   // may suspend
    }
}
```

CODE EXPLAINED

→ `cancelAndJoin` cancels and waits for the coroutine to finish

→ The CPU loop exits because it checks `isActive`

→ `withTimeoutOrNull` is the friendly way to bound slow calls

→ Rethrowing CancellationException keeps cancellation working

JAVA EQUIVALENT

- `future.cancel(true)` + checking `Thread.interrupted()`
- `future.get(2, TimeUnit.SECONDS)` throws TimeoutException
- `orTimeout(2, SECONDS)` on CompletableFuture

IDIOMATIC KOTLIN

⇄ `catch (e: Exception)` that eats cancellation → rethrow `CancellationException`

⇄ manual timers for timeouts → `withTimeout` / `withTimeoutOrNull`

API

CALL	DOES
`job.cancel()`	Request cancellation
`job.cancelAndJoin()`	Cancel + wait
`isActive`	Still running?
`ensureActive()`	Throw if cancelled
`yield()`	Check + let others run
`withTimeout(d) { }`	Throw on timeout
`withTimeoutOrNull(d) { }`	null on timeout
`NonCancellable`	Run cleanup anyway

⚠ COMMON MISTAKES

- Busy loops that never check `isActive` (cannot be cancelled)
- `runCatching` or `catch (e: Exception)` swallowing cancellation
- Suspending in `finally` without NonCancellable

✓ TIPS & BEST PRACTICES

- Use `Duration` values for timeouts (`2.seconds`)
- Wrap blocking calls with `runInterruptible` to make them cancellable
- Test cancellation paths with `runTest` and virtual time

QUICK SUMMARY

- cooperative cancellation
- `isActive` / `ensureActive()`
- `withTimeout` / `…OrNull`
- rethrow CancellationException

Exceptions in Coroutines

propagation in `launch` vs `async` · `CoroutineExceptionHandler` · `supervisorScope` · `SupervisorJob` · rethrowing cancellation

WHEN A COROUTINE FAILS

An exception in `launch` propagates **up** to the parent, cancelling it and its other children. An exception in `async` is stored and rethrown when you call `await()`, but it still cancels the parent scope. `supervisorScope` and `SupervisorJob` stop failures from spreading to siblings, and a `CoroutineExceptionHandler` logs uncaught ones.

KEY POINTS

- Wrap `await()` in try/catch to handle async failures
- Handlers only work on root coroutines (and `launch`)
- In a supervisor, each child must handle its own errors
- Cancellation exceptions are not treated as failures

HANDLING FAILURES

```
import kotlinx.coroutines.*

val handler = CoroutineExceptionHandler { ctx, e ->
    println("Uncaught in ${ctx[CoroutineName]}: ${e.message}")
}

fun main() = runBlocking {
    // try/catch around await
    val result = runCatching {
        coroutineScope { async { error("price API down") }.await() }
    }
    println(result.exceptionOrNull()?.message)        // price API down

    // supervisor: one failure does not cancel the others
    supervisorScope {
        val a = launch(handler + CoroutineName("a")) { error("A failed") }
        val b = launch { delay(100); println("B still finished") }
        joinAll(a, b)
    }
    // Uncaught in CoroutineName(a): A failed
    // B still finished

    // long-lived scope with a supervisor job and a handler
    val ctx = SupervisorJob() + Dispatchers.Default + handler
    val appScope = CoroutineScope(ctx)
    appScope.launch { error("task failed") }  // logged, scope survives
    delay(100)
}

suspend fun safeLoad(id: Int): User? = try {
    api.user(id)
} catch (e: CancellationException) {
    throw e                                           // keep cancellation
} catch (e: IOException) {
    null
}
```

CODE EXPLAINED

→ The failed async is caught where `await` is called

→ In `supervisorScope`, coroutine b finishes even though a failed

→ The handler logs the uncaught exception from coroutine a

→ A `SupervisorJob` keeps the app scope alive after one task fails

JAVA EQUIVALENT

- `future.exceptionally(e -> …)` / `handle(…)` on CompletableFuture
- Uncaught thread exceptions: `Thread.setDefaultUncaughtExceptionHandler`
- ExecutorService tasks swallow exceptions until `future.get()`

IDIOMATIC KOTLIN

⇄ try/catch around `launch` (does not catch child failures) → catch inside the coroutine or use a handler

⇄ one failing child killing a whole service scope → `SupervisorJob`

PROPAGATION

BUILDER	EXCEPTION GOES	CATCH WITH
`launch`	Up to parent	try inside / handler
`async`	Stored, rethrown on await	try around await
`coroutineScope`	Rethrown to caller	try around scope
`supervisorScope`	Child only	Child handles it

TOOLS

TOOL	EFFECT
`SupervisorJob()`	Children fail independently
`supervisorScope { }`	Same, as a block
`CoroutineExceptionHandler`	Last-resort logging
`runCatching / try`	Handle locally

⚠ COMMON MISTAKES

- Wrapping `launch { }` in try/catch and expecting to catch its failure
- Installing a handler on a child coroutine (ignored)
- Swallowing CancellationException in catch-all blocks

✓ TIPS & BEST PRACTICES

- Handle errors where you can act on them (inside the coroutine)
- Use SupervisorJob for long-lived service scopes
- Always log with a handler at the root of app scopes

QUICK SUMMARY

- launch — propagates up
- async — rethrows on await
- `supervisorScope` — isolate failures
- handler — log uncaught

Flow

ASYNCHRONOUS STREAMS

A `Flow` is a stream of values produced asynchronously, like a suspending Sequence. Flows are **cold**: the code inside `flow { }` runs each time someone calls `collect`. Operators such as `map`, `filter` and `take` transform the stream, `flowOn` changes where upstream code runs, and `catch` handles errors.

KEY POINTS

- `collect` is a suspend function: it runs in a coroutine
- Operators are applied lazily as values flow
- Use `flowOn(Dispatchers.IO)` instead of `withContext` inside `flow { }`
- Terminal operators: `collect`, `toList`, `first`, `reduce`

UNDER THE JVM

- Each operator wraps the upstream flow; `collect` drives the whole chain
- `flowOn` adds a channel boundary so upstream runs on another dispatcher
- Flows are sequential by default: one value at a time, no buffering

BUILDING AND COLLECTING FLOWS

```
import kotlinx.coroutines.*
import kotlinx.coroutines.flow.*

fun prices(symbol: String): Flow<Double> = flow {
    var p = 100.0
    while (true) {
        p += (-1..1).random()
        emit(p)                          // send a value
        delay(500)
    }
}

fun readLines(path: String): Flow<String> = flow {
    File(path).useLines { lines -> lines.forEach { emit(it) } }
}.flowOn(Dispatchers.IO)                 // upstream runs on IO

fun main() = runBlocking {
    prices("ACME")
        .map { "%.2f".format(it) }
        .take(3)                         // stop after 3 values
        .collect { println("price $it") }

    val errors = readLines("app.log")
        .filter { "ERROR" in it }
        .catch { e -> emit("read failed: ${e.message}") }
        .onCompletion { println("log scan done") }
        .toList()

    flowOf(1, 2, 3).collect { println(it) }
    (1..5).asFlow().map { it * it }.collect { print("$it ") }   // 1 4 9 16 25
}
```

CODE EXPLAINED

- → `flow { }` runs only when collected, and again for each collector
- → `take(3)` cancels the infinite producer after three values
- → `flowOn(Dispatchers.IO)` moves file reading off the caller's thread
- → `catch` handles upstream errors and can emit a fallback

JAVA EQUIVALENT

- Reactor `Flux` / RxJava `Observable` are the nearest equivalents
- `java.util.concurrent.Flow` (Reactive Streams interfaces)
- Streams are synchronous and cannot suspend

IDIOMATIC KOTLIN

- ⇄ callback listeners → `callbackFlow { }` / `Flow`
- ⇄ `withContext` inside `flow { }` (not allowed) → `flowOn`

BUILDERS

BUILDER	CREATES
`flow { emit(x) }`	Custom flow
`flowOf(a, b)`	Fixed values
`list.asFlow()`	From a collection
`callbackFlow { }`	From callbacks
`channelFlow { }`	Concurrent emitters

OPERATORS

OPERATOR	DOES
`map / filter / take`	Transform
`onEach { }`	Side effect
`flatMapLatest { }`	Switch to new inner flow
`debounce(ms)`	Wait for quiet
`combine / zip`	Merge flows
`flowOn(d)`	Upstream dispatcher
`catch { }`	Handle errors
`collect / toList / first`	Terminal

⚠ COMMON MISTAKES

- Expecting a flow to run without collecting it
- Calling `withContext` inside `flow { }` (IllegalStateException)
- Collecting an infinite flow without `take` or cancellation

✓ TIPS & BEST PRACTICES

- Return `Flow` from repositories for streams of updates
- Put `catch` after the operators whose errors you want to handle
- Use `stateIn` / `shareIn` to share one upstream (sheet 89)

QUICK SUMMARY

- `flow { emit() }` — cold stream
- `collect` — run it
- operators — map / filter / take
- `flowOn` — upstream thread

StateFlow, SharedFlow & Channels

HOT STREAMS AND MESSAGING

`StateFlow` holds a single **current value** and emits each change to collectors, which makes it ideal for UI or service state. `SharedFlow` broadcasts events to all collectors. Both are **hot**: they exist whether or not anyone collects. A `Channel` passes each value to exactly **one** receiver, like a queue between coroutines.

KEY POINTS

- `StateFlow` always has a value and skips equal repeats
- `SharedFlow` can replay recent values to new collectors
- `stateIn` / `shareIn` turn a cold flow into a hot one
- Channels must be closed when the producer finishes

STATE, EVENTS AND CHANNELS

```
import kotlinx.coroutines.*
import kotlinx.coroutines.channels.*
import kotlinx.coroutines.flow.*

class CartViewModel(scope: CoroutineScope) {
    private val _count = MutableStateFlow(0)        // state
    val count: StateFlow<Int> = _count.asStateFlow()

    private val _events = MutableSharedFlow<String>()   // one-off events
    val events: SharedFlow<String> = _events

    fun add() { _count.update { it + 1 } }
    suspend fun checkout() { _events.emit("Checked out ${_count.value} items") }

    val total: StateFlow<Double> = count
        .map { it * 9.99 }
        .stateIn(scope, SharingStarted.WhileSubscribed(5_000), 0.0)
}

fun main() = runBlocking {
    val vm = CartViewModel(this)
    val watcher = launch {
        vm.count.collect { println("count = $it") }    // count = 0
    }
    vm.add(); vm.add()                                 // count = 1, count = 2
    delay(100); watcher.cancel()

    val jobs = Channel<Int>(capacity = 10)             // work queue
    repeat(3) { w ->
        launch { for (j in jobs) println("worker $w got job $j") }
    }
    for (j in 1..6) jobs.send(j)
    jobs.close()                                       // workers' loops end

    val numbers = produce { for (i in 1..3) send(i * i) }
    println(numbers.toList())                          // [1, 4, 9]
    coroutineContext.cancelChildren()
}
```

CODE EXPLAINED

- → Private mutable state, public read-only `StateFlow`
- → `update { }` changes the value atomically
- → `stateIn` keeps `total` hot while someone is watching (plus 5 s)
- → Each job in the channel goes to only one worker

JAVA EQUIVALENT

- Observable state: `PropertyChangeSupport` or Reactor `Sinks.many()`
- Event bus libraries for SharedFlow-style broadcasts
- `BlockingQueue` between threads ≈ Channel (but blocking)

IDIOMATIC KOTLIN

- ⇄ `MutableStateFlow` exposed publicly → private `_state` + `asStateFlow()`
- ⇄ SharedFlow for one consumer queues → Channel

COMPARE

TYPE	HOT?	DELIVERS TO	HAS VALUE
`Flow`	Cold	Each collector separately	No
`StateFlow`	Hot	All collectors	Yes, current
`SharedFlow`	Hot	All collectors	Optional replay
`Channel`	Hot	One receiver per value	No

SHARINGSTARTED

POLICY	UPSTREAM RUNS
`Eagerly`	Immediately, forever
`Lazily`	From first collector, forever
`WhileSubscribed(ms)`	While collected (+ timeout)

⚠ COMMON MISTAKES

- Using StateFlow for one-time events (they replay as state)
- Forgetting to close channels, so receivers wait forever
- Exposing mutable flows publicly

✓ TIPS & BEST PRACTICES

- StateFlow for state, SharedFlow for events, Channel for work queues
- Use `WhileSubscribed(5_000)` for UI-driven flows
- Use `update { }` instead of read-modify-write on `value`

QUICK SUMMARY

- StateFlow — current state
- SharedFlow — broadcast events
- `stateIn` / `shareIn` — make hot
- Channel — one receiver per value

Annotations & Reflection

annotation class · @Target / @Retention · use-site targets @get: · ::class / KClass · kotlin-reflect · ::prop references

METADATA AND RUNTIME INSPECTION

Annotations attach metadata to code, and Kotlin's **use-site targets** (`@field:`, `@get:`, `@param:`) say exactly where a Java annotation lands on a property. **Reflection** uses `::class` for a KClass and `::name` for function and property references; full Kotlin reflection needs the `kotlin-reflect` library.

KEY POINTS

- `x::class` is a Kotlin `KClass`; `x::class.java` is a Java `Class`
- Without `kotlin-reflect`, basic `::class` and references still work
- Use-site targets matter for frameworks (JPA, Jackson, validation)
- Prefer compile-time tools (KSP, serialization) over runtime reflection

UNDER THE JVM

- A property produces a field, getter, setter and constructor parameter; use-site targets pick one
- Kotlin metadata is stored in a `@Metadata` annotation on every class
- `kotlin-reflect` reads that metadata (about 3 MB jar)

ANNOTATIONS AND REFLECTION

```
@Target(AnnotationTarget.CLASS, AnnotationTarget.PROPERTY)
@Retention(AnnotationRetention.RUNTIME)
annotation class Audited(val by: String = "system")

@Audited(by = "ops")
data class Invoice(
    @field:NotBlank val number: String,      // on the Java field
    @get:JsonProperty("amt") val amount: Double,   // on the getter
    @param:Min(0) val lines: Int             // on the constructor parameter
)

val k = Invoice::class
println(k.simpleName)                        // Invoice
println(k.java.name)                         // com.example.Invoice
println(k.findAnnotation<Audited>()?.by)     // ops (needs kotlin-reflect)

val inv = Invoice("A-1", 99.5, 3)
println(inv::class == Invoice::class)        // true

// with kotlin-reflect:
k.memberProperties.forEach { p -> println("${p.name} = ${p.get(inv)}") }
// amount = 99.5, lines = 3, number = A-1

val prop = Invoice::amount
println(prop.name)                           // amount
println(prop.get(inv))                       // 99.5
val ctor = ::Invoice
println(ctor("B-2", 10.0, 1))
```

CODE EXPLAINED

- → `@Target` and `@Retention` define where and how long the annotation exists
- → `@field:` puts `NotBlank` on the field, where validators look
- → `findAnnotation` and `memberProperties` come from kotlin-reflect
- → Property and constructor references work without extra libraries

JAVA EQUIVALENT

- `@interface Audited { String by() default "system"; }`
- `Invoice.class.getDeclaredFields()` for reflection
- Annotations go directly on fields, getters or parameters in Java

IDIOMATIC KOTLIN

- ⇄ annotations without use-site targets on properties → `@field:` / `@get:` as the framework needs
- ⇄ runtime reflection for codegen jobs → KSP or kotlinx.serialization

USE-SITE TARGETS

TARGET	APPLIES TO
`@field:`	Backing field
`@get:` / `@set:`	Getter / setter
`@param:`	Constructor parameter
`@property:`	Kotlin property (Kotlin-only)
`@file:`	The file class
`@receiver:`	Extension receiver

REFLECTION

EXPRESSION	GIVES
`x::class`	KClass<out T>
`T::class.java`	java.lang.Class
`::function`	KFunction
`T::property`	KProperty1
`::Class`	Constructor ref
`memberProperties`	Needs kotlin-reflect

⚠ COMMON MISTAKES

- Validation annotations on the constructor param instead of the field (ignored)
- Calling `memberProperties` without kotlin-reflect (error)
- Heavy reflection in hot paths

✓ TIPS & BEST PRACTICES

- Check your framework docs for the right use-site target
- Use `T::class.java` when a Java API wants `Class<T>`
- Prefer KSP for code generation

QUICK SUMMARY

- `annotation class` — define
- `@field:` / `@get:` — target
- `::class` — KClass
- kotlin-reflect — full reflection

Sheet 91

Type-Safe Builders & DSLs

lambdas with receiver · `@DslMarker` · `buildList` / `buildString` · HTML builder example · Gradle Kotlin DSL as a DSL

BUILDING YOUR OWN MINI-LANGUAGES

Kotlin DSLs combine **lambdas with receivers** (`T.() -> Unit`), extension functions, infix calls and trailing lambdas, so nested blocks read like configuration. The Gradle Kotlin DSL, Ktor routing and kotlinx.html are all built this way. `@DslMarker` stops inner blocks from accidentally calling outer builders.

KEY POINTS

- Inside `T.() -> Unit`, `this` is a T
- Each nesting level is a builder class
- `@DslMarker` restricts implicit receivers to the nearest one
- Standard builders: `buildList`, `buildString`, `buildMap`, `sequence`

A SMALL HTML DSL

```
@DslMarker annotation class HtmlDsl

@HtmlDsl
abstract class Tag(val name: String) {
    protected val children = mutableListOf<Tag>()
    private val attrs = mutableMapOf<String, String>()
    var text = ""
    fun attr(key: String, value: String) { attrs[key] = value }
    protected fun <T : Tag> add(tag: T, init: T.() -> Unit): T =
        tag.apply(init).also { children += it }
    override fun toString(): String {
        val a = attrs.entries.joinToString("") { " ${it.key}=\"${it.value}\"" }
        return "<$name$a>$text${children.joinToString("")}</$name>"
    }
}
class Html : Tag("html") { fun body(init: Body.() -> Unit) = add(Body(), init) }
class Body : Tag("body") {
    fun h1(t: String) = add(H1()) { text = t }
    fun a(href: String, t: String) = add(A()) { attr("href", href); text = t }
}
class H1 : Tag("h1"); class A : Tag("a")

fun html(init: Html.() -> Unit) = Html().apply(init)

val page = html {
    body {
        h1("Cheat Sheets")
        a("/kotlin", "Kotlin set")
    }
}
println(page)
// <html><body><h1>Cheat Sheets</h1>
//   <a href="/kotlin">Kotlin set</a></body></html>

val names = buildList { add("Ada"); if (true) add("Bob") }
```

CODE EXPLAINED

- → `html { }` takes an `Html.() -> Unit` lambda
- → Inside `body { }`, calls like `h1(...)` go to the Body builder
- → `@HtmlDsl` prevents calling `body { }` inside another `body`
- → The same pattern drives Gradle's `dependencies { }` blocks

JAVA EQUIVALENT

- Builder classes with chained methods: `Html.builder().body(…).build()`
- Java has no lambdas with receivers
- Fluent APIs (jOOQ, AssertJ) are Java's closest equivalent

IDIOMATIC KOTLIN

- ⇄ long fluent builder chains → a receiver-lambda DSL
- ⇄ deeply nested DSL scopes without `@DslMarker` → add a marker

DSL INGREDIENTS

FEATURE	ROLE
`T.() -> Unit`	Block with receiver
`Trailing lambda`	Block syntax
`Extension functions`	Add keywords
`infix functions`	Readable phrases
`Operator overloading`	+, [], invoke
`@DslMarker`	Scope control

REAL DSLS

DSL	EXAMPLE
`Gradle`	dependencies { implementation(…) }
`Ktor`	routing { get("/") { } }
`kotlinx.html`	body { h1 { +"Hi" } }
`Exposed`	Users.select { Users.age greater 18 }
`Kotest`	"x" shouldBe "x"

⚠ COMMON MISTAKES

- Calling outer-scope builders from inner blocks by accident
- Building DSLs where a simple function call would do
- Mutable builder state leaking after construction

✓ TIPS & BEST PRACTICES

- Start with `apply`-based builders before custom DSL classes
- Use `@DslMarker` on every builder in the hierarchy
- Keep DSLs for configuration and structured content

QUICK SUMMARY

- `T.() -> Unit` — receiver lambda
- nested builders
- `@DslMarker` — scope safety
- Gradle / Ktor — real DSLs

Ktor

`embeddedServer(Netty)` · `routing { get("/") }` · `call.respond` · `ContentNegotiation` · plugins · Ktor client

KOTLIN'S WEB FRAMEWORK

Ktor is JetBrains' coroutine-based framework for HTTP servers and clients. A server is built from a routing DSL and **plugins** (content negotiation, auth, logging, CORS). Handlers are suspend functions, so they can call other suspend APIs directly. The same library offers an HTTP client.

KEY POINTS

- Engines: Netty, CIO, Jetty, Tomcat
- Plugins are installed with `install(...)`
- JSON via `ContentNegotiation` + kotlinx.serialization
- Generate projects at start.ktor.io

A SMALL REST API AND CLIENT

```
import io.ktor.server.engine.*
import io.ktor.server.netty.*
import io.ktor.server.application.*
import io.ktor.server.routing.*
import io.ktor.server.response.*
import io.ktor.server.request.*
import io.ktor.server.plugins.contentnegotiation.*
import io.ktor.serialization.kotlinx.json.*
import io.ktor.http.*

@Serializable
data class Todo(val id: Int, val title: String, val done: Boolean = false)
val todos = mutableListOf(Todo(1, "Write Kotlin sheets"))

fun main() {
    embeddedServer(Netty, port = 8080) {
        install(ContentNegotiation) { json() }
        routing {
            get("/todos") { call.respond(todos) }
            get("/todos/{id}") {
                val id = call.parameters["id"]?.toIntOrNull()
                val todo = todos.find { it.id == id }
                    ?: return@get call.respond(HttpStatusCode.NotFound)
                call.respond(todo)
            }
            post("/todos") {
                val todo = call.receive<Todo>()
                todos += todo
                call.respond(HttpStatusCode.Created, todo)
            }
        }
    }.start(wait = true)
}

// Client
// val client = HttpClient(CIO) { install(ContentNegotiation) { json() } }
// val list: List<Todo> = client.get("http://localhost:8080/todos").body()
```

CODE EXPLAINED

- → `install(ContentNegotiation) { json() }` turns data classes into JSON
- → Route parameters come from `call.parameters`
- → `return@get call.respond(...)` ends the handler early
- → `call.receive<Todo>()` parses the request body

JAVA EQUIVALENT

- Spring Boot `@RestController` / `@GetMapping`
- Javalin or Micronaut for small services
- `HttpClient.newHttpClient().send(…)` for the client

IDIOMATIC KOTLIN

- ⇄ blocking JDBC in handlers → `withContext(Dispatchers.IO)` or a suspend driver
- ⇄ all routes in one file → `Route` extension functions per feature

COMMON PLUGINS

PLUGIN	DOES
`ContentNegotiation`	JSON / XML
`Authentication`	JWT, basic, OAuth
`StatusPages`	Exception → response
`CallLogging`	Request logs
`CORS`	Cross-origin rules
`Compression`	gzip

ROUTING

CALL	DOES
`get / post / put / delete`	Method routes
`route("/api") { }`	Group
`call.parameters["id"]`	Path param
`call.request.queryParameters`	Query
`call.receive<T>()`	Body
`call.respond(x)`	Response

⚠ COMMON MISTAKES

- Forgetting `install(ContentNegotiation)` and getting serialization errors
- Blocking calls on Ktor's event-loop threads
- Not handling bad path parameters (null IDs)

✓ TIPS & BEST PRACTICES

- Install StatusPages to map exceptions to HTTP codes
- Split routes with `fun Route.todoRoutes()` extensions
- Test with `testApplication { client.get(...) }`

QUICK SUMMARY

- `embeddedServer` — start
- `routing { get { } }` — routes
- `install(...)` — plugins
- suspend handlers

Sheet 93

Spring Boot with Kotlin

`kotlin("plugin.spring")` (all-open) · constructor injection · data classes as DTOs · `kotlin("plugin.jpa")` · coroutines in controllers · `runApplication<App>()`

KOTLIN AS A FIRST-CLASS SPRING LANGUAGE

Spring Boot officially supports Kotlin. The **spring** compiler plugin opens classes Spring needs to proxy (Kotlin classes are final), the **jpa** plugin adds no-arg constructors for entities, and constructor injection works through primary constructors with no `@Autowired`. Controllers can be `suspend` functions or return `Flow`.

KEY POINTS

- Start at start.spring.io with "Kotlin" selected
- `jackson-module-kotlin` handles data classes and defaults
- Spring's Kotlin extensions add `runApplication`, bean DSL and more
- See the Spring set for the full framework

KOTLIN SPRING BOOT APP

```
// build.gradle.kts plugins:
//   kotlin("jvm"), kotlin("plugin.spring"), kotlin("plugin.jpa"),
//   id("org.springframework.boot"), id("io.spring.dependency-management")

@SpringBootApplication
class ShopApplication

fun main(args: Array<String>) {
    runApplication<ShopApplication>(*args)
}

@Entity
class Product(
    @Id @GeneratedValue var id: Long? = null,
    var name: String,
    var price: BigDecimal
)

interface ProductRepository : JpaRepository<Product, Long>

data class ProductDto(val id: Long, val name: String, val price: BigDecimal)

@Service
class ProductService(private val repo: ProductRepository) {  // injected
    fun all() = repo.findAll().map { ProductDto(it.id!!, it.name, it.price) }
}

@RestController
@RequestMapping("/products")
class ProductController(private val service: ProductService) {
    @GetMapping fun list() = service.all()

    @GetMapping("/{id}")
    suspend fun one(@PathVariable id: Long): ProductDto =   // coroutine handler
        withContext(Dispatchers.IO) { service.all().first { it.id == id } }
}
```

CODE EXPLAINED

- → `runApplication<…>` is Spring's Kotlin entry point
- → The jpa plugin gives `Product` the no-arg constructor JPA needs
- → Constructor parameters are injected with no annotations
- → A `suspend` handler is supported with Spring WebFlux or MVC coroutine support

JAVA EQUIVALENT

- `SpringApplication.run(ShopApplication.class, args)`
- Lombok `@RequiredArgsConstructor` for constructor injection
- Records as DTOs: `record ProductDto(long id, String name, BigDecimal price)`

IDIOMATIC KOTLIN

- ⇄ `lateinit var` + `@Autowired` field injection → constructor injection
- ⇄ data classes as JPA entities → regular classes (entities need mutability and identity)

KOTLIN PLUGINS

PLUGIN	DOES
`plugin.spring (all-open)`	Opens @Component, @Configuration…
`plugin.jpa (no-arg)`	No-arg ctor for @Entity
`plugin.serialization`	kotlinx.serialization
`jackson-module-kotlin`	Jackson + data classes

KOTLIN EXTRAS

FEATURE	EXAMPLE
`runApplication`	runApplication<App>(*args)
`Bean DSL`	beans { bean<Service>() }
`Router DSL`	coRouter { GET("/") { … } }
`Coroutines`	suspend fun handlers
`Flow`	Return Flow<T> (WebFlux)

⚠ COMMON MISTAKES

- Missing the spring plugin, so `@Transactional` classes cannot be proxied
- Data classes as JPA entities (equals/hashCode on mutable IDs)
- Using `!!` on generated IDs before they are saved

✓ TIPS & BEST PRACTICES

- Let start.spring.io generate the Kotlin build
- Use data classes for DTOs, plain classes for entities
- Use `@ConfigurationProperties` with constructor-bound data classes

QUICK SUMMARY

- spring plugin — open classes
- jpa plugin — no-arg entities
- constructor injection
- suspend controllers

Multiplatform & Android Overview

KOTLIN BEYOND THE SERVER

Kotlin Multiplatform (KMP) shares code across Android, iOS, desktop, web and servers. Common code lives in `commonMain`; platform-specific parts use `expect` declarations in common code with `actual` implementations per platform. On **Android**, Kotlin is the default language and Jetpack Compose is the modern UI toolkit.

KEY POINTS

- Share business logic, networking and data first
- Ktor client, kotlinx.serialization and SQLDelight work in common code
- Compose Multiplatform shares UI (stable for Android, iOS, desktop)
- Jetpack Compose gets its own set in this repo

EXPECT / ACTUAL AND A COMPOSE PEEK

```
// commonMain
expect fun platformName(): String

class Greeting {
    fun greet() = "Hello from ${platformName()}"
}

expect class UuidGenerator() { fun next(): String }

// androidMain / jvmMain
actual fun platformName() = "JVM ${System.getProperty("java.version")}"
actual class UuidGenerator {
    actual fun next() = java.util.UUID.randomUUID().toString()
}

// iosMain
// actual fun platformName() = UIDevice.currentDevice.systemName()
// actual class UuidGenerator { actual fun next() = NSUUID().UUIDString() }

// Android UI with Jetpack Compose (also Compose Multiplatform)
@Composable
fun Counter() {
    var count by remember { mutableStateOf(0) }
    Column(modifier = Modifier.padding(16.dp)) {
        Text("Clicked $count times")
        Button(onClick = { count++ }) { Text("Click me") }
    }
}
```

CODE EXPLAINED

- → `expect` declares an API that each platform must provide
- → `actual` supplies the platform version
- → Common code calls `platformName()` without knowing the platform
- → `@Composable` functions describe UI from state

JAVA EQUIVALENT

- Java runs on Android but new Android APIs and samples are Kotlin-first
- Cross-platform Java: limited (J2ObjC for iOS)
- Android Views + XML layouts were the pre-Compose approach

IDIOMATIC KOTLIN

- ⇄ duplicate logic in Android and iOS apps → share it with KMP
- ⇄ XML layouts in new Android screens → Jetpack Compose

SOURCE SETS

SOURCE SET	CODE FOR
`commonMain`	All platforms
`commonTest`	Shared tests
`androidMain`	Android
`jvmMain`	Desktop / server
`iosMain`	iOS (all iOS targets)
`jsMain / wasmJsMain`	Web

KMP LIBRARIES

LIBRARY	FOR
`Ktor client`	HTTP
`kotlinx.serialization`	JSON
`kotlinx.coroutines`	Async
`kotlinx-datetime`	Dates
`SQLDelight / Room KMP`	Database
`Koin`	Dependency injection

⚠ COMMON MISTAKES

- Using `java.*` APIs in `commonMain`
- Trying to share everything (including platform UI) on day one
- Mismatched `expect` / `actual` signatures

✓ TIPS & BEST PRACTICES

- Start with a shared data/networking module
- Use interfaces + DI instead of `expect` / `actual` when possible
- See the planned Jetpack Compose set for Android UI

QUICK SUMMARY

- `commonMain` — shared code
- `expect` / `actual` — platform hooks
- Compose Multiplatform — shared UI
- Android — Kotlin + Compose

Sheet 95

Quick Reference — Syntax, Collections & Coroutines

declarations · null-safety operators · when · classes · scope functions · Java ↔ Kotlin syntax · collection builders · transformations · aggregation · coroutine builders · dispatchers · Flow operators

DECLARATIONS (08–15, 36)

SYNTAX	MEANING
`val x = 1`	Read-only
`var x = 1`	Mutable
`const val X = 1`	Compile-time
`lateinit var x: T`	Set later
`val x by lazy { }`	Lazy
`fun f(a: Int): Int`	Function
`fun f(a: Int) = a * 2`	Expression body
`fun T.ext()`	Extension

NULL SAFETY (12, 24)

SYNTAX	MEANING
`T?`	Nullable
`x?.f()`	Safe call
`x ?: d`	Default
`x ?: return`	Early exit
`x!!`	Assert non-null
`x?.let { }`	If not null
`x as? T`	Safe cast
`mapNotNull { }`	Drop nulls

CONTROL FLOW (26–31)

SYNTAX	MEANING
`val v = if (c) a else b`	If expression
`when (x) { 1 -> … }`	Match
`in 1..9 ->`	Range branch
`is T ->`	Type branch
`for (i in 0..<n)`	Loop
`10 downTo 0 step 2`	Progression
`break@outer`	Labeled jump

CLASSES (56–67)

SYNTAX	MEANING
`class A(val x: Int)`	Primary ctor
`open class / override`	Inheritance
`data class`	equals, copy…
`sealed interface`	Closed hierarchy
`enum class`	Constants
`object`	Singleton
`companion object`	Class-level
`@JvmInline value class`	Wrapper type
`class B(a: A) : I by a`	Delegation

SCOPE FUNCTIONS (44)

FUNCTION	OBJECT / RETURNS
`let`	it / result
`run`	this / result
`with(x)`	this / result
`apply`	this / object
`also`	it / object
`takeIf`	it / object or null

JAVA ↔ KOTLIN

JAVA	KOTLIN
`final String s`	val s: String
`a ? b : c`	if (a) b else c
`switch`	when
`static`	companion / top-level
`a.equals(b)`	a == b
`a == b (refs)`	a === b
`instanceof`	is
`(T) x`	x as T
`void`	Unit

BUILDERS (46–51)

CALL	CREATES
`listOf / mutableListOf`	Lists
`setOf / mutableSetOf`	Sets
`mapOf(k to v)`	Map
`buildList { }`	Built list
`List(n) { }`	Generated
`intArrayOf / IntArray(n)`	Arrays
`emptyList()`	Empty
`asSequence()`	Lazy (55)

TRANSFORM (52)

CALL	DOES
`map / mapNotNull`	Transform
`filter / filterNot`	Subset
`flatMap / flatten`	Flatten
`zip`	Pairs
`associateBy / associate`	To map
`chunked / windowed`	Groups
`partition`	Two lists
`distinct`	Unique

AGGREGATE & SORT (53, 54)

CALL	DOES
`sumOf / count / average`	Totals
`maxByOrNull / minOf`	Extremes
`fold / reduce`	Custom
`groupBy`	Map of lists
`groupingBy { }.eachCount()`	Counts
`sortedBy / sortedWith`	Sort
`compareBy { }.thenBy { }`	Comparator
`find / any / all / none`	Search

COROUTINE BUILDERS (83–87)

CALL	DOES
`suspend fun`	Can pause
`launch { }`	Job
`async { } / await()`	Result
`coroutineScope { }`	Structured block
`supervisorScope { }`	Isolated failures
`runBlocking { }`	Bridge (main/tests)
`withContext(d) { }`	Switch dispatcher
`withTimeoutOrNull(t) { }`	Time limit

DISPATCHERS & CONTROL (85, 86)

ITEM	USE
`Dispatchers.Default`	CPU
`Dispatchers.IO`	Blocking I/O
`Dispatchers.Main`	UI
`job.cancel()`	Cancel
`isActive / ensureActive()`	Check
`SupervisorJob()`	Independent children
`CoroutineExceptionHandler`	Log uncaught

FLOW (88, 89)

CALL	DOES
`flow { emit(x) }`	Cold stream
`collect { }`	Terminal
`map / filter / take`	Transform
`flowOn(d)`	Upstream thread
`catch { }`	Errors
`MutableStateFlow(v)`	State
`MutableSharedFlow()`	Events
`stateIn / shareIn`	Make hot
`Channel()`	Queue